



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA EN
TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

TierraViva

Monitorización sostenible y riego inteligente en cultivos desatendidos

Autora: Lucía Tejedor de la Cruz

Tutor: Francisco Javier Simó Reigadas

(Co-Tutora): Ana Zazo Moratalla

Curso académico 2025-2026



Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-SA International License (Creative Commons Attribution-ShareAlike 4.0). Usted es libre de *(a) compartir*: copiar y redistribuir el material en cualquier medio o formato para cualquier propósito, incluso comercialmente; y *(b) adaptar*: remezclar, transformar y construir a partir del material para cualquier propósito, incluso comercialmente. La licenciente no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciente.
- *Compartir igual.* Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.

No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

Agradecimientos

Unas bonitas palabras...

Quizás un segundo párrafo esté bien. No te olvides de nadie.

Un tercero tampoco viene mal para contar alguna anécdota...

¿Alguien más? Aunque sean *actores* secundarios.

Un quinto párrafo como colofón.

*A mi abuela, mi mejor amiga;
nuestra Toyi*

Madrid, 26 de mayo de 2026
Lucía Tejedor

Resumen

Escribe aquí el resumen del trabajo. Un primer párrafo para dar contexto sobre la temática que rodea al trabajo.

Un segundo párrafo concretando el contexto del problema abordado.

En el tercer párrafo, comenta cómo has resuelto la problemática descrita en el anterior párrafo.

Por último, en este cuarto párrafo, describe cómo han ido los experimentos.

Acrónimos

IoT *Internet of Things*

IA/AI *Inteligencia Artificial/Artificial Intelligence*

API *Application Programming Interface*

LPWAN *Low Power Wide Area Networks*

LTE *Long Term Evolution*

HTTP *Hypertext Transfer Protocol*

IP *Internet Protocol*

MQTT *Message Queuing Telemetry Transport*

CoAP *Constrained Application Protocol*

GPIO *General Purpose Input/Output*

SIM *Subscriber Identity Module*

Índice general

Prólogo	xiii
1. Introducción	1
1.1. Contexto general: agricultura, agua y digitalización	1
1.2. Motivación del proyecto	3
1.3. Planteamiento del problema	4
1.4. Alcance y limitaciones	6
1.5. Organización de la memoria	8
2. Estado del arte y fundamentos tecnológicos	9
2.1. Arquitecturas IoT	9
2.2. Agricultura de precisión y riego inteligente	12
2.3. IoT aplicado a la agricultura	14
2.4. Redes de sensores y nodos de campo	15
2.5. Controladores y plataformas embebidas para IoT	17
2.6. Comunicaciones LPWAN y conectividad en entornos rurales	19
2.7. Plataformas cloud para IoT	22
2.8. Protocolos de aplicación en IoT: HTTP, MQTT y CoAP	23
2.9. Automatización, reglas e inteligencia artificial en riego	25
3. Métodos, materiales y decisiones de diseño	28
3.1. Requisitos funcionales del sistema	28
3.2. Requisitos no funcionales	32
3.3. Especificaciones temporales y operativas	35
3.4. Metodología de trabajo	38
3.5. Arquitectura general planteada	42
3.6. Elección de variables y sensores	43
3.7. Elección de plataforma de control local y supervisión	47
3.8. Selección de solución de comunicaciones para estaciones remotas	50

3.9. Elección de plataforma cloud: ThingSpeak	54
3.10. Elección de arquitectura software de supervisión en Raspberry Pi	58
3.11. Materiales empleados	62
3.12. Riesgos técnicos y medidas de mitigación	65
4. Arquitectura final de TierraViva	69
4.1. Visión general de la arquitectura	69
4.2. Estaciones de campo A y B	71
4.3. Arquitectura de comunicaciones LTE/HTTP	73
4.4. ThingSpeak como capa intermedia	75
4.4.1. Canales de estación	76
4.4.2. Descartes del control descendente desde la nube	78
4.5. Raspberry Pi como sistema de supervisión	78
4.6. Decisión local de riego y eventos	80
4.7. Modo seguro	82
5. Implementación hardware y firmware de estaciones	85
5.1. Preparación de Arduino UNO	85
5.2. Integración de sensores	87
5.3. Firmware de adquisición	91
5.4. Integración del A7670E	96
5.5. Envío de datos a ThingSpeak	104
5.6. Control de relé, LED y bomba de agua	109
5.7. Alimentación de estación	113
A. Materiales y componentes empleados	117
A.1. Referencias comerciales de compra	119
B. Esquemas eléctricos y conexiones	120
C. Pruebas iniciales de sensores	121
D. Validación del módulo LTE A7670E-LASA	122
E. Fragmentos de código relevantes	123
F. Estructura de canales ThingSpeak y codificación de comandos	124
Bibliografía	125

Índice de figuras

2.1. Esquema genérico de una arquitectura IoT por capas.	12
2.2. Estructura general de un nodo de campo en una red IoT agrícola. . . .	17
3.1. Arquitectura general planteada para TierraViva.	43
4.1. Arquitectura final de TierraViva con decisión local en las estaciones. . .	70
4.2. Canales ThingSpeak creados para las estaciones de TierraViva.	76
B.1. Esquema de conexión de una estación de campo de TierraViva. En la fase de prueba se representa un LED como carga de verificación; en el montaje final se sustituye por la bomba de agua de 12 V controlada mediante relé.	120

Listado de códigos

5.1. Lectura agrupada de sensores en la estación	93
5.2. Comprobación del periodo de espera entre riegos	93
5.3. Decisión local de riego en la estación	94
5.4. Activación temporizada del relé de riego	94
5.5. Estructura de la petición HTTP de telemetría	95
5.6. Configuración serie del A7670E en Arduino UNO	99
5.7. Detección de velocidad serie del módulo LTE	99
5.8. Comprobación de registro LTE del A7670E	101
5.9. Petición HTTP GET mediante el A7670E	103
5.10. Construcción de la URL de actualización de ThingSpeak	106
5.11. Envío de telemetría a ThingSpeak	107
5.12. Definición del pin y estados del relé	110
5.13. Inicialización segura del relé	110
5.14. Activación temporizada del relé	111
5.15. Desactivación segura del relé	111
5.16. Supervisión del estado del relé	112

Listado de ecuaciones

Índice de tablas

2.1. Comparación general entre sensores analógicos y digitales	16
2.2. Comparación general entre microcontroladores y computadores de placa reducida	18
2.3. Comparación general de tecnologías de conectividad para IoT en campo	21
2.4. Comparación general entre HTTP, MQTT y CoAP en sistemas IoT . .	25
3.1. Requisitos funcionales del sistema TierraViva	30
3.2. Requisitos no funcionales del sistema TierraViva	33
3.3. Especificaciones temporales y operativas iniciales del sistema	36
3.4. Fases principales de la metodología de trabajo	39
3.5. Variables seleccionadas para la monitorización de TierraViva	44
3.6. Sensores seleccionados para las estaciones de campo	46
3.7. Comparación funcional entre Arduino UNO y Raspberry Pi 5 en TierraViva	49
3.8. Comparación entre LoRa/LoRaWAN y LTE para TierraViva	53
3.9. Uso previsto de ThingSpeak dentro de TierraViva	56
3.10. Bloques principales de la arquitectura software de supervisión en Raspberry Pi	61
3.11. Resumen de materiales principales empleados en TierraViva	63
3.12. Riesgos técnicos principales y medidas de mitigación	66
4.1. Configuración lógica de las estaciones de campo	71
4.2. Bloques internos de una estación de campo	72
4.3. Fases de comunicación LTE/HTTP en una estación de campo	74
4.4. Asignación de campos en los canales ThingSpeak de estación	77
4.5. Funciones principales de la Raspberry Pi dentro de TierraViva	79
4.6. Códigos de estado o evento publicados por las estaciones	82
4.7. Situaciones contempladas por el modo seguro de la estación	83
5.1. Preparación inicial del Arduino UNO compatible	86

5.2. Conexiones de los sensores integrados en la estación de campo	88
5.3. Pruebas iniciales de validación de sensores	89
5.4. Variables obtenidas en la prueba conjunta de sensores	90
5.5. Bloques principales del firmware de adquisición y decisión local	92
5.6. Campos de telemetría enviados por el firmware de estación	95
5.7. Validación previa del módulo A7670E-LASA	97
5.8. Conexiones principales entre Arduino UNO y el A7670E-LASA	98
5.9. Comandos AT utilizados en la inicialización de red del A7670E	100
5.10. Secuencia HTTP utilizada con el módulo A7670E	102
5.11. Mapeo de variables del firmware a campos de ThingSpeak	105
5.12. Errores considerados durante el envío de telemetría a ThingSpeak	108
5.13. Conexiones principales del bloque de actuación mediante relé	109
5.14. Comportamiento del bloque de actuación ante distintas condiciones	112
5.15. Distribución de alimentación en la estación de campo	114
5.16. Condensadores de estabilización en la alimentación del A7670E	115
A.1. Resumen de componentes empleados en TierraViva	118
A.2. Referencias comerciales de sensores empleados	119

Prólogo

La idea de TierraViva nace de una situación cercana. En mi familia hay un pequeño huerto en el pueblo y, especialmente en épocas de calor, aparece siempre el mismo problema: no es posible revisarlo todas las semanas. A veces las plantas reciben menos agua de la necesaria y otras veces ocurre lo contrario, porque el riego depende de visitas puntuales, de la intuición y de la disponibilidad de cada momento.

La frase que terminó de dar forma a la idea surgió casi en broma. Un día, hablando sobre estos problemas, mi tío me dijo: “¿*No vas a ser ingeniera? Pues arrégrame el huerto*”. Aunque fue un comentario informal, planteaba un problema muy real: cómo conocer el estado de un cultivo cuando no se está físicamente allí y cómo actuar sobre el riego sin depender siempre de una revisión manual.

A partir de esa necesidad surgió la posibilidad de desarrollar un sistema capaz de medir el entorno, enviar información, visualizar el estado del cultivo y actuar sobre el riego de forma controlada. Lo que inicialmente parecía una solución sencilla fue tomando forma como un problema de ingeniería más completo, en el que intervienen sensorización, comunicaciones, sistemas embebidos, almacenamiento de datos, visualización y control.

Capítulo 1

Introducción

Este capítulo sitúa TierraViva dentro del contexto de la digitalización agrícola y del riego inteligente. A partir de una necesidad práctica (supervisar y actuar sobre el riego sin depender siempre de una revisión presencial) se introduce el problema técnico, la motivación del trabajo, su alcance y la organización de la memoria.

1.1. Contexto general: agricultura, agua y digitalización

La agricultura es una de las actividades en las que el uso eficiente de los recursos tiene un impacto más directo, tanto desde el punto de vista económico como medioambiental. Entre todos los recursos implicados, el agua ocupa un papel especialmente importante, ya que condiciona de forma directa el desarrollo de los cultivos y, al mismo tiempo, es un recurso limitado que debe gestionarse con cuidado. Según AQUASTAT, el uso agrícola supone alrededor del 69% de las extracciones mundiales de agua [Food and Agriculture Organization of the United Nations, 2021]. En este contexto, el riego no puede entenderse únicamente como una tarea rutinaria, sino como una decisión técnica que depende de múltiples factores: el estado del suelo, las condiciones ambientales, el tipo de cultivo, la época del año y la disponibilidad de agua.

Tradicionalmente, muchos sistemas de riego se han basado en criterios fijos, como horarios predefinidos o estimaciones manuales del estado del terreno. Aunque este enfoque puede ser suficiente en determinadas situaciones, existen limitaciones claras. Un riego programado de forma rígida puede aportar agua cuando no es necesario o, por el contrario, no hacerlo en momentos en los que el cultivo sí lo necesita. Esta falta de adaptación al estado real del entorno puede traducirse en un consumo innecesario de agua, un peor aprovechamiento de los recursos y una menor capacidad de respuesta ante cambios de temperatura, humedad o exposición solar.

Para dar solución a estas limitaciones, la digitalización del entorno agrícola

proporciona las herramientas necesarias para desarrollar sistemas informados y adaptativos. Mediante el despliegue de redes de sensores y equipos de comunicación, es posible adquirir datos en tiempo real y observar el estado del entorno de forma continua. Este planteamiento se relaciona directamente con la agricultura de precisión, cuyo objetivo no es únicamente automatizar tareas, sino utilizar información del propio entorno para mejorar la eficiencia, reducir desperdicios y facilitar la gestión de los recursos.

En este tipo de sistemas, la monitorización ambiental desempeña un papel fundamental. Medir variables como la humedad del suelo, la temperatura, la humedad relativa, la presión atmosférica o la luminosidad permite disponer de una visión más completa del estado de una zona de cultivo, y no todas estas variables tienen el mismo peso en la decisión de riego. Por ejemplo, la humedad del suelo puede indicar de forma directa la necesidad de aportar agua, mientras que la temperatura o la luz ambiental ayudan a interpretar las condiciones en las que se está produciendo la evaporación o el déficit de agua.

La automatización del riego aparece, por tanto, como una evolución natural de la monitorización. Un sistema que únicamente muestra datos puede ser útil para el agricultor, pero sigue dependiendo de una intervención manual para actuar. En cambio, un sistema capaz de medir, analizar y accionar un elemento de riego permite cerrar el ciclo completo: observar el entorno, interpretar su estado y ejecutar una acción cuando se cumplen determinadas condiciones. Esta capacidad resulta especialmente interesante en zonas donde la supervisión constante no siempre es posible o donde se quiere reducir la dependencia de decisiones manuales repetitivas.

Además, en muchos escenarios agrícolas o rurales existen limitaciones de infraestructura que condicionan el diseño de cualquier solución tecnológica. No siempre se dispone de conexión WiFi estable, alimentación eléctrica cercana o cobertura de red garantizada. Por ello, un sistema de riego inteligente no debe plantearse únicamente desde el punto de vista del software o de la sensorización, sino como una integración completa de hardware, comunicaciones, energía, almacenamiento de datos y actuación física sobre el riego. La dificultad real no está solo en medir una variable, sino en conseguir que todos estos elementos funcionen de forma coordinada y fiable.

Dentro de este contexto surge TierraViva, un proyecto orientado al diseño de un prototipo IoT para monitorización y control de riego. El objetivo principal no es construir un producto agrícola industrial cerrado, sino desarrollar una solución funcional que permita comprobar cómo diferentes tecnologías pueden integrarse para resolver un problema real. El proyecto parte de una idea sencilla, medir el estado

del entorno y actuar sobre el riego cuando sea necesario, pero implica varias capas técnicas: adquisición de datos mediante sensores, comunicación remota, publicación de telemetría, almacenamiento, visualización, lógica local de decisión en las estaciones y actuación física sobre el sistema de riego.

Por tanto, este trabajo se sitúa en la intersección de varias disciplinas: agricultura de precisión, Internet de las Cosas, comunicaciones y sistemas embebidos. Esta combinación permite abordar un problema práctico desde una perspectiva propia de la ingeniería: analizar necesidades, estudiar alternativas, justificar decisiones, implementar un prototipo y validar su funcionamiento mediante pruebas. La finalidad última es demostrar que una arquitectura bien diseñada puede ayudar a mejorar la gestión del riego, aportando información objetiva y capacidad de actuación automática o supervisada.

1.2. Motivación del proyecto

La motivación de este proyecto surge de la dificultad de gestionar correctamente el riego en pequeños cultivos o huertos que no pueden supervisarse de forma continua. En estos escenarios, especialmente en épocas de más calor, pueden aparecer problemas tanto por falta de agua como por exceso de riego. Aunque pueda parecer una situación sencilla, refleja una necesidad habitual: cuidar correctamente un terreno cuando no se está físicamente allí de forma continua.

Detrás de esa necesidad existe un problema técnico real. No se trata solo de regar más o menos, sino de saber cuándo hace falta regar, en qué condiciones se encuentra el terreno y si sería posible actuar sin depender siempre de una revisión manual. Por tanto, el interés del proyecto no está únicamente en automatizar una tarea, sino en diseñar un sistema capaz de observar el entorno, interpretar su estado y actuar de forma controlada.

La idea inicial puede entenderse como una solución de aviso remoto, pero al analizar el caso con más detalle aparecen cuestiones que hacen el problema más interesante: la cobertura en entornos rurales, la alimentación del sistema, la necesidad de funcionamiento desatendido, la fiabilidad de las comunicaciones y la posibilidad de controlar físicamente el riego.

La evolución natural de esta idea conduce hacia un sistema más completo de monitorización y control. El proyecto no consiste únicamente en conectar sensores, sino en diseñar una arquitectura capaz de integrar distintas áreas de la ingeniería de forma coherente. Esta integración es una de las partes más relevantes del trabajo, porque

permite transformar una necesidad cotidiana en un problema técnico con suficiente profundidad para un Trabajo Fin de Grado.

Otro aspecto importante en la motivación es que TierraViva permite aplicar conocimientos que normalmente se estudian por separado. En el proyecto intervienen electrónica, programación embebida, comunicaciones, bases de datos, desarrollo web, visualización de información y control de actuadores. Esta combinación hace que el trabajo sea más exigente, pero también más representativo de un sistema real, donde el valor no está solo en que cada parte funcione de manera aislada, sino en conseguir que todas trabajen coordinadamente.

Además, el proyecto tiene una componente práctica y sostenible que lo hace especialmente atractivo. El objetivo no es desarrollar tecnología por sí misma, sino utilizarla para responder a una necesidad concreta: gestionar mejor el riego y reducir decisiones basadas únicamente en intuición, costumbre u horarios fijos. En este sentido, TierraViva conecta la ingeniería con una aplicación tangible, cercana y fácil de entender, pero con suficiente profundidad técnica como para justificar un desarrollo completo.

Por todo ello, este TFG no se plantea simplemente como un ejercicio académico más, sino como una oportunidad para construir una solución con valor real. La idea nace de una necesidad cercana, pero se plantea con una visión modular y escalable, de forma que pueda servir como base para futuras ampliaciones, como nuevas estaciones de campo, autonomía energética mediante panel solar o mecanismos predictivos basados en datos históricos.

1.3. Planteamiento del problema

El reto principal de este TFG es transformar una situación tradicionalmente manual en un sistema monitorizado, trazable y parcialmente automatizado.

El primer aspecto del problema es la **adquisición de datos**. Para que el sistema pueda tomar decisiones con cierto criterio, necesita disponer de medidas representativas del entorno. La humedad del suelo es la variable más directamente relacionada con la necesidad de riego, pero no es la única información útil, ya que otras variables como la temperatura, la humedad relativa o la luminosidad permiten contextualizar mejor el estado de la zona monitorizada. El sistema debe ser capaz de leer estas magnitudes de forma periódica y convertirlas en datos utilizables, evitando que las lecturas queden como valores aislados sin relación con el funcionamiento global del sistema.

El segundo aspecto es la **transmisión de la información**. En un sistema de riego desatendido, las estaciones de campo no pueden depender de una supervisión constante

ni de una conexión física permanente a un ordenador. Los datos deben poder enviarse desde la zona donde se encuentran los sensores hasta un punto donde puedan ser procesados, almacenados y visualizados. Esta comunicación debe ser suficientemente fiable para que el agricultor o el usuario pueda conocer el estado del sistema, pero también debe tener en cuenta las limitaciones propias de un entorno rural, donde no siempre existe una infraestructura de red estable.

Una vez recibidos los datos, el siguiente problema es su **almacenamiento y tratamiento**. Mostrar una lectura puntual puede servir para una comprobación rápida, pero no permite analizar la evolución del sistema. Para que el prototipo tenga utilidad real, es necesario registrar las medidas recibidas, asociarlas a una estación concreta y conservar un histórico. De esta forma, se puede comprobar cómo varían las condiciones del entorno a lo largo del tiempo, detectar comportamientos anómalos y justificar las decisiones de riego tomadas por el sistema.

La **visualización** también forma parte del problema. Un sistema técnico puede estar funcionando correctamente de manera interna, pero si el agricultor o el usuario no puede interpretar su estado de forma clara, su utilidad queda limitada. Por ello, es necesario disponer de una interfaz que permita consultar las últimas lecturas, observar la evolución de las variables principales y conocer el estado del riego. Esta interfaz debe presentar la información de forma comprensible, sin exigir que el usuario tenga que interpretar directamente datos brutos, mensajes de consola o registros internos difíciles de entender para alguien no experto.

Otro punto fundamental es la **toma de decisiones**. El sistema debe ser capaz de determinar cuándo conviene actuar sobre el riego a partir de las medidas disponibles. En el alcance de este TFG, esta decisión se plantea mediante reglas verificables, como umbrales, condiciones de seguridad y tiempos máximos de actuación. Este enfoque resulta adecuado para un prototipo, ya que permite justificar de forma clara por qué se activa o se detiene el riego, sin depender de modelos predictivos complejos que requerirían una cantidad elevada de datos históricos para entrenarse y validarse correctamente.

El último elemento del problema es la **actuación física**. Un sistema que solo mide y muestra información se queda en una solución de monitorización. Para avanzar hacia un sistema de riego inteligente, es necesario cerrar el ciclo mediante el control de un actuador. Esto introduce requisitos adicionales de seguridad, ya que una lectura incorrecta, una regla mal configurada, una pérdida de comunicación o un fallo eléctrico no deben provocar que la bomba permanezca activa indefinidamente. Por tanto, el diseño debe contemplar un comportamiento seguro por defecto dentro de la propia

estación de campo, priorizando la desactivación de la bomba ante situaciones de error. En conjunto, el problema planteado puede entenderse como un problema de integración de subsistemas. TierraViva debe coordinar sensores, estación de campo, comunicación remota, plataforma de publicación de datos, sistema de supervisión, interfaz de usuario y actuador. Cada una de estas partes tiene una función concreta, pero el valor del proyecto aparece cuando todas trabajan de forma conjunta. Por esta razón, el desarrollo no se limita a seleccionar componentes, sino que exige definir una arquitectura modular, validar cada bloque por separado y comprobar posteriormente el funcionamiento del sistema completo.

Además, el sistema se diseña pensando en su posible ampliación. Aunque el prototipo pueda validarse inicialmente con una estación, la arquitectura permite incorporar nuevas estaciones de campo, nuevas variables medidas o nuevos mecanismos de decisión. Esta modularidad es importante para que el trabajo no quede como una solución cerrada y difícil de extender, sino como una base sobre la que puedan plantearse mejoras futuras, como autonomía energética, despliegue multizona o mecanismos predictivos basados en datos históricos.

1.4. Alcance y limitaciones

El objetivo no es desarrollar un producto comercial cerrado ni una solución industrial certificada, sino construir una arquitectura completa que permita comprobar, de forma experimental, cómo diferentes tecnologías pueden integrarse para resolver un problema real. Por tanto, el trabajo se evalúa principalmente desde el punto de vista de la integración tecnológica, el funcionamiento del sistema y la validación mediante pruebas.

El prototipo debe ser capaz de medir variables relevantes del entorno, transmitir la información obtenida, registrar o consultar los datos recibidos, permitir su visualización y actuar sobre el riego bajo determinadas condiciones evaluadas localmente en cada estación. De esta forma, el proyecto no se limita a una prueba aislada de sensores ni a una aplicación de visualización, sino que busca cerrar el ciclo completo de monitorización, decisión y actuación. Esta integración constituye el núcleo del trabajo y permite valorar el sistema como una solución IoT aplicada a un caso práctico.

No obstante, es importante delimitar claramente el alcance para evitar interpretar el proyecto como un sistema agrícola final preparado para su instalación directa en cualquier explotación. El prototipo desarrollado no pretende sustituir a soluciones profesionales de riego industrial, ni contempla certificaciones eléctricas, hidráulicas o de

comunicaciones. Tampoco se aborda el diseño mecánico definitivo de una caja estanca, la fabricación de una placa electrónica propia ni la optimización completa del consumo energético para funcionamiento durante largos periodos sin mantenimiento.

Otra limitación relevante es la validación agronómica. Aunque el sistema está orientado al riego y se basa en variables relacionadas con el estado del entorno, no se realizará un ensayo prolongado durante varios meses sobre un cultivo real. Una validación de ese tipo requeriría controlar factores externos como el tipo de suelo, la especie cultivada, la climatología, la frecuencia de riego, la evolución del crecimiento de las plantas y la comparación con métodos tradicionales. En este TFG, la validación se centrará en comprobar que el prototipo mide, comunica, registra, visualiza y actúa correctamente, dejando los ensayos agronómicos de larga duración como una posible línea futura.

También se establece como limitación el uso de inteligencia artificial. Aunque el proyecto se plantea dentro de un contexto en el que la analítica de datos y los mecanismos predictivos pueden aportar mucho valor, no tendría rigor presentar como inteligencia artificial una lógica basada únicamente en reglas o umbrales. Para implementar modelos predictivos fiables sería necesario disponer de un histórico amplio de datos, recogido durante suficiente tiempo y bajo condiciones representativas. Por ello, en este trabajo la toma de decisiones se planteará de forma verificable mediante reglas, dejando la inteligencia artificial como una ampliación futura si se dispone de datos suficientes para entrenar, validar y justificar un modelo.

El sistema se diseñará con una visión modular, de modo que pueda ampliarse posteriormente con nuevas estaciones, nuevas variables medidas, mejoras en la autonomía energética o mecanismos más avanzados de análisis. Sin embargo, dentro del alcance de esta memoria, la prioridad será demostrar que la arquitectura propuesta funciona de extremo a extremo: desde la adquisición de datos en campo hasta la visualización y actuación sobre el riego. Esta delimitación permite mantener un equilibrio entre ambición y viabilidad, evitando prometer funcionalidades que no puedan ser implementadas y validadas con suficiente solidez durante el desarrollo del TFG.

En resumen, el alcance del proyecto es el de un prototipo funcional, documentado y validado, orientado a demostrar la viabilidad técnica de un sistema IoT para riego inteligente. Sus limitaciones no se consideran una debilidad del trabajo, sino una forma de definir con precisión qué se ha implementado, qué se ha probado y qué queda abierto para futuras fases de desarrollo.

1.5. Organización de la memoria

La memoria se organiza de forma progresiva, comenzando por la introducción del problema y avanzando después hacia los fundamentos técnicos, las decisiones de diseño, la implementación y la validación del sistema.

En el Capítulo 1 se presenta el contexto del proyecto, la motivación, el planteamiento del problema y el alcance del trabajo. El Capítulo 2 recoge el estado del arte y los fundamentos tecnológicos necesarios para comprender la solución propuesta, incluyendo arquitecturas IoT, agricultura de precisión, comunicaciones, plataformas cloud y automatización del riego.

El Capítulo 3 describe los métodos, materiales y decisiones de diseño, definiendo los requisitos del sistema, la metodología seguida, la arquitectura general planteada y la justificación de las principales elecciones tecnológicas. A continuación, el Capítulo 4 presenta la arquitectura final de TierraViva y explica cómo se integran las estaciones de campo, la plataforma cloud, la Raspberry Pi como sistema de supervisión y los mecanismos de control.

Los Capítulos 5 y 6 se centran en la implementación del prototipo. El primero aborda la parte hardware y firmware de las estaciones, incluyendo la lectura de sensores, la publicación de telemetría y la decisión local de riego, mientras que el segundo describe la implementación software de supervisión en Raspberry Pi y nube. Posteriormente, el Capítulo 7 recoge la validación y las pruebas realizadas, y el Capítulo 8 analiza los resultados obtenidos y las limitaciones detectadas.

Finalmente, el Capítulo 9 presenta las conclusiones del trabajo, el grado de cumplimiento del alcance planteado, las competencias aplicadas y adquiridas, y las posibles líneas de trabajo futuro.

Capítulo 2

Estado del arte y fundamentos tecnológicos

Este capítulo recoge los fundamentos tecnológicos que sirven de base para el diseño de TierraViva. La revisión se centra en las áreas que han condicionado decisiones reales del proyecto: arquitecturas IoT, agricultura de precisión, redes de sensores, plataformas embebidas, conectividad en campo, plataformas cloud y automatización aplicada al riego.

El capítulo no busca valorar cada tecnología de forma aislada, sino explicar qué aporta cada una dentro de una arquitectura de riego inteligente. Por eso se presta atención tanto a sus ventajas como a sus restricciones prácticas: consumo, cobertura, facilidad de integración, dependencia de infraestructura y capacidad de validación dentro del alcance del TFG.

La revisión se orienta, por tanto, a justificar una arquitectura distribuida. TierraViva no se plantea como un sistema en el que un equipo central decide cada actuación, sino como un conjunto de estaciones de campo autónomas que miden, aplican reglas locales y publican su estado. El equipo de supervisión queda reservado para tareas de consulta, histórico y visualización.

2.1. Arquitecturas IoT

El concepto de Internet de las Cosas, o *Internet of Things* (IoT), hace referencia a la interconexión de objetos físicos o virtuales capaces de identificarse, comunicarse y participar en servicios digitales. La recomendación ITU-T Y.2060 define IoT como una infraestructura global para la sociedad de la información que permite ofrecer servicios avanzados mediante la interconexión de cosas, tanto físicas como virtuales, a través de tecnologías de información y comunicación interoperables [International Telecommunication Union, 2012]. Esta definición es amplia, pero resulta adecuada para este trabajo porque sitúa IoT como una arquitectura completa: adquisición de datos, comunicación, procesamiento, almacenamiento y actuación.

Desde un punto de vista práctico, un sistema IoT conecta el mundo físico con el mundo digital. En el extremo de campo se encuentran dispositivos capaces de medir variables, detectar eventos o modificar el estado de un sistema. Esa información se transmite después hacia otros elementos, donde puede almacenarse, procesarse, visualizarse o utilizarse para apoyar decisiones. Cuando el sistema incorpora control, la actuación física puede resolverse de distintas formas: mediante decisiones centralizadas, mediante control remoto o mediante lógica local en el propio nodo de campo. Esta última alternativa resulta especialmente relevante cuando se busca que el sistema mantenga un comportamiento seguro aunque la conectividad externa no esté disponible.

En la práctica, IoT debe entenderse como una integración de componentes, no como una tecnología aislada. Sensores, actuadores, microcontroladores, redes de comunicación, plataformas cloud, bases de datos, servicios backend, APIs y dashboards deben encajar entre sí. En un proyecto de ingeniería, el funcionamiento individual de cada bloque no garantiza el funcionamiento del sistema completo. Un diseño puede tener sensores correctos o una interfaz clara, pero fallar si la comunicación, el almacenamiento o la lógica de actuación no se han planteado de forma coherente.

Uno de los elementos básicos de una arquitectura IoT es el nodo o *end-device*. Este nodo se sitúa cerca del fenómeno que se quiere observar o controlar y normalmente integra sensores, una unidad de procesamiento, un sistema de comunicación y una fuente de alimentación. En algunos casos solo mide y envía información; en otros también ejecuta decisiones sencillas o controla actuadores. Estos dispositivos suelen estar limitados en memoria, capacidad de cálculo, consumo energético o ancho de banda disponible, por lo que sus restricciones deben considerarse desde el diseño inicial [Al-Fuqaha et al., 2015].

Los sensores convierten una magnitud física en una señal eléctrica o digital interpretable por el sistema. Los actuadores realizan la operación inversa: reciben una señal de control y modifican el estado del mundo físico. Esta relación permite diferenciar entre monitorización y control. La monitorización observa el entorno y presenta información; el control utiliza esa información para ejecutar acciones. La segunda opción introduce más responsabilidad técnica, ya que una señal de control incorrecta, un dato erróneo o una pérdida de comunicación pueden tener consecuencias físicas.

En muchas arquitecturas IoT aparece también una pasarela o *gateway*. Su función es actuar como intermediario entre los dispositivos de campo y el resto del sistema. Puede traducir protocolos, agregar datos, filtrar información, ejecutar procesamiento

local o proporcionar salida hacia una red de mayor alcance. En arquitecturas más completas, esta pasarela puede realizar funciones de *edge computing*, reduciendo latencia y manteniendo ciertas funciones aunque la conexión externa no esté disponible [IEEE, 2020].

Relacionado con este planteamiento aparece el concepto de control local o computación en el borde. En lugar de desplazar todas las decisiones hacia un servidor, una nube o un equipo supervisor, parte de la lógica puede ejecutarse cerca del fenómeno físico que se quiere controlar. En sistemas con actuadores, esta decisión tiene una implicación importante: el nodo de campo puede aplicar reglas de seguridad, limitar tiempos de activación y reaccionar ante fallos sin depender de una comunicación continua con el exterior.

En aplicaciones de riego, esta aproximación resulta especialmente adecuada. La humedad del suelo no varía a escala de milisegundos y las reglas iniciales de actuación pueden implementarse de forma verificable en el microcontrolador de la estación. De este modo, la plataforma remota y el sistema de supervisión pueden centrarse en registrar, visualizar y analizar el comportamiento del sistema, mientras que la estación conserva la responsabilidad inmediata sobre la bomba o el actuador de riego.

La nube o plataforma cloud constituye otra capa habitual. Su función es proporcionar almacenamiento, disponibilidad remota, APIs de acceso y, en muchos casos, herramientas de visualización o análisis. Esta capa facilita consultar los datos desde distintos lugares, pero introduce dependencias que deben valorarse: conectividad, latencia, límites de uso, disponibilidad del servicio y seguridad de la información.

El backend es la parte software encargada de recibir, transformar, validar y servir los datos. Puede ejecutarse en una plataforma cloud, en un servidor o en un equipo local con capacidad suficiente. Normalmente se comunica con una base de datos, donde se guardan medidas y eventos. Este almacenamiento permite analizar la evolución temporal del sistema, generar históricos y revisar decisiones tomadas a partir de datos previos.

La visualización suele realizarse mediante dashboards, aplicaciones web o aplicaciones móviles. Su objetivo es convertir datos técnicos en información comprensible. En un sistema de monitorización y control, la interfaz debe permitir consultar el estado actual, identificar fallos, observar tendencias y detectar situaciones que requieren atención, sin obligar al usuario a interpretar registros internos o tramas de comunicación.

Las arquitecturas IoT suelen representarse por capas. Una división sencilla distingue entre capa física, formada por sensores y actuadores; capa de conectividad; capa de

procesamiento y almacenamiento; y capa de aplicación. Modelos más completos, como el propuesto por el IoT World Forum, añaden niveles como computación en el borde, acumulación de datos, abstracción y procesos de negocio [Hanes et al., 2017].

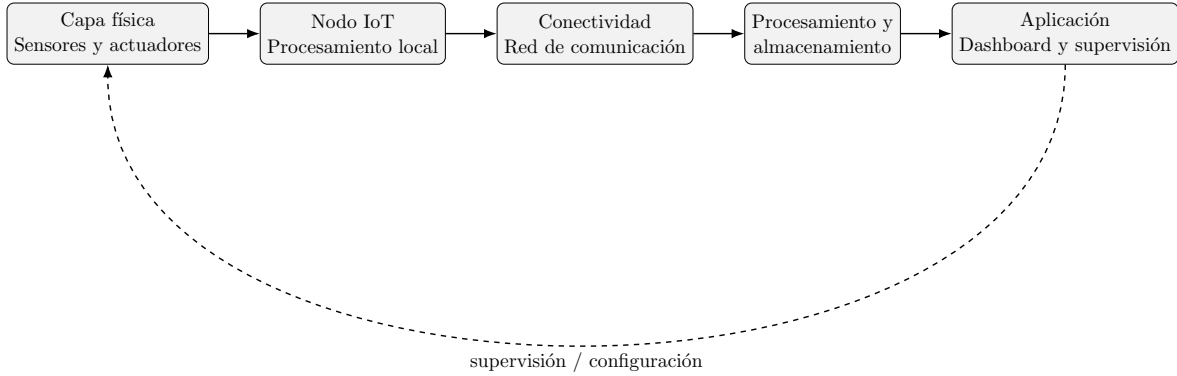


Figura 2.1: Esquema genérico de una arquitectura IoT por capas.

La Figura 2.1 resume esta estructura. En el caso de un sistema de riego, la arquitectura debe cubrir el ciclo completo: medir en campo, comunicar, almacenar, visualizar y actuar de forma segura. No obstante, la actuación no tiene por qué depender siempre de un control remoto; en diseños distribuidos puede ejecutarse localmente en la estación, utilizando la comunicación externa para supervisión, histórico y diagnóstico.

2.2. Agricultura de precisión y riego inteligente

La agricultura de precisión es una estrategia de gestión agrícola que utiliza datos para tomar decisiones más ajustadas a las condiciones reales de cada explotación, parcela o zona de cultivo. Según la International Society of Precision Agriculture, este enfoque se basa en recopilar, procesar y analizar datos temporales, espaciales e individuales, combinándolos con otra información para apoyar decisiones de manejo orientadas a mejorar la eficiencia en el uso de recursos, la productividad, la calidad, la rentabilidad y la sostenibilidad de la producción agrícola [International Society of Precision Agriculture, 2024].

El interés de este enfoque está en la variabilidad del entorno agrícola. Una misma parcela puede presentar diferencias de humedad, exposición solar, tipo de suelo, pendiente o desarrollo de las plantas. Si todas las zonas se tratan de la misma manera, parte de esa variabilidad queda oculta. La agricultura de precisión intenta reducir esa simplificación utilizando datos para adaptar mejor las decisiones.

El agua es una variable crítica dentro de este contexto. La agricultura representa

una parte muy relevante de las extracciones de agua dulce a escala global; AQUASTAT indica que aproximadamente el 69 % de las extracciones corresponden al uso agrícola [Food and Agriculture Organization of the United Nations, 2021]. Una mejora en la gestión del riego puede afectar tanto al rendimiento del cultivo como a la sostenibilidad del recurso. Regar de forma inadecuada puede provocar déficit hídrico, exceso de agua, pérdida de eficiencia y deterioro del cultivo.

El riego manual depende de la presencia y experiencia del usuario. Puede funcionar en cultivos pequeños o con supervisión frecuente, pero presenta limitaciones cuando el terreno no puede revisarse de forma continua. El riego temporizado reduce la intervención manual programando horarios o duraciones concretas, aunque sigue siendo rígido: puede mantener el mismo patrón aunque cambien la temperatura, la humedad, la lluvia o la exposición solar.

El riego basado en datos introduce una lógica diferente. La decisión de regar se apoya en medidas del entorno, como humedad del suelo, temperatura ambiente, humedad relativa, radiación o luminosidad, previsión meteorológica, tipo de cultivo o histórico de riego. No todas las variables tienen el mismo peso, pero su combinación permite interpretar mejor el estado del sistema agrícola.

La monitorización ambiental permite pasar de una decisión basada en intuición a una decisión basada en información observable. Además, el registro de datos a lo largo del tiempo permite estudiar tendencias, detectar cambios bruscos y comprobar si una acción de riego ha tenido el efecto esperado.

Cuando la monitorización se conecta con un mecanismo de actuación, el sistema pasa a formar parte del control del proceso. En ese punto aparecen requisitos adicionales: reglas claras, límites de seguridad y comportamiento seguro ante fallo. En riego, una actuación incorrecta puede traducirse en pérdida de agua, daño al cultivo o funcionamiento indebido del actuador encargado de impulsar o permitir el paso del agua.

Los sistemas de riego inteligente basados en IoT suelen combinar nodos de medida, comunicaciones, almacenamiento de datos, procesamiento y actuación. Boursianis et al. presentan en *IEEE Sensors Journal* una plataforma de riego inteligente para agricultura de precisión basada en IoT, planteada como un sistema formado por varios subsistemas coordinados [Boursianis et al., 2021]. Esta línea de trabajo es relevante porque desplaza el foco desde el sensor aislado hacia la arquitectura completa.

En TierraViva este enfoque se concreta en una decisión sencilla pero importante: no basta con medir humedad o temperatura, la estación debe poder convertir esas lecturas en una actuación local limitada y segura. Por eso la eficiencia del sistema no depende

solo de la calidad de las medidas, sino también de cómo se transforma una lectura en una acción de riego.

2.3. IoT aplicado a la agricultura

La aplicación de IoT en agricultura responde a la necesidad de observar el entorno agrícola de forma continua, distribuida y objetiva. La agricultura de precisión define el enfoque de gestión basado en datos; IoT aporta la infraestructura que permite capturarlos, transmitirlos, almacenarlos y utilizarlos.

En estos sistemas, las estaciones de monitorización se sitúan en campo y recogen información del entorno mediante sensores. Su función no es obtener una medida aislada, sino generar observaciones periódicas de una zona de cultivo. Según la arquitectura, una estación puede limitarse a monitorizar o incorporar también capacidad de actuación.

Las variables medidas dependen del objetivo agrícola. En riego suelen combinarse variables del suelo y variables ambientales. Los sensores de suelo aportan información más directa sobre la disponibilidad de agua en la zona de raíces; los sensores ambientales ayudan a contextualizar esa lectura mediante temperatura, humedad relativa, presión, radiación solar o luminosidad [García et al., 2020].

El dato agrícola necesita contexto. Una lectura de humedad del suelo no se interpreta igual según la hora del día, la temperatura, la exposición solar o el histórico de riego anterior. Por este motivo, estos sistemas suelen organizar las medidas como series temporales asociadas a una estación y a un instante concreto.

El despliegue en campo introduce restricciones que no aparecen con la misma intensidad en un laboratorio. Las estaciones pueden estar expuestas a lluvia, polvo, humedad, cambios de temperatura, radiación solar o manipulaciones accidentales. También deben funcionar durante periodos prolongados sin intervención directa. Protección física, mantenimiento, alimentación, conectividad y recuperación ante fallos pasan a ser parte del problema técnico.

La alimentación condiciona el diseño desde el principio. Las estaciones agrícolas no siempre tienen una toma eléctrica cercana, por lo que pueden requerir baterías, paneles solares o modos de bajo consumo. Esta decisión afecta a la frecuencia de muestreo, al tipo de comunicación, a la potencia de transmisión y a la disponibilidad de sensores o actuadores.

La conectividad plantea una restricción similar. En muchos entornos rurales no se puede asumir una red WiFi estable ni una cobertura móvil uniforme. Las estaciones

pueden estar separadas por decenas, cientos o miles de metros, dependiendo de la parcela. Esto obliga a estudiar tecnologías adecuadas para dispositivos distribuidos, bajo consumo y cobertura amplia [Soussi et al., 2024].

La robustez es un requisito técnico de primer nivel. Un sistema agrícola puede tomar decisiones que afectan al consumo de agua o al estado de un cultivo. Debe contemplar pérdidas de comunicación, lecturas erróneas, reinicios inesperados, fallos de alimentación o indisponibilidad temporal de la plataforma de datos. Si incorpora actuación, también debe definir estados seguros ante situaciones anómalas.

La literatura sobre agricultura inteligente coincide en que el valor de IoT está en la integración entre medida, comunicación, procesamiento y toma de decisiones [Shafi et al., 2019, Soussi et al., 2024]. En la misma línea, los trabajos sobre riego inteligente resaltan que la eficiencia depende tanto de la calidad de las medidas como de la estrategia de control y de la fiabilidad de la arquitectura completa [Abioye et al., 2020].

Desde esta perspectiva, el reto no está simplemente en añadir sensores a un cultivo. El reto está en construir una arquitectura capaz de identificar estaciones, registrar históricos diferenciados, crecer de forma ordenada y mantener un comportamiento seguro cuando alguna parte del sistema falle.

2.4. Redes de sensores y nodos de campo

Una red de sensores es un conjunto de dispositivos distribuidos que capturan información del entorno y la transmiten hacia un sistema capaz de procesarla, almacenarla o utilizarla para tomar decisiones. En IoT, estas redes permiten observar fenómenos físicos de forma continua y en distintos puntos del espacio.

El elemento básico es el nodo de campo o *end-node*. Se sitúa cerca del entorno que se quiere medir o controlar y normalmente incorpora una unidad de procesamiento, uno o varios sensores, un sistema de comunicación y una fuente de alimentación [Hanes et al., 2017]. En agricultura, estos nodos pueden instalarse en distintas zonas de cultivo para recoger datos periódicos y enviarlos a un sistema de supervisión.

Dentro de una red conviene distinguir entre nodos sensores y nodos actuadores. Un nodo actuador modifica físicamente el estado de un sistema, por ejemplo activando una bomba, cerrando un circuito mediante un relé o accionando otro elemento físico de riego. En muchos sistemas reales ambas funciones conviven en un mismo nodo.

Los sensores transforman una magnitud física en una señal interpretable por el sistema. En agricultura son habituales variables como humedad del suelo, temperatura

ambiente, humedad relativa, luminosidad, presión atmosférica, radiación solar, caudal o parámetros químicos del suelo. En riego, las variables relacionadas con el agua disponible y las condiciones ambientales tienen prioridad [García et al., 2020].

Una diferencia importante es el tipo de salida del sensor. Los sensores analógicos generan una señal continua, normalmente una tensión o corriente proporcional a la magnitud medida, y requieren conversión analógico-digital. Los sensores digitales entregan directamente la medida mediante un protocolo de comunicación. Los primeros suelen ser sencillos y económicos, pero más sensibles al ruido y a la alimentación; los segundos facilitan la lectura, aunque dependen de protocolos, direcciones y librerías concretas.

Aspecto	Sensores analógicos	Sensores digitales
Salida	Señal continua proporcional a la magnitud medida.	Dato digital entregado por un bus o protocolo.
Lectura	Requieren conversión analógico-digital.	Se leen mediante comunicación digital.
Ventajas	Simples, económicos y fáciles de conectar.	Lectura más directa y mayor integración interna.
Limitaciones	Sensibles a ruido, alimentación y cableado.	Dependencia de protocolos, direcciones y librerías.
Uso habitual	Medidas sencillas con salida proporcional.	Sensores con varias magnitudes o calibración interna.

Tabla 2.1: Comparación general entre sensores analógicos y digitales

La Tabla 2.1 recoge esta comparación. En ambos casos, una lectura de sensor no debe asumirse como una medida absoluta. Muchos sensores necesitan calibración para que sus valores sean útiles en un contexto concreto. En agricultura esto es evidente en variables como la humedad del suelo, cuya respuesta puede depender del sustrato, la compactación, la salinidad, la temperatura o la forma de instalación.

También deben contemplarse errores de medida. Un sensor puede devolver valores erróneos por ruido eléctrico, mala conexión, humedad, degradación, fallos de alimentación o condiciones fuera de rango. Si esas medidas se utilizan para tomar decisiones automáticas, el sistema debe diferenciar entre datos válidos, datos dudosos y ausencia de datos. Esta distinción evita que una actuación dependa de una única lectura incoherente.

Los nodos de campo suelen tener recursos limitados. Pueden funcionar con batería, comunicarse mediante enlaces de bajo ancho de banda y operar en zonas de cobertura irregular. El diseño debe decidir qué datos se envían, con qué frecuencia, a qué distancia

y cuántos nodos formarán parte de la red [Al-Fuqaha et al., 2015].

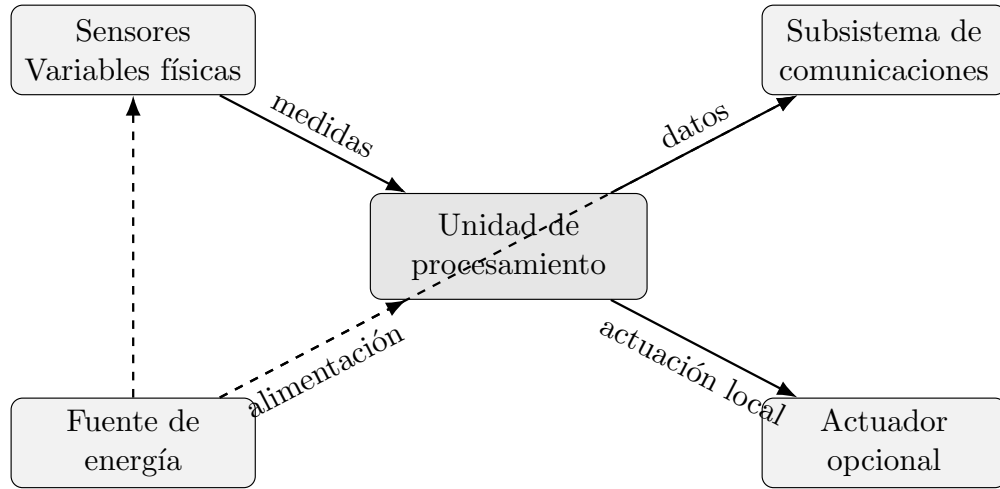


Figura 2.2: Estructura general de un nodo de campo en una red IoT agrícola.

La Figura 2.2 muestra los bloques habituales de un nodo de campo. En una aplicación agrícola real, la fiabilidad depende del conjunto formado por sensor, nodo, alimentación, comunicación, protección física y tratamiento de datos [Soussi et al., 2024]. Cuando el nodo incorpora un actuador, también debe asumir reglas mínimas de actuación segura, ya que es el elemento que se encuentra físicamente conectado al sistema de riego.

2.5. Controladores y plataformas embebidas para IoT

En una arquitectura IoT, los sensores y actuadores necesitan una plataforma capaz de leer señales, ejecutar lógica de control y comunicarse con el resto del sistema. Esta función puede recaer en microcontroladores, computadores de placa reducida o dispositivos embebidos específicos. La elección afecta al consumo energético, potencia de cálculo, interfaces disponibles, coste, facilidad de programación y capacidad de integración.

Los microcontroladores son una opción habitual para nodos de campo. Son dispositivos de bajo coste y consumo reducido, pensados para ejecutar tareas concretas de forma repetitiva: leer sensores, activar salidas digitales, comunicarse mediante buses sencillos o enviar datos a través de un módulo externo. Plataformas como Arduino, ESP32, STM32 u otras equivalentes son frecuentes en prototipos IoT porque permiten trabajar directamente con entradas y salidas físicas, temporización, comunicaciones serie y buses como I2C o SPI [Al-Fuqaha et al., 2015].

Arduino es una familia muy extendida en educación y prototipado. Su principal ventaja es el entorno accesible, la abundante documentación y la compatibilidad con muchos sensores y módulos. ESP32 incorpora WiFi y Bluetooth, además de mayor potencia de cálculo. Las familias STM32 ofrecen más control técnico y una gama amplia de prestaciones, aunque suelen exigir mayor conocimiento del entorno de desarrollo y de la configuración de periféricos.

Los computadores de placa reducida, como Raspberry Pi u otras plataformas similares, cubren otro tipo de necesidades. Permiten ejecutar un sistema operativo completo, levantar servicios, gestionar bases de datos, servir aplicaciones web o coordinar varios procesos software. Su mayor potencia y flexibilidad los hace adecuados para almacenamiento, visualización, supervisión, análisis histórico o funciones de gateway. A cambio, consumen más, arrancan más lentamente y no son la opción natural para depender de ellos como único punto de decisión en una actuación física crítica.

Esta separación ayuda a distinguir dos roles dentro de una arquitectura IoT. Un nodo de campo suele priorizar bajo consumo, robustez, lectura directa de sensores, control de salidas y funcionamiento desatendido. Un sistema de supervisión puede necesitar procesamiento, almacenamiento local, conectividad con servicios externos, backend e interfaz de usuario.

Aspecto	Microcontroladores	Computadores de placa reducida
Función	Sensado, decisión local sencilla, control de salidas y tareas embebidas.	Supervisión, almacenamiento, servicios y visualización.
Consumo	Bajo o moderado, según diseño.	Superior, por sistema operativo y servicios.
Cálculo	Suficiente para adquisición y lógica sencilla.	Adecuado para procesos software más complejos.
Interfaces	GPIO, ADC, PWM, UART, I2C o SPI.	GPIO, USB, red, almacenamiento y sistema operativo.
Uso típico	Nodo de campo autónomo o dispositivo final.	Gateway, servidor local o sistema de supervisión.

Tabla 2.2: Comparación general entre microcontroladores y computadores de placa reducida

La Tabla 2.2 resume esta diferencia. En un sistema IoT agrícola, una arquitectura híbrida puede aprovechar la eficiencia del microcontrolador en campo para la medida y actuación local, y la capacidad de procesamiento de un computador de placa reducida para supervisión, almacenamiento y visualización [Hanes et al., 2017,

Soussi et al., 2024].

2.6. Comunicaciones LPWAN y conectividad en entornos rurales

La conectividad condiciona la viabilidad de cualquier sistema IoT desplegado en campo. En aplicaciones agrícolas, los nodos pueden estar separados de la vivienda, del laboratorio o del sistema de supervisión, y no siempre existe una infraestructura de red disponible. No se puede asumir WiFi estable, alimentación cercana ni cobertura móvil suficiente en todos los puntos.

El tráfico típico de estos sistemas suele ser reducido. Normalmente se envían medidas pequeñas cada cierto intervalo: humedad, temperatura, estado de un actuador, tensión de batería o avisos de error. La prioridad no es el caudal, sino la cobertura, el consumo, la disponibilidad y la fiabilidad del enlace.

Las redes LPWAN, del inglés *Low Power Wide Area Networks*, se diseñan para ese tipo de escenario. Proporcionan comunicaciones de largo alcance y bajo consumo para dispositivos distribuidos que envían pequeñas cantidades de datos. A cambio, aceptan bajo caudal, mayor latencia o mensajes poco frecuentes [Raza et al., 2017]. Este compromiso encaja con muchas aplicaciones agrícolas, donde las variables del entorno no requieren actualización continua a escala de milisegundos.

WiFi y Bluetooth son tecnologías conocidas, pero su utilidad en campo abierto es limitada. WiFi ofrece alta capacidad, aunque depende de una red local y su alcance es reducido. Bluetooth y BLE tienen bajo consumo y pueden ser útiles para configuración cercana, pero no conectan por sí solos estaciones separadas por cientos de metros o varios kilómetros.

Entre las tecnologías de largo alcance destaca LoRa, una modulación de capa física desarrollada por Semtech y basada en espectro ensanchado mediante chirps. Permite alcanzar largas distancias con potencias reducidas, a costa de tasas de transmisión bajas. Al aumentar el factor de ensanchamiento mejora la sensibilidad, pero crece el tiempo de transmisión y disminuye el caudal disponible [Semtech Corporation, 2015, Augustin et al., 2016].

LoRa y LoRaWAN no son equivalentes. LoRa hace referencia a la modulación radio. LoRaWAN define una arquitectura de red sobre LoRa, con dispositivos finales, pasarelas, servidor de red y servidor de aplicación [LoRa Alliance, 2020]. Usar LoRaWAN implica trabajar con una infraestructura completa, no solo con módulos de radio.

LoRaWAN resulta atractivo en entornos rurales por su alcance, bajo consumo y uso de bandas no licenciadas. Sus restricciones también son claras: bajo caudal, límites regulatorios de uso del canal, dependencia de gateways, configuración más compleja y limitaciones en tráfico descendente. Adelantado et al. analizan precisamente estos límites de capacidad, latencia y escalabilidad [Adelantado et al., 2017].

Las redes móviles LTE/4G ofrecen otro enfoque. Frente a LoRaWAN, LTE utiliza infraestructura ya desplegada por operadores, proporciona mayor caudal y permite una integración directa con servicios de Internet. Si hay cobertura, evita instalar un gateway propio. Como contrapartida, exige tarjeta SIM, depende del operador, consume más energía y puede implicar coste asociado al plan de datos.

Desde un punto de vista práctico, la conectividad celular en sistemas embebidos suele apoyarse en módulos de comunicación externos controlados mediante comandos AT. Estos módulos permiten al microcontrolador gestionar operaciones como el registro en red, la comprobación de señal, la configuración del APN, la apertura de una sesión de datos o el envío de peticiones HTTP. Esta aproximación separa la lógica de sensado y control, ejecutada por el microcontrolador, de la complejidad de acceso a la red móvil, delegada en el módem celular.

Los módulos LTE Cat 1 controlados mediante comandos AT resultan especialmente relevantes en prototipos IoT que necesitan conexión directa a Internet sin desplegar infraestructura radio propia. Su uso permite aprovechar redes móviles existentes y enviar datos a plataformas cloud mediante protocolos ampliamente soportados, aunque introduce dependencias asociadas a la cobertura, la tarjeta SIM, el consumo energético y la correcta inicialización del módem [SIMCom Wireless Solutions, 2026, SIMCom Wireless Solutions, 2024].

Dentro del ecosistema celular existen tecnologías más orientadas a IoT, como NB-IoT y LTE-M. NB-IoT se orienta a mensajes pequeños, buena penetración y bajo consumo. LTE-M ofrece mayor velocidad, menor latencia y mejor soporte de movilidad, con más consumo y complejidad que NB-IoT [Raza et al., 2017]. Ambas dependen de la disponibilidad del operador, de módulos compatibles y de condiciones comerciales concretas.

Tecnología	Alcance	Consumo	Infraestructura	Limitación principal
WiFi	Corto/medio	Medio/alto	Red local y punto de acceso	Dependencia de red cercana
Bluetooth/BLE	Corto	Bajo	Dispositivo o pasarela cercana	Alcance insuficiente en campo
LoRa	Largo	Bajo	Enlace radio propio	Bajo caudal y diseño propio
LoRaWAN	Largo	Bajo	Gateways y servidores LoRaWAN	Más complejidad de despliegue
LTE/4G / LTE Cat 1	Según cobertura	Medio/alto	Red de operador, SIM y módem celular	Consumo, SIM y dependencia de cobertura
NB-IoT	Amplio	Bajo	Red celular compatible	Disponibilidad variable
LTE-M	Amplio	Medio/bajo	Red celular compatible	Disponibilidad y coste

Tabla 2.3: Comparación general de tecnologías de conectividad para IoT en campo

La Tabla 2.3 debe interpretarse de forma cualitativa. El rendimiento real depende de antena, orografía, distancia, carga de red, frecuencia utilizada, tamaño de mensajes y frecuencia de transmisión. En campo abierto también importan la instalación, la cobertura real, el mantenimiento y la recuperación ante fallos.

La decisión de conectividad exige elegir compromisos. LoRaWAN puede encajar cuando se priorizan bajo consumo, mensajes esporádicos y despliegue propio. LTE/4G y LTE Cat 1 pueden ser más adecuados cuando se busca aprovechar infraestructura existente y disponer de comunicación directa con servicios de Internet. NB-IoT y LTE-M son opciones relevantes para IoT celular, aunque dependen de disponibilidad comercial y compatibilidad hardware.

En el contexto de TierraViva, esta revisión permite distinguir entre tecnologías atractivas desde el punto de vista teórico y tecnologías asumibles dentro de un prototipo validable. La conectividad no se valora únicamente por su alcance, sino también por la disponibilidad de hardware, la facilidad de depuración, la integración con plataformas cloud y la posibilidad de demostrar el funcionamiento extremo a extremo dentro del alcance temporal del TFG.

Esta distinción fue relevante durante el desarrollo, porque una tecnología técnicamente interesante puede no ser la más adecuada si obliga a dedicar la mayor parte del TFG a desplegar infraestructura de comunicaciones en lugar de validar el sistema completo.

2.7. Plataformas cloud para IoT

En una arquitectura IoT, la plataforma cloud actúa como punto intermedio entre los dispositivos de campo y las aplicaciones que consumen la información. Recibe datos de los nodos, los almacena, los organiza y permite consultarlos desde otros sistemas o interfaces de usuario. Esta capa aporta acceso remoto y reduce la infraestructura inicial necesaria. En arquitecturas con control local, su papel principal no es decidir la actuación, sino facilitar la publicación de telemetría, la consulta remota y la supervisión del sistema.

Una función habitual es el almacenamiento de telemetría. La telemetría incluye medidas de sensores, estados de funcionamiento, eventos o errores. Las plataformas IoT suelen organizar esta información como series temporales asociadas a un dispositivo, canal, variable o identificador. Así puede consultarse tanto el último valor recibido como la evolución de una magnitud.

Muchas plataformas cloud incorporan herramientas de visualización. Gráficas, paneles de control e indicadores de estado permiten comprobar rápidamente si los datos llegan correctamente y si las variables evolucionan de forma coherente. En prototipos, esta capacidad acelera la validación inicial.

Las APIs permiten que otros programas lean o escriban datos en la plataforma de forma controlada. En IoT son necesarias para enviar telemetría desde los dispositivos y para que otros sistemas puedan recuperar esa información, almacenarla, analizarla o mostrarla. Dependiendo de la plataforma, estas APIs pueden utilizar HTTP, MQTT u otros mecanismos de integración.

La organización de datos cambia según la herramienta: canales, dispositivos, variables, feeds o datastreams. La idea común es agrupar datos relacionados y asociarlos a una fuente concreta. Esta estructura evita mezclar flujos de distintas estaciones y facilita la visualización posterior.

ThingSpeak es una plataforma de análisis IoT que permite agregar, visualizar y analizar flujos de datos en la nube [MathWorks, 2026b]. Su estructura se basa en canales y campos, de forma que un dispositivo puede escribir datos mediante HTTP o MQTT y otros sistemas pueden leerlos mediante su API [MathWorks, 2026c, MathWorks, 2026a]. Por su sencillez, se utiliza con frecuencia en prototipos académicos y sistemas de monitorización sencilla.

En TierraViva, esta sencillez resulta especialmente útil: permite comprobar pronto si las estaciones publican datos correctamente, antes de invertir esfuerzo en un backend público propio o en una infraestructura cloud más compleja. Esto no elimina

la necesidad de diseñar correctamente el sistema de supervisión, pero sí reduce la complejidad inicial y permite validar antes el flujo estación–plataforma–visualización.

Existen otras alternativas relevantes. Ubidots organiza la información mediante dispositivos, variables, dashboards y eventos [Ubidots, 2025]. Adafruit IO proporciona una API HTTP orientada a proyectos de electrónica conectada [Adafruit, 2026]. Blynk utiliza datastreams para estructurar los datos entre dispositivo y plataforma [Blynk, 2025]. ThingsBoard permite recoger telemetría mediante distintos protocolos, almacenarla como series temporales y utilizarla en dashboards, reglas y APIs [ThingsBoard, 2026]. También puede construirse una plataforma propia con broker MQTT, base de datos y herramienta de visualización, asumiendo a cambio su instalación, mantenimiento y seguridad.

El uso de una plataforma cloud introduce dependencias. Si el dispositivo no puede conectarse a Internet o si la plataforma no está disponible, el envío o la lectura de datos puede fallar. La latencia también debe valorarse, sobre todo si la plataforma participa en decisiones de control. Por este motivo, una arquitectura que mantiene la decisión crítica en el nodo de campo reduce la dependencia operativa de la nube: la pérdida temporal de conectividad puede afectar a la supervisión, pero no debe impedir que la estación mantenga un estado seguro. Además, cada servicio puede imponer límites de uso, frecuencia de actualización, volumen de datos, histórico disponible o funcionalidades incluidas en planes gratuitos.

En este proyecto, esta idea se traduce en utilizar la plataforma cloud como apoyo para la telemetría y la supervisión, no como sustituto de la lógica local de seguridad.

2.8. Protocolos de aplicación en IoT: HTTP, MQTT y CoAP

En un sistema IoT, la tecnología de comunicación transporta los datos, pero el protocolo de aplicación define cómo se intercambia la información entre dispositivos, plataformas, servidores y aplicaciones. HTTP, MQTT y CoAP representan tres enfoques habituales con modelos distintos.

HTTP, del inglés *Hypertext Transfer Protocol*, es uno de los protocolos de aplicación más extendidos en Internet. Funciona mediante un modelo de petición-respuesta: un cliente realiza una solicitud a un servidor y este devuelve una respuesta. La especificación RFC 9110 lo define como un protocolo de aplicación sin estado para sistemas de información distribuidos [Fielding et al., 2022]. En IoT se utiliza con frecuencia para enviar datos a APIs mediante peticiones GET o POST, especialmente

cuando la plataforma cloud ya ofrece este tipo de interfaz.

Su principal ventaja es la compatibilidad. Existen muchas herramientas, librerías, servidores y plataformas capaces de trabajar con HTTP, lo que facilita la depuración y la integración. Su limitación es conocida: no fue diseñado específicamente para dispositivos IoT restringidos, puede introducir más sobrecarga que otras opciones y su modelo petición-respuesta no siempre es el más eficiente para comunicación asíncrona o mensajes muy frecuentes.

En prototipos que utilizan módulos celulares controlados mediante comandos AT, HTTP puede resultar especialmente conveniente. Muchos módems LTE permiten construir peticiones HTTP directamente desde el propio módulo, lo que simplifica la publicación de datos hacia plataformas cloud compatibles. Esta opción no siempre es la más eficiente desde el punto de vista energético, pero facilita la depuración y reduce el número de servicios intermedios necesarios para validar el sistema. Además, durante la depuración de un prototipo, una petición HTTP resulta relativamente fácil de inspeccionar y reproducir, lo que simplifica la localización de errores frente a arquitecturas que requieren broker, suscripciones persistentes o una infraestructura adicional.

MQTT, del inglés *Message Queuing Telemetry Transport*, utiliza un modelo de publicación/suscripción. Los dispositivos publican mensajes en un intermediario llamado *broker*, y los clientes interesados se suscriben a determinados *topics*. MQTT está estandarizado por OASIS como un protocolo ligero de mensajería publicación/suscripción para contextos IoT y M2M [OASIS, 2019].

Este modelo desacopla productores y consumidores. Un nodo no necesita conocer qué aplicación utilizará sus mensajes, solo el broker al que debe publicarlos. Esto encaja bien con arquitecturas donde hay varios dispositivos, un backend, una base de datos, dashboards o servicios de automatización. A cambio, el broker debe estar configurado, disponible y protegido.

CoAP, del inglés *Constrained Application Protocol*, fue diseñado para dispositivos y redes con recursos limitados. La especificación RFC 7252 lo define como un protocolo web especializado para nodos restringidos y redes restringidas [Shelby et al., 2014]. Mantiene una filosofía similar a HTTP, basada en recursos y operaciones tipo petición-respuesta, pero con una representación más ligera y habitualmente sobre UDP.

CoAP es interesante para redes de sensores y dispositivos embebidos de bajo consumo. Su adopción práctica depende de que la plataforma, las librerías y la infraestructura lo soporten de forma sencilla. En muchos prototipos, HTTP y MQTT están más directamente disponibles en plataformas cloud y herramientas de desarrollo.

Protocolo	Modelo	Ventaja principal	Limitación principal
HTTP	Petición-respuesta	Muy compatible con APIs web	Más sobrecarga; no específico de IoT
MQTT	Publicación/suscripción	Ligero y desacoplado mediante broker	Requiere broker disponible
CoAP	Petición-respuesta ligera	Diseñado para nodos restringidos	Menor disponibilidad práctica

Tabla 2.4: Comparación general entre HTTP, MQTT y CoAP en sistemas IoT

La Tabla 2.4 resume las diferencias principales. MQTT encaja bien con mensajería ligera y comunicación asíncrona. CoAP es adecuado para nodos restringidos y redes de bajo consumo. HTTP puede ser razonable cuando la prioridad es integrarse con APIs existentes, plataformas cloud sencillas o servicios que ya exponen endpoints HTTP.

La elección del protocolo debe considerar la arquitectura completa: plataforma cloud, librerías disponibles, frecuencia de envío, sentido de la comunicación, facilidad de depuración y tiempo disponible para validar el prototipo. En TierraViva, esta comparación permite justificar el uso de mecanismos compatibles con plataformas cloud sencillas y con módulos celulares controlados mediante comandos AT para publicar telemetría y estado, sin perder de vista que MQTT o CoAP podrían resultar adecuados en evoluciones futuras con una infraestructura diferente.

2.9. Automatización, reglas e inteligencia artificial en riego

En riego inteligente conviene diferenciar entre automatización basada en reglas e inteligencia artificial. Ambos enfoques pueden utilizar datos procedentes de sensores, pero no toman decisiones del mismo modo. Un sistema automatizado trabaja con condiciones explícitas definidas por el diseñador. Un sistema basado en inteligencia artificial requiere modelos entrenados o ajustados a partir de datos históricos suficientes.

La automatización basada en reglas es habitual en prototipos y sistemas de control sencillos. Consiste en definir condiciones del tipo: si una variable supera o cae por debajo de un valor, se ejecuta una acción. En riego, un ejemplo típico sería activar el sistema cuando la humedad del suelo desciende por debajo de un umbral y detenerlo cuando se alcanza una condición de cierre. Este control es fácil de interpretar, validar y explicar. Además, cuando las reglas se ejecutan en el propio nodo de campo, la actuación no

depende de que exista comunicación permanente con un servidor o equipo externo.

El uso de umbrales permite establecer límites claros, pero un único umbral puede provocar conmutaciones repetidas si la medida oscila alrededor del valor definido. Para evitarlo se introduce histéresis: dos valores distintos, uno para activar el riego y otro para desactivarlo. Es una técnica sencilla, pero mejora la estabilidad ante ruido o pequeñas variaciones.

Además de las reglas de decisión, en riego son necesarias reglas de seguridad. Estas no buscan optimizar el riego, sino evitar situaciones no deseadas: limitar el tiempo máximo de activación del actuador, detener el riego ante lecturas inválidas, ignorar valores imposibles, aplicar un tiempo mínimo entre actuaciones y actuar de forma conservadora ante errores de sensor, alimentación o comunicación.

La inteligencia artificial introduce un enfoque distinto. Los modelos pueden aprender patrones a partir de datos históricos y utilizarlos para predecir comportamientos futuros o detectar situaciones anómalas. En riego, esto podría aplicarse a predicción de humedad del suelo, estimación de necesidades hídricas, detección de fallos en sensores o identificación de patrones de consumo. Las revisiones sobre riego inteligente y agricultura de precisión destacan el interés de estas técnicas cuando existe información suficiente [Abioye et al., 2020, Bwambale et al., 2022].

No basta con utilizar la palabra “inteligente” ni con aplicar condiciones programadas manualmente para hablar de IA. Un conjunto de reglas del tipo *si-entonces* puede ser útil y técnicamente adecuado, pero no constituye por sí mismo un modelo de inteligencia artificial. Para hablar con rigor de IA sería necesario disponer de datos históricos representativos, seleccionar un modelo, entrenarlo, validarlo y comprobar que aporta valor frente a una estrategia basada en reglas simples.

En agricultura, esta exigencia es especialmente importante. Los datos dependen del tipo de suelo, clima, estación del año, cultivo, ubicación de los sensores, frecuencia de riego y condiciones de mantenimiento. Un modelo entrenado con pocos datos o con datos poco representativos podría tomar decisiones poco fiables. Aunque las técnicas de IA aplicadas a sensores agrícolas y monitorización de cultivos tienen un potencial claro [Taha et al., 2025, Naqvi et al., 2025], su uso debe plantearse con cuidado.

La detección de anomalías puede ser una línea especialmente interesante. Un sistema podría identificar valores fuera de rango, sensores que dejan de variar, caídas bruscas de una variable o diferencias anormales entre estaciones. Estas funciones pueden implementarse inicialmente mediante reglas simples y evolucionar hacia modelos más avanzados si se dispone de suficiente histórico.

Para TierraViva, el enfoque basado en reglas verificables es una primera fase

razonable. Permite justificar cada actuación, facilita las pruebas y evita atribuir al prototipo capacidades que todavía no han sido validadas. En esta versión, cada estación evalúa condiciones simples y seguras, mientras que la Raspberry Pi conserva un papel de supervisión e histórico. La inteligencia artificial queda como una línea futura cuando exista un histórico amplio, fiable y suficientemente representativo.

Capítulo 3

Métodos, materiales y decisiones de diseño

Este capítulo concreta el diseño de TierraViva a partir del marco técnico descrito en el capítulo anterior. Aquí se fijan los requisitos del sistema, se explican los criterios de elección y se documentan las decisiones que han guiado el prototipo: qué debe medir cada estación, cómo se comunica, qué papel conserva la Raspberry Pi y qué condiciones deben cumplirse antes de actuar sobre el riego.

A diferencia del estado del arte, el análisis se centra ya en el proyecto desarrollado. Algunas decisiones se tomaron de forma incremental, a medida que se validaban sensores, comunicaciones y software de supervisión. Por ello, el capítulo no se limita a enumerar materiales, sino que justifica por qué TierraViva adopta una arquitectura distribuida con decisión local en las estaciones.

3.1. Requisitos funcionales del sistema

Los requisitos funcionales definen las capacidades que debe proporcionar el sistema para cumplir con el alcance planteado para el proyecto. En el caso de TierraViva, estos requisitos no se limitan a la lectura de sensores, sino que cubren el ciclo completo de funcionamiento: adquisición de datos en campo, transmisión remota, almacenamiento, visualización, toma de decisiones y actuación sobre el riego.

La definición de estos requisitos permite separar con claridad qué debe hacer el sistema de cómo se implementa cada parte. Esta distinción es importante porque las decisiones tecnológicas concretas se justifican en apartados posteriores. En este punto se describe el comportamiento esperado del prototipo y la forma prevista de comprobar que cada función queda cubierta durante la validación.

El sistema se plantea a partir de estaciones de campo capaces de obtener medidas del entorno, evaluar localmente reglas sencillas de riego y enviar la información a una plataforma intermedia. A partir de esos datos, el sistema de supervisión debe poder registrar el histórico, visualizar el estado del prototipo y mostrar las actuaciones

realizadas por cada estación. La estación de campo debe ejecutar la actuación de forma autónoma y segura, evitando activaciones indefinidas, riegos repetidos o estados peligrosos ante lecturas inválidas, errores de comunicación o fallos eléctricos.

En la Tabla 3.1 se recogen los principales requisitos funcionales considerados para el diseño del prototipo.

Código	Requisito	Descripción	Validación prevista
RF-01	Medición de humedad del suelo	El sistema debe obtener lecturas periódicas de la humedad del sustrato en la zona monitorizada.	Prueba con sensor capacitivo de humedad del suelo.
RF-02	Medición ambiental	El sistema debe medir variables ambientales relevantes, incluyendo temperatura, humedad relativa y presión atmosférica.	Prueba con sensor ambiental.
RF-03	Medición de luminosidad	El sistema debe registrar la luminosidad o nivel de luz ambiente para contextualizar las condiciones del entorno.	Prueba con variación de luz.
RF-04	Envío remoto de telemetría	Las estaciones de campo deben enviar sus medidas a una plataforma remota para que puedan ser consultadas posteriormente.	Prueba de comunicación remota y recepción en plataforma cloud.
RF-05	Identificación de estación	Cada lectura debe poder asociarse a la estación de origen, permitiendo distinguir los datos de distintas zonas o nodos.	Comprobación de registros diferenciados por estación.
RF-06	Almacenamiento histórico	El sistema debe registrar las lecturas recibidas para conservar un histórico de funcionamiento.	Verificación de inserciones en SQLite.
RF-07	Visualización del estado	El sistema debe ofrecer una interfaz de consulta que permita ver las últimas lecturas y el estado general del prototipo.	Prueba de dashboard web en navegador.
RF-08	Decisión local de riego	Cada estación debe evaluar localmente si las condiciones medidas justifican una actuación sobre el riego.	Prueba de regla local de humedad, cooldown y estado seguro.
RF-09	Actuación local sobre el riego	La estación debe accionar físicamente el sistema de riego mediante relé y bomba de agua durante un tiempo limitado por firmware.	Prueba controlada con relé y bomba/carga de prueba.
RF-10	Publicación de estado de actuación	La estación debe publicar el estado del relé, el indicador de modo seguro y un código de estado que permita conocer si se ha producido una actuación, una situación de reposo o una incidencia.	Verificación de relay_state, safe_mode y statusCode en ThingSpeak.
RF-11	Seguridad de funcionamiento	El sistema debe desactivar o mantener apagada la bomba ante error de sensor, lectura inválida, fallo de comunicación, reinicio, cooldown activo o tiempo máximo excedido.	Pruebas de modo seguro, reinicio y límites de actuación.

Los tres primeros requisitos se relacionan con la adquisición de información en campo. La humedad del suelo es la variable más directamente vinculada con la decisión de riego, mientras que las variables ambientales permiten interpretar mejor las condiciones en las que se encuentra la zona monitorizada. En esta fase no se busca realizar un análisis agronómico completo, sino obtener un conjunto de medidas suficiente para validar el comportamiento del prototipo.

Los requisitos RF-04, RF-05 y RF-06 cubren el flujo de datos desde la estación hasta la plataforma intermedia y el sistema de supervisión. La telemetría debe recibirse de forma remota, quedar asociada a una estación concreta y almacenarse para su consulta posterior. Esta trazabilidad es necesaria para que el sistema no dependa únicamente de la última lectura recibida y para poder analizar la evolución de las variables durante las pruebas.

El requisito RF-07 se centra en la interacción con el usuario. Aunque el sistema pueda funcionar internamente, su utilidad queda limitada si no existe una forma clara de consultar el estado de las estaciones. Por ello, el dashboard debe permitir revisar las medidas recientes, detectar si los datos llegan correctamente y comprobar el estado general del sistema sin recurrir a mensajes de consola o consultas manuales a la base de datos.

Los requisitos RF-08, RF-09 y RF-10 introducen la parte de control. En la arquitectura final, la decisión de riego se ejecuta localmente en cada estación mediante reglas verificables, especialmente durante esta primera versión del prototipo. La estación no depende de una decisión externa para actuar: evalúa sus sensores, comprueba las condiciones de seguridad, activa la bomba si corresponde y publica después su estado. Esta publicación incluye las medidas ambientales, el estado del relé, el indicador de modo seguro y un código de estado. Con esa información, la Raspberry Pi puede supervisar lo ocurrido y conservar un histórico sin convertirse en un punto único de decisión para el riego.

El requisito RF-11 se ha definido de forma explícita porque la actuación sobre riego implica un riesgo mayor que la simple monitorización. Un fallo de software, una lectura inválida, un reinicio inesperado o una comunicación incompleta no deben provocar una activación indefinida de la bomba. Por ese motivo, el sistema debe adoptar un comportamiento conservador: ante una situación no válida o no verificable, el estado seguro será mantener la bomba apagada.

Estos requisitos funcionales servirán como base para las decisiones de diseño del resto del capítulo y para la estrategia de validación descrita más adelante. Cada requisito debe poder relacionarse con una prueba concreta, de forma que el

cumplimiento del sistema no se justifique solo por diseño, sino también mediante evidencias experimentales.

3.2. Requisitos no funcionales

Los requisitos no funcionales describen las condiciones de calidad que debe cumplir el sistema, más allá de las funciones concretas indicadas en el apartado anterior. En TierraViva estos requisitos son especialmente importantes porque el proyecto no consiste únicamente en que cada bloque funcione de forma aislada, sino en que el conjunto pueda integrarse, probarse, ampliarse y defenderse como un prototipo de ingeniería.

Al tratarse de un Trabajo Fin de Grado, el sistema debe mantener un equilibrio entre ambición técnica y viabilidad. No se pretende desarrollar un producto industrial cerrado, pero sí una solución suficientemente robusta, ordenada y reproducible como para demostrar el funcionamiento completo de la arquitectura propuesta. La Tabla 3.2 resume los principales requisitos no funcionales considerados.

Código	Requisito	Descripción
RNF-01	Modularidad	El sistema debe organizarse en bloques diferenciados: estación de campo autónoma, comunicación, almacenamiento, supervisión, visualización y actuación local.
RNF-02	Bajo coste	Los componentes seleccionados deben ser asumibles para un prototipo académico, evitando soluciones industriales de coste elevado cuando no sean necesarias.
RNF-03	Reproducibilidad	El montaje, la configuración y las pruebas deben poder repetirse a partir de la documentación generada durante el proyecto.
RNF-04	Facilidad de depuración	El sistema debe permitir probar los bloques por separado, identificar errores y consultar salidas intermedias durante el desarrollo.
RNF-05	Consumo razonable	El diseño debe evitar consumos innecesarios y permitir una futura alimentación autónoma, aunque la optimización energética completa no sea el objetivo principal.
RNF-06	Tolerancia a fallos	El sistema debe contemplar fallos de lectura, comunicación o actuación, adoptando estados seguros cuando no sea posible garantizar un funcionamiento correcto.
RNF-07	Escalabilidad	La arquitectura debe permitir incorporar más de una estación de campo sin rediseñar completamente el sistema.
RNF-08	Claridad de visualización	La interfaz debe presentar la información de forma comprensible, diferenciando lecturas, estados y posibles incidencias.
RNF-09	Adecuación académica	El alcance debe ser realizable dentro del tiempo y medios disponibles para el TFG, priorizando funciones verificables frente a desarrollos excesivamente complejos.

Tabla 3.2: Requisitos no funcionales del sistema TierraViva

La modularidad es uno de los requisitos más importantes del proyecto. Separar el sistema en bloques permite desarrollar y validar cada parte de forma progresiva, evitando que la actuación sobre el riego dependa de un único componente externo. Por ejemplo, la lectura de sensores puede comprobarse antes de integrar la comunicación remota, y la visualización puede probarse inicialmente con datos simulados o recibidos por otros medios. Esta forma de trabajo reduce el riesgo de que un fallo en una capa impida avanzar en el resto del sistema.

El bajo coste no se interpreta como elegir siempre el componente más barato, sino como seleccionar materiales razonables para el alcance del prototipo. En un TFG resulta más adecuado emplear plataformas y módulos accesibles, bien documentados

y fáciles de sustituir, siempre que proporcionen prestaciones suficientes para validar la arquitectura. Esta decisión también favorece la reproducibilidad, ya que otros estudiantes o desarrolladores podrían replicar el montaje sin depender de equipos propietarios o excesivamente específicos.

La reproducibilidad está directamente relacionada con la documentación del proceso. Las conexiones, configuraciones, pruebas, errores encontrados y decisiones de diseño deben quedar registrados con suficiente detalle. Esto no solo facilita la redacción de la memoria, sino que también permite justificar ante el tribunal que el sistema no se ha construido como una prueba puntual difícil de repetir, sino como un prototipo técnico verificable.

La facilidad de depuración también ha condicionado el planteamiento del trabajo. En un sistema con sensores, comunicaciones, base de datos, dashboard y actuación física, no es práctico probar todo únicamente al final. El sistema debe permitir observar salidas intermedias, comprobar lecturas por separado, revisar registros, analizar respuestas de comunicación y validar la actuación en condiciones controladas. Esta capacidad resulta clave para detectar errores de cableado, problemas de alimentación, fallos de formato en los datos o comportamientos inesperados en la lógica de control.

El consumo energético debe mantenerse en valores razonables. Aunque la optimización completa de autonomía no forma parte del núcleo del TFG, el sistema se orienta a estaciones de campo que podrían funcionar con batería o alimentación solar en futuras versiones. Por ello, el diseño no debe ignorar el consumo de módulos de comunicación, sensores y actuadores. En especial, la actuación sobre una bomba de agua y el uso de comunicaciones remotas obligan a considerar la alimentación como una restricción real del sistema.

La tolerancia a fallos es especialmente relevante porque TierraViva incorpora actuación sobre el riego. Un error en una medida puede ser aceptable si queda registrado y no se utiliza de forma crítica, pero un error que provoque la activación indefinida de la bomba sí comprometería el funcionamiento del sistema. Por este motivo, el comportamiento ante fallos debe ser conservador: si falta información, si una lectura no es válida o si se supera un límite de seguridad, el sistema debe priorizar el apagado de la bomba.

La escalabilidad se plantea de forma limitada pero necesaria. El prototipo debe poder trabajar con más de una estación de campo, al menos desde el punto de vista de identificación, almacenamiento y visualización. Esto implica que los datos no deben tratarse como si procedieran de un único nodo, sino que deben conservar la referencia de la estación de origen. Esta decisión facilita futuras ampliaciones a varias zonas de

riego o parcelas diferenciadas.

Por último, la claridad de visualización y la adecuación al alcance académico condicionan la forma de presentar el sistema. El dashboard debe servir para entender el estado del prototipo sin obligar al usuario a interpretar directamente tramas, códigos internos o registros de depuración. Al mismo tiempo, el proyecto debe evitar prometer funcionalidades que no puedan validarse con rigor dentro del tiempo disponible. En este sentido, se priorizan decisiones simples, trazables y defendibles frente a soluciones más complejas que aumentarían el riesgo de integración sin aportar una mejora proporcional al objetivo del TFG.

3.3. Especificaciones temporales y operativas

Además de los requisitos funcionales y no funcionales, el sistema necesita definir una serie de parámetros temporales y criterios operativos que regulen su comportamiento durante el funcionamiento. En un sistema IoT de riego, estos valores son importantes porque afectan al consumo energético, a la carga de comunicaciones, a la capacidad de respuesta y, sobre todo, a la seguridad de la actuación sobre la bomba de agua.

Las especificaciones temporales no se plantean como valores rígidos e inmodificables, sino como parámetros iniciales de diseño. Durante el desarrollo y la validación del prototipo pueden ajustarse en función de los resultados obtenidos, siempre manteniendo dos criterios principales: evitar comunicaciones innecesarias y garantizar que cualquier actuación sobre el riego quede limitada en el tiempo.

En la Tabla 3.3 se recogen las principales especificaciones temporales y operativas consideradas para TierraViva.

Código	Parámetro	Valor inicial	Criterio operativo
ET-01	Periodo de lectura de sensores	30 s	Las estaciones toman medidas periódicas del entorno sin necesidad de lectura continua.
ET-02	Frecuencia de envío de telemetría	30 s	Cada estación envía sus datos de forma periódica, evitando tráfico innecesario.
ET-03	Frecuencia de actualización del dashboard	3–5 s	La interfaz consulta el estado con mayor frecuencia que la llegada real de nuevos datos.
ET-04	Intervalo de evaluación local	30 s	Cada estación evalúa periódicamente sus lecturas y comprueba si se cumplen las condiciones locales de riego.
ET-05	Tiempo máximo de bomba activa	60 s	La activación de la bomba queda limitada para evitar riego indefinido ante errores de control.
ET-06	Tiempo mínimo entre riegos	Configurable	Se aplica un periodo de espera para evitar actuaciones repetidas en poco tiempo.
ET-07	Tiempo máximo sin publicación válida	5 min	Si la estación no consigue publicar telemetría durante este periodo, registra la incidencia cuando sea posible y garantiza que una actuación en curso no permanezca activa sin control.
ET-08	Estado seguro por defecto	Bomba apagada	Al arrancar, ante error de lectura, fallo de comunicación o condición no verificable, la estación mantiene desactivado el riego.
ET-09	Comportamiento ante dato inválido	Ignorar y registrar	Las lecturas no válidas no deben provocar una actuación automática.
ET-10	Comportamiento ante pérdida de comunicación	Bomba apagada o no actuación	La pérdida de comunicación no debe dejar la bomba activa.

Tabla 3.3: Especificaciones temporales y operativas iniciales del sistema

El periodo de lectura y envío de telemetría se fija inicialmente en 30 segundos. Este valor permite observar la evolución de las variables medidas sin generar un volumen excesivo de comunicaciones. En un sistema de riego no resulta necesario enviar datos cada segundo, ya que la humedad del suelo y las condiciones ambientales no cambian con tanta rapidez como para justificar una frecuencia tan elevada. Para demostraciones o pruebas concretas, este intervalo podría reducirse temporalmente, pero no se considera necesario para el funcionamiento normal del prototipo.

La actualización del dashboard se plantea con una frecuencia mayor que la recepción de nuevos datos. Esto permite que la interfaz sea más dinámica y pueda mostrar información como la última actualización recibida, el estado de conexión o posibles avisos de datos obsoletos. No obstante, debe quedar claro que refrescar la interfaz cada pocos segundos no implica que la estación esté generando nuevas medidas con esa misma frecuencia.

El intervalo de evaluación local se vincula al periodo de lectura y envío de telemetría. Como la decisión de riego se ejecuta en la propia estación, no es necesario mantener un mecanismo remoto de control para decidir la actuación. La estación lee sus sensores, actualiza su estado interno y comprueba si se cumplen las condiciones de riego en cada ciclo. Este enfoque reduce la dependencia de la conectividad externa y simplifica la lógica de actuación.

La actuación sobre la bomba de agua se trata como el punto más crítico desde el punto de vista operativo. Cualquier activación debe estar limitada por un tiempo máximo, ya que un fallo software, una lectura errónea, un reinicio inesperado o un problema de comunicación no deben traducirse en un riego indefinido. Aunque la duración concreta pueda ajustarse durante las pruebas, el firmware debe impedir que la bomba permanezca activa más tiempo del permitido.

Además, después de cada actuación debe aplicarse un tiempo mínimo entre riegos. Este periodo de espera evita activaciones continuas provocadas por lecturas inestables, reinicios, ruido en la medida o pequeñas oscilaciones alrededor del umbral de decisión. En una primera versión del prototipo, este parámetro puede configurarse de forma conservadora y ajustarse posteriormente a partir de las pruebas.

El estado seguro por defecto del sistema es la bomba apagada. Esta decisión afecta tanto al arranque como a las situaciones de error. Si la estación se reinicia, si detecta una lectura inválida, si entra en cooldown, si pierde comunicación o si se supera el tiempo máximo de actuación, el sistema debe volver a un estado conservador. En este proyecto se considera preferible no regar ante una situación no verificable antes que mantener la bomba funcionando sin control.

Finalmente, los datos inválidos no deben utilizarse para tomar decisiones automáticas. Una lectura fuera de rango, ausente o incoherente debe registrarse como incidencia o descartarse, pero no debe activar la bomba por sí sola. Esta especificación refuerza el criterio general del sistema: TierraViva debe poder actuar sobre el riego, pero solo cuando la información disponible y el estado operativo sean suficientemente fiables.

3.4. Metodología de trabajo

La metodología seguida en TierraViva se ha basado en un desarrollo incremental por bloques. En lugar de intentar construir el sistema completo desde el primer momento, se ha dividido el proyecto en fases más pequeñas, cada una orientada a validar una parte concreta de la arquitectura. Este enfoque resulta adecuado para un sistema IoT con sensores, comunicaciones, almacenamiento, visualización y actuación física, ya que permite aislar errores y reducir el riesgo de integración.

La idea principal ha sido avanzar desde pruebas controladas hacia una arquitectura cada vez más cercana al funcionamiento final. Primero se analizaron las necesidades del problema y las alternativas tecnológicas disponibles. Después se desarrollaron prototipos parciales para comprobar sensores, comunicaciones, almacenamiento y visualización. Finalmente, las distintas partes se fueron conectando hasta formar una solución de monitorización y control más completa.

Esta forma de trabajo también ha permitido documentar decisiones y cambios de enfoque. En un proyecto de este tipo, no todas las decisiones se pueden cerrar desde el principio, ya que algunas dependen de la disponibilidad real de hardware, de la facilidad de integración o de los resultados obtenidos en pruebas. Por ello, la metodología no se ha limitado a implementar una arquitectura previamente fija, sino que ha incorporado revisión, validación y corrección de decisiones durante el desarrollo.

En la Tabla 3.4 se resumen las fases principales seguidas durante el proyecto.

Fase	Nombre	Objetivo principal
1	Análisis inicial del problema	Identificar la necesidad de monitorizar y controlar el riego en un entorno no supervisado de forma continua.
2	Revisión tecnológica	Estudiar alternativas de sensorización, plataformas embebidas, comunicaciones, almacenamiento, visualización y actuación.
3	Diseño conceptual de la arquitectura	Definir los bloques principales del sistema y la relación entre estaciones autónomas de campo, plataforma intermedia, sistema de supervisión y actuador.
4	Prototipo local con Raspberry	Validar una primera cadena de adquisición, mensajería, almacenamiento y visualización mediante Raspberry Pi, MQTT, SQLite, FastAPI y dashboard.
5	Validación de sensores	Comprobar el funcionamiento de los sensores de humedad de suelo, luminosidad y variables ambientales antes de integrarlos en una estación.
6	Validación de comunicaciones	Probar la comunicación remota necesaria para publicar telemetría y estado desde las estaciones.
7	Elección de plataforma cloud	Seleccionar una plataforma intermedia que permitiera publicar telemetría, consultar datos remotos y supervisar el estado de las estaciones de forma sencilla y verificable.
8	Diseño de actuación local	Definir el control autónomo de la bomba de agua mediante relé, incluyendo reglas locales, tiempos máximos, cooldown y estado seguro.
9	Integración de subsistemas	Conectar adquisición, comunicación, supervisión, almacenamiento, visualización, decisión local y actuación en una arquitectura coherente.
10	Validación experimental	Comprobar el funcionamiento del sistema mediante pruebas parciales, pruebas de estación autónoma y pruebas de supervisión extremo a extremo.
11	Documentación de incidencias y decisiones	Registrar problemas encontrados, alternativas descartadas, cambios de enfoque y criterios técnicos adoptados.

Tabla 3.4: Fases principales de la metodología de trabajo

La primera fase consistió en analizar el problema desde el punto de vista funcional. No se trataba únicamente de medir humedad o mostrar datos, sino de diseñar un sistema capaz de observar el entorno, transmitir información, registrar un histórico, visualizar el estado y actuar sobre el riego de forma controlada. Esta fase permitió transformar una necesidad práctica en un conjunto de requisitos técnicos, descritos en

los apartados anteriores.

A continuación, se realizó una revisión tecnológica centrada en las áreas que condicionaban el diseño: sensores agrícolas, arquitecturas IoT, plataformas embebidas, comunicaciones para entornos rurales, plataformas cloud, protocolos de aplicación y mecanismos de automatización. Esta revisión no tuvo como objetivo elegir la tecnología más avanzada en términos absolutos, sino identificar qué opciones eran viables para un prototipo académico, reproducible y defendible.

La tercera fase fue el diseño conceptual de la arquitectura. En este punto se definieron los bloques principales del sistema: estaciones de campo, adquisición de datos, comunicación remota, plataforma intermedia, sistema de supervisión, almacenamiento, dashboard y actuación. Esta separación en bloques permitió trabajar de forma progresiva, validando cada parte antes de integrarla con el resto.

Una fase especialmente útil fue el desarrollo de un prototipo local previo con Raspberry Pi. En esta etapa se utilizó una arquitectura basada en MQTT, SQLite, FastAPI y un dashboard web para comprobar la cadena de tratamiento de datos. Este prototipo no debe interpretarse como la arquitectura final de comunicaciones del sistema, sino como una fase intermedia de validación. Permitted probar la recepción de mensajes, el almacenamiento en base de datos, la exposición de datos mediante API y la visualización en navegador antes de incorporar la comunicación remota definitiva de las estaciones.

La validación de sensores se realizó de forma independiente. Primero se comprobó el bus I2C y las direcciones de los sensores digitales, y después se integraron las lecturas de luminosidad, temperatura, humedad relativa, presión y humedad de suelo. Esta fase fue necesaria para evitar que un fallo de cableado, dirección I2C o librería se confundiera posteriormente con un problema de comunicación o de backend. En el caso del sensor de humedad de suelo, además, se trabajó inicialmente con valores brutos, dejando la calibración completa con suelo seco y húmedo como parte de la validación experimental.

La validación de comunicaciones se planteó también de forma separada. Antes de integrar el módulo de comunicaciones con el firmware de estación, se comprobó su funcionamiento mediante comandos AT, verificando aspectos como identificación del módulo, estado de la SIM, nivel de señal, registro en red, dirección IP y capacidad de comunicación HTTP. Esta forma de trabajo permitió aislar los problemas propios del módem antes de mezclarlos con la lectura de sensores o la lógica de control.

Tras estas pruebas, se valoró la plataforma cloud más adecuada para el prototipo. La decisión debía tener en cuenta que las estaciones de campo no estarían conectadas físicamente a la Raspberry Pi, por lo que la telemetría y el estado de funcionamiento

debían publicarse en una plataforma accesible por red. Por este motivo, la plataforma intermedia debía permitir la escritura de datos desde las estaciones y la consulta posterior desde el sistema de supervisión, sin exigir una comunicación directa entre los nodos de campo y la Raspberry Pi.

El diseño de actuación se abordó como un bloque crítico. A diferencia de la monitorización, la actuación sobre riego tiene consecuencias físicas: una bomba activa durante demasiado tiempo puede provocar consumo innecesario de agua o funcionamiento incorrecto del sistema. Por ello, el control mediante relé se diseñó con un criterio conservador: bomba apagada por defecto, tiempo máximo de activación, validación de lecturas, cooldown, bloqueo ante estados no verificables y publicación del estado de la actuación.

La integración de subsistemas se realizó después de validar las partes principales. Esta integración incluyó la unión entre adquisición de datos, comunicación, almacenamiento, visualización, lógica local de decisión y actuación. El objetivo no era solo que cada bloque funcionara por separado, sino comprobar que la estación podía medir, decidir, actuar y publicar su estado, y que la Raspberry Pi podía supervisar ese comportamiento a partir de los datos disponibles en la plataforma intermedia.

La validación experimental se planteó en varios niveles. En primer lugar, se realizaron pruebas unitarias de sensores y comunicación. Después, pruebas parciales de almacenamiento, API y dashboard. Finalmente, se prevén pruebas de integración para comprobar el flujo completo: adquisición de datos en estación, decisión local de riego, actuación sobre el relé, publicación de telemetría y estado, registro histórico y visualización desde la Raspberry Pi.

Por último, durante el desarrollo se ha mantenido una documentación de incidencias y decisiones. Esta parte es relevante porque el proyecto ha requerido corregir supuestos iniciales, separar prototipos previos de arquitectura final y ajustar decisiones conforme se obtenían resultados reales. Registrar estos cambios no se considera una debilidad, sino una parte natural del proceso de ingeniería, ya que permite justificar por qué se adoptó una solución concreta y qué alternativas quedaron descartadas o reservadas para trabajos futuros.

El cambio de enfoque más importante fue separar definitivamente supervisión y control local. En una versión inicial se valoró que la Raspberry Pi centralizara parte de la lógica de riego, pero finalmente se optó por desplazar esa responsabilidad al firmware de cada estación. Esta modificación simplificó el flujo de comunicación, eliminó la necesidad de transportar órdenes de riego por la nube y redujo la dependencia de la Raspberry como punto único de decisión.

3.5. Arquitectura general planteada

A partir de los requisitos definidos en los apartados anteriores, la arquitectura general de TierraViva se plantea como un sistema IoT distribuido, formado por estaciones autónomas de campo, una plataforma intermedia de comunicación y un sistema de supervisión basado en Raspberry Pi. El objetivo de esta arquitectura es separar claramente la adquisición de datos, la decisión local, la actuación física, la publicación de telemetría, el almacenamiento histórico y la visualización.

La arquitectura se ha diseñado teniendo en cuenta una restricción importante: las estaciones de campo no están conectadas físicamente a la Raspberry Pi. Por tanto, no se puede asumir una comunicación directa mediante USB, puerto serie o red local entre ambos extremos. Las estaciones deben poder enviar datos de forma remota mediante el módulo LTE, y la Raspberry debe consultar esa información desde la plataforma intermedia. Esto condiciona el diseño completo del sistema y evita que el riego dependa de una conexión local o de una orden enviada desde la Raspberry.

En términos generales, el sistema se divide en cuatro bloques principales:

- **Estaciones autónomas de campo:** encargadas de medir las variables del entorno, evaluar reglas locales de riego, accionar la bomba mediante relé y publicar telemetría y estado.
- **Capa de comunicaciones y plataforma intermedia:** responsable de recibir los datos publicados por las estaciones y hacerlos accesibles para consulta remota.
- **Sistema de supervisión:** implementado sobre Raspberry Pi, encargado de consultar datos, almacenarlos, exponer una API y servir el dashboard.
- **Interfaz de usuario:** orientada a visualizar el estado del sistema, revisar el histórico y supervisar las actuaciones realizadas por las estaciones.

La Figura 3.1 muestra una visión general de la arquitectura planteada.

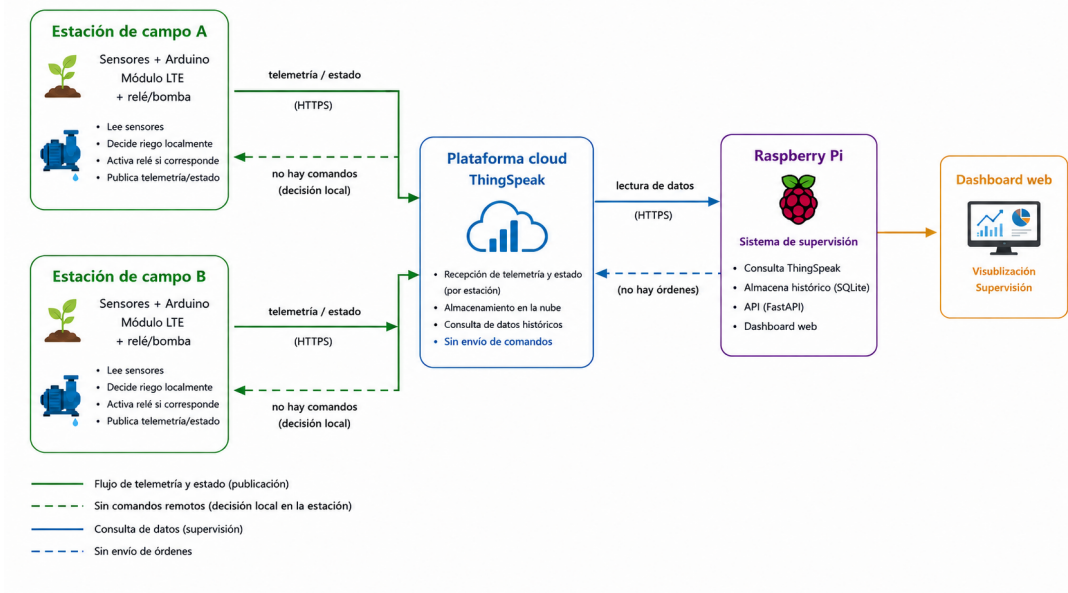


Figura 3.1: Arquitectura general planteada para TierraViva.

En esta sección no se entra todavía en el detalle de implementación; ese desarrollo se reserva para el capítulo siguiente. Lo importante aquí es dejar fijado el reparto de responsabilidades entre estaciones, plataforma intermedia, Raspberry Pi y dashboard. Las estaciones de campo se encargan de la interacción directa con el entorno físico y de la actuación sobre el riego. ThingSpeak permite publicar y consultar la telemetría de forma remota. La Raspberry Pi actúa como sistema de supervisión, histórico y visualización, sin convertirse en el elemento responsable de decidir la activación de la bomba.

Esta división condiciona el resto de decisiones del proyecto. La actuación sobre el riego debe ejecutarse localmente en la estación, donde se encuentra físicamente la bomba. De este modo, una pérdida de conectividad o una indisponibilidad temporal de la Raspberry no impide que la estación conserve un comportamiento seguro. El detalle de la arquitectura final, incluyendo los canales concretos de ThingSpeak, el flujo de telemetría, la publicación de estado y el papel de la Raspberry Pi como sistema de supervisión, se desarrolla en el Capítulo 4.

3.6. Elección de variables y sensores

La selección de variables medidas se ha realizado teniendo en cuenta el papel que deben cumplir dentro del prototipo: monitorizar el estado de una zona de cultivo y aportar información suficiente para alimentar la decisión local de riego en cada estación. No se pretende medir todos los parámetros agronómicos posibles, sino elegir un conjunto

reducido de variables que permita validar el comportamiento del prototipo de forma razonable, reproducible y coherente con el alcance del TFG.

En TierraViva se combinan variables del suelo y variables ambientales. La humedad del suelo se considera la variable principal para la decisión local de riego, ya que aporta información directa sobre la disponibilidad de agua en el sustrato. Las variables ambientales, como temperatura, humedad relativa, presión atmosférica y luminosidad, no sustituyen a la humedad del suelo, pero ayudan a interpretar el contexto en el que evoluciona la zona monitorizada. Esta combinación es habitual en sistemas de agricultura de precisión basados en sensores, donde el interés no está solo en una lectura aislada, sino en la observación conjunta del entorno [Soussi et al., 2024].

Variable	Función en el sistema	Motivo de selección
Humedad del suelo	Variable principal para la decisión de riego.	Indica de forma directa si el sustrato presenta condiciones de sequedad o humedad.
Temperatura ambiente	Variable de contexto ambiental.	Ayuda a interpretar evaporación, estrés térmico y condiciones generales del entorno.
Humedad relativa	Información ambiental complementaria.	Permite contextualizar la pérdida de agua y las condiciones de evaporación.
Presión atmosférica	Dato adicional del sensor ambiental.	Aporta contexto ambiental y se obtiene sin añadir un sensor específico.
Luminosidad	Indicador de exposición solar.	Permite identificar condiciones de luz que pueden influir en evaporación y demanda hídrica.

Tabla 3.5: Variables seleccionadas para la monitorización de TierraViva

La humedad del suelo se ha seleccionado como variable principal porque es la más relacionada con la necesidad de riego. Un sistema que decidiera únicamente a partir de temperatura o luminosidad podría detectar condiciones de calor o exposición solar, pero no sabría si el terreno conserva todavía suficiente agua. Por este motivo, la lógica de riego debe apoyarse prioritariamente en el estado del sustrato.

Para esta medida se ha utilizado un sensor capacitivo de humedad de suelo. Frente a sensores resistivos más simples, este tipo de sensor reduce el problema de corrosión asociado al contacto con humedad, ya que la medida se basa en el cambio capacitivo del entorno del sensor. Además, proporciona una salida analógica fácil de leer desde Arduino mediante una entrada ADC. En el prototipo se emplea un módulo capacitivo de humedad de suelo V1.2 compatible con Arduino, comercializado por AZ-Delivery, cuya

documentación lo describe como un sensor analógico de medida capacitiva orientado a la monitorización de humedad en sustrato [AZ-Delivery, 2026]. La lectura se obtiene inicialmente como un valor bruto, por lo que no debe interpretarse directamente como un porcentaje absoluto sin calibración previa.

Esto afecta directamente a la forma de validar la lectura. El sistema puede trabajar inicialmente con valores *raw*, pero para una decisión de riego más ajustada será necesario calibrar el sensor con muestras de suelo seco y suelo húmedo. Durante las primeras pruebas se comprobó que el valor analógico variaba al modificar las condiciones de contacto del sensor, lo que permitió validar la respuesta básica de la lectura antes de plantear una calibración completa en sustrato real.

La temperatura ambiente se incluye como variable de contexto. No permite decidir por sí sola si hay que regar, pero ayuda a interpretar el comportamiento del suelo. A igualdad de humedad inicial, un entorno con temperatura elevada puede favorecer mayor evaporación y acelerar la pérdida de agua. Registrar la temperatura permite entender mejor la evolución de las medidas y comparar condiciones de prueba distintas.

La humedad relativa aporta información complementaria sobre el ambiente. Una humedad relativa baja puede estar asociada a mayor demanda evaporativa, mientras que una humedad relativa elevada puede indicar condiciones en las que la pérdida de agua por evaporación es menor. En esta primera versión del sistema no se utiliza como criterio principal de riego, pero sí mejora la interpretación del entorno y puede ser útil para futuras reglas más completas.

La presión atmosférica se obtiene como dato adicional del sensor ambiental. Su influencia directa sobre la decisión de riego es menor que la de la humedad del suelo, la temperatura o la humedad relativa. Sin embargo, al estar disponible en el mismo sensor, no añade complejidad hardware significativa. En este trabajo se conserva como dato de contexto y como parte del histórico ambiental registrado.

Para medir temperatura, humedad relativa y presión se ha seleccionado el BME280. Este componente integra en un único sensor digital la medida de temperatura, humedad y presión, lo que reduce el número de módulos necesarios y simplifica el cableado. Bosch Sensortec lo describe como un sensor combinado de humedad, presión y temperatura, orientado a aplicaciones de bajo consumo y tamaño reducido [Bosch Sensortec, 2024]. En el prototipo se utiliza mediante bus I2C, lo que permite compartir las líneas de comunicación con otros sensores digitales.

La luminosidad se incorpora como indicador de exposición solar. Aunque no determina directamente el contenido de agua del suelo, ayuda a contextualizar las condiciones ambientales. Una zona con mayor exposición solar puede presentar mayor

evaporación y calentamiento, mientras que una zona sombreada puede conservar humedad durante más tiempo. Esta variable resulta útil para interpretar diferencias entre estaciones o entre distintos momentos del día.

Para esta medida se ha seleccionado el BH1750, integrado habitualmente en módulos GY-302. Se trata de un sensor digital de luminosidad con interfaz I2C. Su elección se debe a que entrega directamente medidas en lux, evitando estimar la luminosidad a partir de una lectura analógica sin escala física clara. La documentación del BH1750FVI lo describe como un sensor digital de luz ambiente con interfaz I2C y conversión directa de iluminancia [ROHM Semiconductor, 2011].

Sensor	Variable medida	Justificación de elección
Sensor capacitivo de humedad de suelo V1.2	Humedad del sustrato mediante lectura analógica.	Variable principal para riego; bajo coste; integración sencilla con Arduino; menor corrosión que sensores resistivos.
BME280	Temperatura, humedad relativa y presión atmosférica.	Integra tres variables ambientales en un único módulo digital; reduce cableado; comunicación I2C.
BH1750 / GY-302	Luminosidad ambiente en lux.	Proporciona medida digital de luz; comparte bus I2C; facilita interpretar exposición solar.

Tabla 3.6: Sensores seleccionados para las estaciones de campo

La elección de estos sensores responde también a criterios prácticos. Son módulos habituales en prototipos con Arduino, tienen bajo coste, disponen de documentación y librerías accesibles, y permiten validar el sistema sin recurrir a instrumentación industrial. Además, el uso de I2C para el BME280 y el BH1750 simplifica la integración, ya que ambos sensores pueden compartir las líneas SDA y SCL del Arduino UNO siempre que sus direcciones no coincidan.

El detalle de los componentes empleados, las conexiones eléctricas de los sensores, las pruebas iniciales de lectura y los fragmentos de código utilizados durante la validación se recoge en los Anexos A, B, C y E. De esta forma, el cuerpo principal mantiene la justificación técnica de la elección de variables y sensores, mientras que la información reproducible del montaje queda documentada como material complementario.

En conjunto, la combinación seleccionada cubre las variables mínimas necesarias para una primera versión funcional de TierraViva: una variable principal de decisión, la humedad del suelo, y varias variables ambientales de contexto. En fases

posteriores podrían añadirse sensores más avanzados, como caudal, nivel de depósito, conductividad eléctrica o pH, pero incorporarlos en esta primera versión aumentaría la complejidad sin ser imprescindible para validar la arquitectura general del sistema.

3.7. Elección de plataforma de control local y supervisión

La arquitectura de TierraViva separa claramente dos niveles funcionales. Por un lado, las estaciones de campo necesitan una plataforma sencilla y cercana al hardware, capaz de leer sensores, comunicarse con el módulo LTE, aplicar reglas locales de riego y controlar el relé de la bomba. Por otro lado, el sistema necesita una plataforma con mayor capacidad software para consultar datos, almacenarlos, exponer una API y servir el dashboard web. Esta segunda plataforma no actúa como cerebro del riego, sino como sistema de supervisión e histórico. Esta diferencia de funciones justifica el uso combinado de Arduino UNO y Raspberry Pi 5, en lugar de concentrar todo el sistema en una única plataforma.

Arduino UNO se ha seleccionado como plataforma principal de las estaciones de campo. Se trata de una placa basada en el microcontrolador ATmega328P, con funcionamiento a 5 V, 14 pines digitales, 6 entradas analógicas, 32 KB de memoria flash, 2 KB de SRAM y frecuencia de reloj de 16 MHz [Arduino, 2026b]. Aunque sus prestaciones son limitadas frente a plataformas más modernas, resultan suficientes para las tareas previstas en la estación: lectura de sensores, gestión de tiempos, comunicación serie con el módulo A7670E-LASA y control de una salida digital asociada al relé.

Una ventaja importante de Arduino UNO en este proyecto es su sencillez de integración. La lectura del sensor capacitivo de humedad del suelo puede realizarse mediante una entrada analógica, mientras que los sensores BME280 y BH1750 pueden compartir el bus I2C del Arduino. Además, el control del relé requiere únicamente una salida digital. Esta correspondencia directa entre necesidades del sistema y recursos de la placa facilita el desarrollo y la depuración del prototipo.

También se ha tenido en cuenta la disponibilidad de documentación, librerías y ejemplos. En un TFG con integración de varios subsistemas, la plataforma de estación no debe convertirse en el principal foco de incertidumbre. Arduino UNO permite validar de forma rápida sensores, salidas digitales, comunicación serie y temporización. Esta característica ha resultado útil durante las pruebas iniciales, donde se pudo comprobar primero el funcionamiento básico de la placa, después el bus I2C y finalmente la lectura integrada de sensores.

No obstante, la elección de Arduino UNO implica aceptar limitaciones. La memoria disponible es reducida, no dispone de sistema operativo, no incorpora conectividad inalámbrica integrada y su capacidad de procesamiento es limitada. Tampoco es una plataforma adecuada para ejecutar una base de datos, un servidor web o procesos software complejos. Por ello, en TierraViva Arduino no se utiliza como controlador global del sistema, sino como controlador local de estación. Su papel es ejecutar tareas concretas, repetitivas y cercanas al hardware.

La Raspberry Pi 5 se ha seleccionado como sistema de supervisión del prototipo. A diferencia de Arduino UNO, Raspberry Pi 5 es un computador de placa reducida capaz de ejecutar un sistema operativo completo. La documentación oficial la describe con un procesador Arm Cortex-A76 de cuatro núcleos a 2,4 GHz, lo que la sitúa en un nivel de capacidad muy superior al de un microcontrolador clásico [Raspberry Pi Ltd, 2026]. En este proyecto, esa capacidad permite ejecutar varios componentes software: lectura de datos desde la plataforma cloud, almacenamiento en SQLite, API mediante FastAPI y dashboard web.

Esta elección permite separar las funciones críticas de adquisición y actuación local de las funciones de supervisión y análisis. La Raspberry Pi no necesita estar conectada físicamente a las estaciones ni leer directamente los sensores. Su función es actuar como sistema de supervisión: consulta la información disponible, mantiene un histórico, expone datos mediante una API y permite visualizar el estado del prototipo. Esta separación es importante en TierraViva porque las estaciones se plantean como nodos remotos desplegados en campo, mientras que la Raspberry puede ubicarse en otro lugar y comunicarse con ellas a través de la plataforma intermedia.

Centralizar todo en Arduino UNO no sería adecuado para el alcance completo del sistema. Aunque Arduino puede leer sensores y controlar un relé, no está pensado para mantener una base de datos, servir una interfaz web, ejecutar una API o gestionar históricos de forma cómoda. Forzar esas funciones en una plataforma tan limitada aumentaría la complejidad del firmware y reduciría la capacidad de depuración. Además, mezclar adquisición, comunicación, almacenamiento, visualización y lógica global en un único microcontrolador haría que el sistema fuera menos modular y más difícil de mantener.

Tampoco sería conveniente utilizar únicamente Raspberry Pi en las estaciones de campo. Aunque Raspberry Pi ofrece mucha más potencia, también introduce un consumo mayor, tiempos de arranque más largos y dependencia de un sistema operativo. Además, no dispone de entradas analógicas nativas, por lo que la lectura directa de sensores analógicos requeriría componentes adicionales. Para una estación

remota alimentada en campo, que debe leer sensores, controlar un relé y mantener un estado seguro, un microcontrolador sencillo resulta más apropiado que un computador completo.

La combinación de ambas plataformas permite asignar a cada una el papel para el que resulta más adecuada. Arduino UNO se encarga de la interacción directa con el entorno físico: sensores, módulo de comunicaciones y relé. Raspberry Pi 5 aporta la capacidad necesaria para almacenar datos, servir una API y ofrecer una interfaz de supervisión. Esta separación reduce riesgos de integración y facilita validar cada bloque por separado.

Aspecto	Arduino UNO	Raspberry Pi 5
Rol en TierraViva	Controlador local autónomo de estación de campo.	Sistema de supervisión, almacenamiento y visualización.
Relación con sensores	Lectura directa de sensores analógicos y digitales.	No lee sensores directamente en la arquitectura final.
Actuación	Decisión local y control del relé asociado a la bomba.	No participa en la actuación física ni genera órdenes de riego.
Capacidad software	Firmware sencillo y tareas embebidas.	Sistema operativo, API, base de datos y dashboard.
Consumo y arranque	Menor consumo y arranque inmediato.	Mayor consumo y arranque dependiente del sistema operativo.
Adecuación principal	Nodo de campo sencillo y robusto.	Procesamiento, histórico y supervisión.

Tabla 3.7: Comparación funcional entre Arduino UNO y Raspberry Pi 5 en TierraViva

La Tabla 3.7 resume esta división de responsabilidades. La decisión no se basa en considerar una plataforma mejor que la otra, sino en asignar cada tarea al dispositivo más adecuado. En una arquitectura distribuida, esta separación es más importante que la potencia individual de cada placa.

Durante el diseño también se consideraron alternativas. ESP32, por ejemplo, incorpora WiFi y Bluetooth, ofrece mayor capacidad de procesamiento que Arduino UNO y trabaja de forma nativa a 3,3 V. Podría ser una opción interesante en sistemas IoT donde se quiera integrar conectividad inalámbrica directamente en el nodo. Sin embargo, en este proyecto la comunicación remota se realiza mediante un módulo LTE externo, por lo que la conectividad integrada del ESP32 no era imprescindible. Además, mantener Arduino UNO permitía aprovechar una plataforma ya disponible, conocida y suficiente para las funciones de estación.

Otras alternativas, como STM32 o placas equivalentes, ofrecen más prestaciones y mayor control sobre periféricos. Sin embargo, también suelen aumentar la complejidad de configuración, depuración y desarrollo. Para el alcance de este TFG, la prioridad no era elegir la plataforma embebida más potente, sino una suficientemente fiable y sencilla para validar la arquitectura completa.

En conjunto, la elección de Arduino UNO y Raspberry Pi 5 responde a una decisión de diseño modular. Arduino permite construir estaciones de campo simples, reproducibles y cercanas al hardware. Raspberry Pi 5 aporta la capacidad necesaria para almacenar datos, servir una API y ofrecer una interfaz de supervisión. Esta combinación mantiene el equilibrio entre sencillez, coste, capacidad técnica y adecuación al alcance académico del proyecto.

3.8. Selección de solución de comunicaciones para estaciones remotas

La comunicación entre las estaciones de campo y el resto del sistema es una de las decisiones más importantes de TierraViva. A diferencia de un montaje de laboratorio, donde los sensores pueden conectarse directamente a un ordenador o a una Raspberry Pi mediante USB o red local, el escenario planteado requiere estaciones remotas capaces de enviar datos y estado desde una zona de cultivo sin depender de una red WiFi local ni de una conexión física con la Raspberry Pi. Por tanto, la solución de comunicaciones debía ser compatible con un despliegue en campo, funcionar sin depender de una red WiFi local y permitir la integración con una plataforma de datos externa para supervisión remota.

Durante la fase inicial del proyecto se valoraron principalmente dos enfoques: una solución basada en LoRa/LoRaWAN y una solución basada en conectividad móvil LTE. Ambas alternativas eran técnicamente razonables, pero respondían a prioridades distintas. LoRa y LoRaWAN resultaban especialmente atractivas por su bajo consumo, largo alcance y orientación a redes de sensores distribuidas. LTE, en cambio, ofrecía una integración más directa con servicios en Internet, a cambio de un mayor consumo energético y de la dependencia de cobertura móvil.

La alternativa LoRa/LoRaWAN encajaba bien con la idea inicial del proyecto. LoRa es una modulación de largo alcance y bajo consumo diseñada para comunicaciones de baja tasa de datos, muy adecuada para sensores que envían medidas periódicas y no necesitan transmitir grandes volúmenes de información [Semtech Corporation, 2015, Augustin et al., 2016]. En un sistema agrícola, esta característica resulta interesante

porque las estaciones suelen transmitir pocos datos: humedad del suelo, temperatura, humedad relativa, luminosidad, presión atmosférica y estado del actuador. Además, el enfoque de bajo consumo es coherente con estaciones alimentadas mediante batería o energía solar.

LoRaWAN añadía una capa de red sobre LoRa, proporcionando una arquitectura más estructurada para gestionar dispositivos, gateways y servidores de red [LoRa Alliance, 2020]. Desde el punto de vista académico, esta opción tenía mucho interés: permitía estudiar una tecnología ampliamente utilizada en IoT, analizar su arquitectura y plantear un despliegue de nodos de campo con comunicaciones de largo alcance. También era coherente con escenarios rurales donde no siempre hay conectividad WiFi y donde interesa cubrir distancias superiores a las habituales en redes domésticas.

Sin embargo, al trasladar esta alternativa al prototipo real aparecieron dificultades prácticas importantes. La primera era la necesidad de disponer de infraestructura adicional. En LoRa punto a punto bastaría con módulos compatibles en las estaciones y en el receptor, pero se perderían parte de las ventajas de una arquitectura LoRaWAN completa. En LoRaWAN, por el contrario, sería necesario contar con un gateway, configurar la red, gestionar la integración con un servidor LoRaWAN y conectar posteriormente esa información con el sistema de supervisión. Esta infraestructura aumentaba el alcance del proyecto y desplazaba una parte importante del esfuerzo hacia la propia red de comunicaciones.

También se detectó un riesgo de integración asociado a la selección de módulos. Para un despliegue en Europa es necesario trabajar en la banda adecuada, normalmente 868 MHz, y comprobar que los módulos, antenas, placas adaptadoras y niveles lógicos sean compatibles entre sí. En el caso de Arduino UNO, además, muchos módulos LoRa trabajan a 3,3 V, mientras que la placa funciona a 5 V, por lo que puede ser necesario añadir conversores de nivel y una alimentación de 3,3 V suficientemente estable. Estos detalles no invalidan la solución, pero introducen más puntos de fallo en una fase del proyecto donde el objetivo principal era validar una arquitectura completa de monitorización, almacenamiento, visualización y actuación.

Otro aspecto relevante fue la diferencia entre LoRa y LoRaWAN. Aunque a veces se mencionan juntos, no son equivalentes. LoRa define la capa física, mientras que LoRaWAN define una arquitectura de red con gateways, servidor de red y mecanismos de gestión de dispositivos [Adelantado et al., 2017]. Para el proyecto, esta distinción era importante porque una solución LoRa punto a punto habría sido más sencilla, pero menos representativa de una red IoT completa; y una solución LoRaWAN habría sido

más completa, pero también más costosa en tiempo e infraestructura.

La alternativa basada en LTE se planteó como una forma de reducir ese riesgo de integración. En lugar de desplegar una infraestructura radio propia, cada estación utiliza un módulo A7670E-LASA con tarjeta SIM para conectarse directamente a la red móvil. La comunicación con el exterior se realiza mediante comandos AT y peticiones HTTP, lo que permite enviar datos a una plataforma cloud sin necesidad de gateway propio. Esta decisión simplifica la arquitectura de comunicaciones: la estación no necesita que exista una Raspberry Pi cercana ni una pasarela local en la parcela, sino únicamente cobertura móvil suficiente y una configuración correcta del APN.

La elección de LTE no elimina las restricciones técnicas. El módulo requiere una alimentación más exigente que una solución LoRa, especialmente por los picos de corriente durante el registro en red o la transmisión de datos. Además, el sistema pasa a depender de la cobertura del operador móvil, de la disponibilidad de una tarjeta SIM y del acceso a Internet. Estas limitaciones son relevantes y deben tenerse en cuenta en el diseño energético y en el modo seguro de funcionamiento. Si la estación no puede comunicar, el sistema no debe quedar en un estado peligroso ni mantener el riego activo de forma indefinida.

Aun así, para el alcance concreto del TFG, LTE ofrecía una ventaja decisiva: permitía validar antes el ciclo completo de publicación y supervisión. Las estaciones podían leer sensores, enviar telemetría mediante HTTP, integrarse con ThingSpeak y dejar sus datos disponibles para que la Raspberry Pi los consultara posteriormente. Esto encaja con la arquitectura final de TierraViva, en la que la Raspberry Pi no está conectada físicamente a las estaciones ni decide el riego, sino que actúa como sistema de supervisión e histórico. La comunicación entre estaciones y sistema de supervisión, por tanto, no se resuelve mediante USB, puerto serie o red local, sino mediante la publicación y consulta de datos a través de Internet y de la plataforma intermedia.

La Tabla 3.8 resume los criterios principales considerados en la decisión.

Criterio	LoRa/LoRaWAN	LTE con A7670E-LASA
Alcance	Muy adecuado para largas distancias con baja tasa de datos.	Depende de la cobertura de red móvil disponible.
Consumo	Bajo consumo, especialmente en envíos periódicos.	Mayor consumo y picos de corriente durante transmisión.
Infraestructura	Puede requerir gateway, servidor LoRaWAN o receptor propio.	No requiere gateway propio; usa red móvil existente.
Integración cloud	Necesita pasarela o backend intermedio.	Integración directa mediante HTTP.
Complejidad inicial	Mayor si se despliega LoRaWAN completo.	Menor para validar telemetría HTTP.
Coste operativo	Sin SIM ni tarifa de datos si la red es propia.	Requiere tarjeta SIM y servicio de datos.
Riesgo para el TFG	Riesgo de consumir mucho tiempo en la red.	Riesgo menor para obtener un prototipo funcional completo.
Adecuación al prototipo	Muy interesante como línea futura o alternativa avanzada.	Adecuada para validar el sistema completo en el alcance actual.

Tabla 3.8: Comparación entre LoRa/LoRaWAN y LTE para TierraViva

La decisión final fue utilizar LTE con el módulo A7670E-LASA. Esta elección no implica que LoRa/LoRaWAN sea una alternativa peor desde el punto de vista tecnológico. De hecho, para una versión futura orientada a bajo consumo y despliegue en parcelas sin cobertura móvil, LoRa o LoRaWAN podrían ser opciones muy adecuadas. La decisión se basa en el contexto del proyecto: se buscaba una solución funcional, verificable y defendible dentro del tiempo disponible, priorizando la integración completa del sistema frente al desarrollo de una infraestructura de comunicaciones propia.

La validación preliminar del módulo LTE reforzó esta decisión. Antes de integrarlo con Arduino, el A7670E-LASA se probó mediante comandos AT desde un ordenador. Las pruebas permitieron comprobar que el módulo respondía correctamente, reconocía la SIM, obtenía señal, se registraba en red LTE, disponía de dirección IP y podía realizar peticiones HTTP. Estos resultados no sustituyen a las pruebas de integración completas, que se desarrollan más adelante, pero sí permitieron confirmar que la vía LTE era viable para el prototipo.

En términos de diseño, la consecuencia principal de esta elección es que ThingSpeak se convierte en una capa intermedia de comunicación. Las estaciones publican su telemetría y estado mediante HTTP, y la Raspberry Pi consulta esos datos desde la

nube para almacenarlos, visualizarlos y supervisar el funcionamiento del prototipo. La decisión de riego no depende de un flujo descendente de órdenes, sino de reglas locales ejecutadas en cada estación. Este enfoque reduce la dependencia de la conectividad externa para la actuación física y reserva la comunicación remota para telemetría, estado, histórico y supervisión.

Por tanto, la solución LTE adoptada responde a un criterio de ingeniería pragmático. LoRa/LoRaWAN se estudió por su adecuación a IoT agrícola, bajo consumo y largo alcance, pero se descartó como solución principal por el riesgo de aumentar demasiado la complejidad de comunicaciones. LTE con A7670E-LASA permite centrar el esfuerzo en el objetivo global de TierraViva: construir un prototipo capaz de medir, comunicar, almacenar, visualizar y actuar de forma segura mediante decisión local en las estaciones.

3.9. Elección de plataforma cloud: ThingSpeak

La elección de la plataforma cloud se ha realizado teniendo en cuenta el papel que debía desempeñar dentro de TierraViva. En este proyecto, la nube no se utiliza como un elemento añadido únicamente para representar gráficas, sino como una capa intermedia entre las estaciones de campo y el sistema de supervisión. Las estaciones deben poder enviar telemetría desde una ubicación remota mediante LTE y la Raspberry Pi debe poder consultar esos datos sin estar conectada físicamente a ellas. En la arquitectura final, esta capa no se utiliza para enviar órdenes de riego, sino para publicar telemetría, estado de relé y códigos de funcionamiento generados por las propias estaciones.

En el capítulo anterior se describieron varias alternativas de plataformas cloud para IoT, como Ubidots, Adafruit IO, Blynk, ThingsBoard o soluciones basadas en un broker MQTT propio. Todas ellas podrían ser válidas en determinados escenarios, pero no ofrecen el mismo equilibrio entre sencillez, control, coste, curva de aprendizaje y facilidad de integración con un prototipo académico. En este TFG no se buscaba desplegar una infraestructura cloud compleja, sino disponer de una plataforma suficientemente fiable y accesible para validar el ciclo completo de monitorización y control.

Una alternativa posible habría sido utilizar un broker MQTT accesible desde Internet. Esta opción resulta atractiva desde el punto de vista técnico, ya que MQTT está diseñado para comunicaciones IoT y permite un modelo de publicación/suscripción más adecuado para sistemas con múltiples nodos. Sin embargo, en el caso concreto de TierraViva habría implicado añadir un servidor o servicio externo, configurar

autenticación, gestionar reconexiones y adaptar la comunicación del módulo LTE a ese esquema. Para un prototipo cuyo objetivo principal es validar una arquitectura completa, esta complejidad adicional no aportaba una mejora proporcional.

También se consideraron plataformas más completas, como ThingsBoard, que ofrecen más posibilidades de gestión de dispositivos, reglas, visualización y escalabilidad. No obstante, precisamente por ser más completas, requieren una configuración inicial mayor. En una solución industrial o en un despliegue con muchos dispositivos, esta inversión puede estar justificada. En TierraViva, donde el número de estaciones es reducido y la prioridad es demostrar el funcionamiento extremo a extremo, una plataforma más ligera resulta más adecuada.

ThingSpeak se ha seleccionado porque proporciona una estructura sencilla basada en canales y campos, una API HTTP fácil de utilizar desde dispositivos embebidos y herramientas básicas de visualización. Según la documentación de MathWorks, ThingSpeak permite escribir datos en canales mediante llamadas HTTP de la API REST, además de otros métodos como MQTT o funciones de MATLAB [MathWorks, 2026b, MathWorks, 2026c]. Esta característica encaja especialmente bien con el módulo A7670E-LASA, ya que el envío de datos puede implementarse mediante comandos AT y peticiones HTTP, sin necesidad de incorporar una pila de comunicación más compleja en el firmware de Arduino.

El concepto de canal resulta especialmente útil para organizar la información del sistema. Cada canal puede agrupar varias variables relacionadas mediante campos numerados. En TierraViva, este modelo permite representar de forma clara las medidas y estados de una estación: humedad del suelo, temperatura, humedad relativa, luminosidad, presión atmosférica, estado del relé, código de estado o evento y un campo auxiliar reservado para depuración o ampliaciones. De esta forma, cada publicación de telemetría queda almacenada como una entrada temporal asociada a un conjunto de campos.

Elemento ThingSpeak	en	Uso en TierraViva	Motivo de utilización
Canal TierraViva_A		Registro de telemetría y estado de la estación A.	Permite separar los datos de la estación A y consultar su histórico de forma independiente.
Canal TierraViva_B		Registro de telemetría y estado de la estación B.	Permite separar los datos de la estación B y facilita el crecimiento a varias zonas monitorizadas.
Field 1		Humedad del suelo normalizada (%).	Valor calculado por el firmware a partir de la lectura bruta del sensor capacitivo y utilizado como variable principal para la decisión local de riego.
Field 2		Temperatura ambiente.	Variable de contexto para interpretar las condiciones del entorno.
Field 3		Humedad relativa ambiente.	Variable ambiental útil para contextualizar evaporación y condiciones de humedad.
Field 4		Luminosidad.	Permite interpretar exposición solar y condiciones de luz.
Field 5		Presión atmosférica.	Dato ambiental adicional proporcionado por el sensor BME280.
Field 6		Estado del relé, 0 apagado y 1 activo.	Permite supervisar si la estación está actuando sobre el riego.
Field 7		Modo seguro.	Indica si la estación se encuentra en modo seguro: 0 para funcionamiento normal y 1 para modo seguro activo.
Field 8		Código de estado.	Código numérico publicado por la estación para indicar el resultado del ciclo de funcionamiento o una incidencia detectada.
API HTTP		Escritura de telemetría desde las estaciones y lectura desde Raspberry Pi.	Reduce la complejidad de integración frente a soluciones que requieren infraestructura propia.

Tabla 3.9: Uso previsto de ThingSpeak dentro de TierraViva

La Tabla 3.9 resume el papel de ThingSpeak en la arquitectura. La separación por canales de estación evita mezclar los datos de distintas zonas monitorizadas y permite

que la Raspberry Pi consulte de forma independiente la telemetría y el estado de cada nodo. Esta decisión mejora la trazabilidad del sistema y facilita la depuración, ya que se puede comprobar por separado si cada estación está enviando medidas correctamente, si publica su estado operativo y si se han registrado eventos de riego o incidencias.

En la estructura actual se utilizan canales diferenciados para cada estación: *TierraViva_A* y *TierraViva_B*. Durante el diseño se valoró utilizar ThingSpeak también como mecanismo de control descendente, pero esa opción se descartó al desplazar la decisión de riego al firmware de cada estación. De este modo, la plataforma queda reservada para telemetría y estado, y se evita que una entrada antigua en la nube pueda condicionar una actuación física.

Otra razón importante para elegir ThingSpeak es la facilidad de lectura desde la Raspberry Pi. La Raspberry Pi puede consultar los últimos datos publicados mediante peticiones HTTP y recibir la información en formato estructurado. La documentación de MathWorks permite leer datos de un canal mediante la API y obtener respuestas en formatos como JSON o CSV [MathWorks, 2026a]. Esto encaja con la arquitectura software planteada para la Raspberry, donde los datos recibidos se almacenan posteriormente en SQLite y se presentan mediante una API y un dashboard propios.

ThingSpeak también permite mantener una primera visualización de los datos directamente en la plataforma. Aunque el dashboard final de *TierraViva* se desarrolla en la Raspberry Pi, las gráficas de ThingSpeak resultan útiles durante la fase de pruebas, porque permiten comprobar rápidamente si los datos están llegando a la nube antes de integrarlos en el resto del sistema. Durante las pruebas, disponer de esa doble visualización ayuda a acotar el fallo: si una lectura aparece en ThingSpeak pero no en el dashboard local, el problema estará probablemente en la lectura desde la Raspberry o en el almacenamiento; si no aparece en ThingSpeak, el fallo estará antes, en la estación, en el módulo LTE o en la petición HTTP.

La plataforma también presenta limitaciones que deben asumirse. ThingSpeak no se comporta como un sistema de tiempo real estricto. El envío y la consulta de datos se basan en peticiones y en intervalos de actualización, por lo que existe una latencia asociada. En la arquitectura final de *TierraViva* esta limitación no compromete la actuación sobre el riego, ya que la decisión crítica se ejecuta localmente en la estación y no depende de recibir una instrucción externa de riego.

En el caso de *TierraViva*, la latencia de ThingSpeak es aceptable porque su papel principal es registrar telemetría, estado y eventos para supervisión posterior. El riego no requiere que la Raspberry responda en milisegundos, sino que la estación aplique

reglas verificables, limite el tiempo de bomba activa y publique información suficiente para reconstruir lo ocurrido. Por ello, resulta más importante diseñar correctamente el firmware local y los códigos de estado que reducir al mínimo la latencia de consulta desde la Raspberry Pi.

La elección de ThingSpeak, por tanto, responde a un criterio de equilibrio. No es la plataforma más completa ni la más flexible para un despliegue industrial, pero sí es adecuada para un prototipo académico donde se necesita integrar sensores, LTE, almacenamiento remoto, consulta desde Raspberry Pi y supervisión de estados publicados por estaciones autónomas. Su principal ventaja es que permite validar la arquitectura completa sin desplazar el esfuerzo hacia el mantenimiento de una infraestructura cloud propia.

En conjunto, ThingSpeak actúa como punto de desacoplo entre las estaciones y la Raspberry Pi. Las estaciones no necesitan conocer la ubicación de la Raspberry ni mantener una conexión directa con ella; publican telemetría, estado del relé y códigos de evento en la nube. La Raspberry, por su parte, consulta esa información para construir el histórico, alimentar la API y servir el dashboard, sin intervenir en la decisión local de riego. Esta separación es coherente con el objetivo de TierraViva: construir un sistema modular, verificable y preparado para funcionar con estaciones remotas autónomas.

La estructura concreta de canales utilizada en TierraViva se desarrolla con mayor detalle en el Capítulo 4, donde se documentan los canales TierraViva_A y TierraViva_B, así como la codificación de los campos empleados para telemetría, estado y eventos.

3.10. Elección de arquitectura software de supervisión en Raspberry Pi

La Raspberry Pi se ha seleccionado como sistema de supervisión software de TierraViva. Su función no es sustituir a las estaciones de campo, leer directamente los sensores ni decidir la activación de la bomba, sino consultar la información publicada por las estaciones, registrar un histórico local y ofrecer una interfaz de consulta. Esta separación resulta importante porque las estaciones se encuentran en campo y se comunican de forma remota, mientras que la Raspberry actúa como nodo de supervisión e histórico.

Durante el desarrollo se implementó primero un prototipo local basado en MQTT, SQLite, FastAPI y un dashboard web. Esta fase previa permitió validar la cadena software sin depender todavía de la comunicación LTE ni de la plataforma cloud final. En ese prototipo, los datos llegaban a la Raspberry, se publicaban en un broker MQTT

local, eran procesados por un backend en Python, se almacenaban en SQLite y se exponían mediante una API para que el dashboard pudiera consultarlos.

Este prototipo local no representa la arquitectura final de comunicación con las estaciones remotas, pero sí tuvo un papel importante dentro del proceso de desarrollo. Permitió comprobar ante el tutor que la Raspberry Pi podía recibir telemetría, almacenarla, servirla mediante una API y mostrarla en una interfaz web. El código de esta fase se conserva en el repositorio del proyecto dentro del directorio de prototipos, dejando explícito que se trata de una validación previa y no del sistema final basado en LTE y ThingSpeak [Tejedor de la Cruz, 2026].

El uso de MQTT en esta fase se justificó por su sencillez para desacoplar productores y consumidores de información. Un proceso podía publicar lecturas de sensores y otro proceso podía escucharlas sin necesidad de una conexión directa entre ambos. Sin embargo, al definir la arquitectura final con estaciones remotas conectadas mediante LTE y ThingSpeak, el broker MQTT local dejó de ser el mecanismo principal de comunicación con las estaciones. Mantener MQTT como solución remota habría requerido desplegar o contratar un broker accesible desde Internet, gestionar autenticación, reconexiones y suscripciones desde el módulo LTE, aumentando la complejidad del prototipo. Por este motivo, MQTT se conserva como parte del prototipo local previo y como experiencia de validación, pero no como núcleo de la comunicación final entre Raspberry y estaciones.

El almacenamiento local se ha planteado mediante SQLite. SQLite es adecuado para este proyecto porque permite disponer de una base de datos ligera, autocontenida y sin necesidad de desplegar un servidor de base de datos independiente. La documentación oficial lo describe como un motor SQL pequeño, rápido y autocontenido, integrado directamente en la aplicación [SQLite, 2026]. Para TierraViva, esto permite guardar lecturas, estados y eventos de riego en la propia Raspberry Pi, manteniendo un histórico independiente de la plataforma cloud.

Esta independencia respecto a la nube tiene valor técnico. ThingSpeak almacena la telemetría recibida y facilita su consulta, pero no debe ser el único punto donde se conserve la información del sistema. Al replicar los datos relevantes en SQLite, la Raspberry puede mantener un histórico propio, adaptar el modelo de datos a las necesidades del proyecto y registrar incidencias de supervisión, como periodos sin datos, campos ausentes o cambios inesperados en el estado de una estación. De esta forma, el sistema de supervisión no se limita a visualizar datos externos, sino que construye su propio registro operativo.

La API se ha desarrollado con FastAPI. Esta herramienta permite construir

servicios web en Python de forma sencilla, utilizando rutas HTTP para exponer los datos que necesita el dashboard. La documentación oficial presenta FastAPI como un framework moderno para construir APIs con Python a partir de anotaciones de tipo estándar [FastAPI, 2026]. En el proyecto, su papel es servir como capa intermedia entre la base de datos y la interfaz web: consulta SQLite, transforma los registros en una respuesta estructurada y permite que el dashboard obtenga las últimas lecturas mediante peticiones HTTP.

En las pruebas realizadas, la API se ejecutó mediante Uvicorn, un servidor ASGI para aplicaciones Python [Uvicorn, 2026]. Esta decisión separa la definición de la aplicación, implementada en FastAPI, del proceso que la sirve por red. Desde el punto de vista del prototipo, esto facilita probar la API desde un navegador o desde otro equipo de la misma red, comprobar los endpoints disponibles y depurar de forma independiente la parte de visualización.

El dashboard web constituye la capa de presentación. Su objetivo es mostrar al usuario el estado del sistema sin obligarle a consultar manualmente la base de datos, revisar mensajes de consola o interpretar directamente las respuestas de ThingSpeak. En una primera versión, el dashboard se centró en mostrar las últimas lecturas almacenadas. Posteriormente, se amplió para incluir variables como humedad de suelo, luminosidad y presión, además de temperatura y humedad ambiental. Esta evolución permitió acercar la interfaz a las necesidades reales del sistema, no solo a una demostración mínima de comunicación.

La arquitectura software de la Raspberry se organiza, por tanto, en varias capas diferenciadas: lectura de datos desde la plataforma cloud, almacenamiento local, API y dashboard. Esta separación evita que toda la lógica quede mezclada en un único programa. También facilita las pruebas: se puede comprobar por separado si ThingSpeak entrega datos, si SQLite almacena correctamente, si la API devuelve la información esperada y si el dashboard representa bien el estado del sistema.

Bloque software	Función en TierraViva	Motivo de elección
Prototipo MQTT local	Validación inicial de recepción y tratamiento de datos en Raspberry.	Permite documentar una fase previa de desarrollo y separar prototipos de la arquitectura final.
Ingesta desde ThingSpeak	Consulta periódica de telemetría y estado publicados por las estaciones.	Permite alimentar el histórico local y el dashboard sin conexión directa con los nodos de campo.
SQLite	Almacenamiento local de lecturas, estados e incidencias.	Base de datos ligera, autocontenida y suficiente para el prototipo.
FastAPI	Exposición de datos mediante endpoints HTTP.	Facilita crear una API sencilla entre la base de datos y el dashboard.
Dashboard web	Visualización de medidas, histórico, estado del relé y estado de las estaciones.	Permite supervisar el prototipo sin acceder a consola o consultas SQL.

Tabla 3.10: Bloques principales de la arquitectura software de supervisión en Raspberry Pi

La lógica de decisión de riego no se mantiene en la Raspberry Pi en la arquitectura final. Aunque este dispositivo tiene capacidad suficiente para comparar estaciones o aplicar reglas más complejas, se ha decidido desplazar la decisión crítica al nodo de campo. Esta elección reduce la dependencia de la red, elimina la necesidad de enviar comandos remotos y permite que cada estación mantenga un comportamiento seguro aunque la Raspberry no esté disponible.

La Raspberry conserva un papel relevante dentro del sistema, pero orientado a supervisión. Puede detectar si una estación deja de publicar datos, mostrar el último estado conocido del relé, conservar históricos y facilitar el análisis posterior de las pruebas. De esta forma, el sistema mantiene trazabilidad sin convertir la supervisión en un requisito para que el riego pueda ejecutarse.

Una ventaja adicional de mantener un histórico local es la posibilidad de analizar el comportamiento del sistema más allá de la última lectura recibida. El histórico permite revisar tendencias, detectar periodos sin comunicación, comparar estaciones y documentar pruebas. También deja preparada la arquitectura para futuras mejoras, como reglas más avanzadas, detección de anomalías o modelos predictivos basados en datos acumulados. En esta versión del TFG, estas posibilidades no se presentan como inteligencia artificial implementada de forma completa, sino como una evolución razonable cuando exista un volumen suficiente de datos fiables.

Con esta organización, la Raspberry Pi mantiene un papel claro dentro del sistema: no controla físicamente el riego, pero sí convierte los datos publicados por las estaciones en información consultable y persistente. SQLite evita desplegar una base de datos pesada, FastAPI proporciona una API clara y el dashboard facilita la supervisión. El prototipo local con MQTT permitió validar esta cadena software en una fase temprana, mientras que la arquitectura final utiliza ThingSpeak como capa intermedia con las estaciones remotas. De esta forma, la Raspberry queda definida como sistema de supervisión: consulta, registra y expone información, mientras que la decisión y la ejecución segura de la actuación permanecen en la estación de campo.

3.11. Materiales empleados

La selección de materiales se ha realizado buscando un equilibrio entre funcionalidad, coste, disponibilidad y facilidad de integración. Al tratarse de un prototipo de TFG, no se han empleado componentes industriales cerrados, sino módulos habituales en sistemas IoT y prototipos electrónicos que permiten validar la arquitectura de forma progresiva y reproducible.

Los materiales empleados no deben entenderse como una simple lista de compra, sino como el conjunto de bloques físicos necesarios para que TierraViva pueda medir, comunicar, almacenar, visualizar y actuar sobre el riego. Cada componente cumple una función concreta dentro de la arquitectura y se ha seleccionado en coherencia con los requisitos definidos anteriormente.

En la Tabla 3.11 se resumen los principales materiales utilizados o previstos en el prototipo.

Componente	Función	Justificación
Sensórica		
Sensor capacitivo de humedad de suelo	Medición de humedad del sustrato mediante lectura analógica.	Variable principal para la decisión de riego; menor degradación que sensores resistivos.
BME280	Medición de temperatura, humedad relativa y presión atmosférica.	Sensor ambiental integrado; reduce el número de módulos y simplifica el cableado.
BH1750 / GY-302	Medición de luminosidad ambiente.	Medida digital en lux mediante I2C; útil para interpretar exposición solar.
Control y procesamiento		
Arduino UNO	Adquisición de datos y control local de la estación.	Plataforma sencilla, disponible, documentada y suficiente para lectura de sensores y control de relé.
Raspberry Pi 5	Backend de supervisión, histórico local, API y dashboard.	Mayor capacidad software para ejecutar servicios, almacenar datos y visualizar el sistema.
Comunicación		
A7670E-LASA	Conectividad LTE de la estación.	Permite envío HTTP mediante red móvil sin necesidad de gateway propio.
Tarjeta SIM de datos	Acceso a red móvil para el módulo LTE.	Necesaria para publicar telemetría y estado desde las estaciones.
Actuación		
Relé + bomba de agua 12 V	Activación física del riego durante un tiempo controlado.	Permite cerrar el ciclo de monitorización y control, manteniendo la bomba apagada como estado seguro.
Alimentación y montaje		
Alimentación 12 V + LM2596 (regulador)	Alimentación principal y adaptación de tensión a 5 V.	Permite alimentar la bomba y regular la tensión para Arduino, sensores y módulos compatibles.
Cableado, protoboard, conectores, resistencias y condensadores	Interconexión, adaptación eléctrica y estabilización del prototipo.	Facilitan pruebas, depuración, adaptación de niveles y reducción de inestabilidades en la alimentación.

Tabla 3.11: Resumen de materiales principales empleados en TierraViva

El bloque de sensórica está formado por tres tipos de sensores. El sensor capacitivo

de humedad de suelo se utiliza como entrada principal para la decisión de riego, ya que proporciona información directa sobre el estado del sustrato. El BME280 permite obtener temperatura, humedad relativa y presión atmosférica en un único módulo, aportando contexto ambiental sin aumentar demasiado la complejidad del montaje. Por último, el BH1750 mide la luminosidad ambiente en lux, lo que permite interpretar mejor la exposición solar de la zona monitorizada.

La elección de Arduino UNO como plataforma de estación responde a una decisión de simplicidad y control local. La estación necesita leer sensores, comunicarse con el módulo LTE y accionar un relé, pero no ejecutar una base de datos ni servir una interfaz web. Para esas tareas, Arduino UNO resulta suficiente y facilita la depuración del prototipo. Además, permite trabajar de forma directa con entradas analógicas, bus I2C y salidas digitales.

El módulo A7670E-LASA se incorpora como solución de comunicación remota mediante red móvil. Su función es permitir que cada estación publique telemetría y estado en la plataforma cloud. Esta elección evita desplegar un gateway propio y permite integrar la estación con servicios externos mediante peticiones HTTP, aunque introduce dependencia de cobertura móvil, tarjeta SIM y una alimentación adecuada para los picos de consumo del módulo.

La Raspberry Pi 5 se utiliza como sistema de supervisión software. A diferencia de Arduino, su papel no es interactuar directamente con los sensores ni accionar la bomba mediante pines físicos, sino consultar datos, almacenarlos, exponer una API y servir el dashboard. Esta separación permite mantener una arquitectura distribuida: las estaciones trabajan en campo y la Raspberry actúa como nodo de supervisión e histórico.

El bloque de actuación está formado por un módulo relé y una bomba de agua de 12 V. La bomba se controla desde la estación de campo, no desde la Raspberry Pi de forma directa. Esta decisión es importante porque el actuador debe encontrarse físicamente en el lugar donde se realiza el riego. La decisión y ejecución segura de la actuación corresponde al firmware de la estación, que aplica reglas locales y mantiene la bomba apagada ante cualquier estado no verificable.

La alimentación se plantea a partir de una fuente o batería de 12 V, ya que la bomba requiere una tensión superior a la utilizada por la lógica de control. Para alimentar Arduino y los módulos compatibles se emplea un regulador LM2596 ajustado a 5 V. Esta separación entre alimentación de potencia y alimentación lógica permite integrar el actuador sin comprometer la estabilidad del resto de la electrónica, siempre que se mantenga una masa común y se verifiquen las tensiones antes de conectar los módulos.

También se consideran materiales auxiliares como cableado Dupont (sin soldadura), protoboard, conectores, resistencias y condensadores. Aunque puedan parecer componentes secundarios, son relevantes durante la fase de prototipado porque permiten modificar conexiones, aislar fallos, adaptar niveles eléctricos y estabilizar la alimentación de los módulos. En particular, la adaptación de nivel entre Arduino y el módulo LTE debe cuidarse para evitar aplicar señales de 5 V a entradas que trabajen a niveles inferiores. Del mismo modo, el uso de condensadores de desacoplo y estabilización ayuda a reducir caídas puntuales de tensión y comportamientos erráticos asociados a módulos con consumo variable, como el A7670E-LASA o el bloque de actuación.

Con estos materiales se puede construir una arquitectura completa sin recurrir a equipos industriales de alto coste. El detalle completo de componentes, referencias comerciales, conexiones eléctricas y estado de validación se recoge en los Anexos A y B.

3.12. Riesgos técnicos y medidas de mitigación

El diseño de TierraViva debe contemplar no solo el funcionamiento esperado del sistema, sino también los fallos que pueden aparecer durante la integración y el uso del prototipo. Esta consideración es especialmente importante porque el sistema incorpora actuación física sobre el riego mediante una bomba de agua. Una lectura incorrecta puede registrarse y descartarse, pero una actuación no controlada podría provocar consumo innecesario de agua, funcionamiento indebido de la bomba o pérdida de fiabilidad del prototipo.

Por este motivo, se han identificado los principales riesgos técnicos del sistema y se han definido medidas de mitigación asociadas. Estas medidas no eliminan por completo la posibilidad de fallo, pero reducen su impacto y permiten que el sistema adopte un comportamiento conservador ante situaciones no verificables. La Tabla 3.12 resume los riesgos considerados.

Código	Riesgo técnico	Impacto posible	Medidas de mitigación
RT-01	Fallo de alimentación	Reinicios, lecturas erróneas, fallo de comunicación o activación inestable del relé.	Regulación mediante LM2596, comprobación de tensiones, masa común y estabilización de alimentación.
RT-02	Picos de consumo del módulo LTE	Reinicio del A7670E-LASA o pérdida de conexión durante transmisión.	Fuente adecuada, condensadores de estabilización y pruebas previas del módulo por comandos AT.
RT-03	Regla local mal configurada	Activación del riego con umbral incorrecto o en condiciones no deseadas.	Umrales configurables, pruebas controladas, histéresis/cooldown y validación progresiva antes de riego real.
RT-04	Lecturas inestables cerca del umbral	Activaciones repetidas o comportamiento oscilante de la bomba.	Tiempo mínimo entre riegos, filtrado básico, descarte de valores incoherentes y límites de seguridad.
RT-05	Activación prolongada de la bomba	Riego indefinido o consumo innecesario de agua.	Tiempo máximo de activación, apagado forzado y estado seguro por defecto.
RT-06	Pérdida de comunicación	Ausencia temporal de telemetría en la plataforma o imposibilidad de supervisión remota.	Decisión local independiente, registro de estado, reintentos de comunicación y conservación del estado seguro.
RT-07	Pérdida de datos históricos	Imposibilidad de revisar pruebas, tendencias o eventos de riego.	Almacenamiento local en SQLite y consulta periódica de la plataforma cloud.
RT-08	Lecturas erróneas de sensores	Decisiones de riego basadas en datos no fiables.	Validación de rangos, descarte de datos incoherentes y registro de errores.
RT-09	Fallo de software en Raspberry Pi	Pérdida temporal de dashboard, API o histórico local.	Separación por módulos, logs, reinicio controlado de servicios y autonomía de decisión en las estaciones.

Tabla 3.12: Riesgos técnicos principales y medidas de mitigación

El fallo de alimentación es uno de los riesgos más importantes en sistemas con

sensores, comunicaciones y actuación. La estación debe alimentar elementos con necesidades distintas: Arduino, sensores, módulo LTE, relé y bomba de agua. Por ello, se plantea una alimentación principal de 12 V y una regulación a 5 V mediante LM2596 para la electrónica compatible. Antes de conectar los módulos, es necesario ajustar y comprobar la tensión de salida del regulador. Además, todos los elementos deben compartir una referencia común de masa para evitar comportamientos erráticos en las señales de control.

El módulo LTE introduce una dificultad adicional porque puede presentar picos de consumo durante el registro en red o el envío de datos. Si la alimentación no es suficientemente estable, el módulo puede reiniciarse o perder comunicación. Para mitigar este riesgo se han realizado pruebas previas del A7670E-LASA por USB mediante comandos AT, comprobando SIM, señal, registro en red, dirección IP y comunicación HTTP. En la integración final se incorporan condensadores de estabilización en la alimentación para reducir caídas puntuales de tensión y mejorar el comportamiento del sistema ante consumos variables durante la transmisión LTE.

La lógica local de riego constituye otro punto crítico. Al estar implementada en la estación, debe diseñarse de forma conservadora: no debe actuar ante lecturas imposibles, debe limitar el tiempo máximo de bomba activa y debe aplicar un periodo mínimo entre riegos. Esta decisión reduce la dependencia de la Raspberry Pi, pero exige que el firmware de estación incorpore las comprobaciones de seguridad necesarias.

La pérdida de comunicación se interpreta de forma distinta en esta arquitectura. Si una estación no puede publicar temporalmente en ThingSpeak, la supervisión remota se ve afectada, pero la estación no debe quedar bloqueada ni activar la bomba por esa causa. El comportamiento seguro debe seguir residiendo en el nodo de campo: ante una situación no verificable, la bomba permanece apagada y se registra el estado cuando la comunicación vuelve a estar disponible.

La lógica local de riego constituye otro punto crítico. Al estar implementada en la estación, debe diseñarse de forma conservadora: no debe actuar ante lecturas imposibles, debe limitar el tiempo máximo de bomba activa y debe aplicar un periodo mínimo entre riegos. Esta decisión reduce la dependencia de la Raspberry Pi, pero exige que el firmware de estación incorpore las comprobaciones de seguridad necesarias.

La activación de la bomba de agua se trata como el riesgo de mayor impacto físico. La regla de diseño es que la bomba debe permanecer apagada por defecto y solo activarse durante un tiempo limitado. Si la lectura de humedad no es válida, si el sistema está en cooldown, si se detecta una condición no verificable o si se supera el tiempo máximo permitido, la estación debe mantener o devolver el relé al estado

apagado. Esta lógica evita que un fallo de lectura, un reinicio o un error interno puedan mantener el riego activo indefinidamente.

Además, el bloque de actuación se ha tratado de forma separada respecto a la lógica de control. Durante las pruebas iniciales puede emplearse un LED o una carga de baja potencia para verificar el funcionamiento del relé sin accionar todavía la bomba real. En el montaje final, esa carga de prueba se sustituye por la bomba de agua de 12 V. Esta estrategia permite validar primero la lógica de activación y apagado, y después realizar la prueba con el actuador real en condiciones más controladas.

La pérdida de comunicación también se aborda con un criterio conservador. Si la estación no puede publicar temporalmente en la plataforma cloud, la supervisión remota se ve afectada, pero la decisión de seguridad sigue residiendo en el firmware local. La pérdida de comunicación puede registrarse como incidencia cuando sea posible, pero no debe dejar la bomba activa ni provocar una actuación no controlada.

El almacenamiento local en SQLite permite mitigar la pérdida de información operativa. Aunque ThingSpeak actúa como capa intermedia de telemetría y estado, la Raspberry conserva un histórico propio de lecturas, estados y eventos relevantes. Esto facilita revisar pruebas, detectar periodos sin comunicación y justificar posteriormente el comportamiento del sistema. Además, separar el almacenamiento local de la plataforma cloud reduce la dependencia de un único punto de consulta.

Por último, los fallos de software se mitigan mediante una arquitectura modular. La lectura de datos, el almacenamiento, la validación de datos, la API y el dashboard se han planteado como bloques diferenciados. Esta separación permite probar cada parte de forma independiente, registrar errores y depurar el sistema sin mezclar todas las responsabilidades en un único programa. En cualquier caso, aunque la Raspberry Pi o el dashboard de supervisión dejen de funcionar temporalmente, la estación de campo debe mantener su lógica local de seguridad y conservar la bomba apagada ante estados no verificables.

En conjunto, las medidas de mitigación adoptadas siguen un criterio común: ante un fallo, dato dudoso o estado no verificable, el sistema debe priorizar la seguridad frente a la actuación. Esta decisión puede provocar que el sistema no riegue en una situación ambigua, pero evita el escenario más peligroso para el prototipo: una activación no controlada de la bomba. De esta forma, TierraViva se plantea como un sistema funcional, pero también prudente y trazable.

Capítulo 4

Arquitectura final de TierraViva

Una vez definidos los requisitos, los materiales y las principales decisiones de diseño, este capítulo presenta la arquitectura final de TierraViva. A diferencia del capítulo anterior, donde se justifican las alternativas valoradas y los criterios de elección, aquí se describe cómo queda organizado el sistema completo y cómo interactúan sus distintos bloques durante el funcionamiento.

La arquitectura final se basa en estaciones de campo autónomas con Arduino UNO, sensores ambientales y de suelo, comunicación LTE mediante el módulo A7670E-LASA, una plataforma cloud intermedia basada en ThingSpeak y un sistema de supervisión implementado sobre Raspberry Pi 5. El sistema se ha diseñado para cubrir el ciclo completo de monitorización y actuación local: adquisición de datos, decisión de riego en la estación, actuación sobre la bomba, publicación de telemetría y estado, almacenamiento histórico, API y visualización mediante dashboard web.

El cambio principal respecto a planteamientos anteriores es que la Raspberry Pi no actúa como elemento central de decisión del riego. La decisión crítica se desplaza al firmware de cada estación, que evalúa sus propias lecturas y aplica reglas locales de seguridad. La Raspberry conserva un papel importante, pero orientado a supervisión, histórico y visualización.

4.1. Visión general de la arquitectura

La arquitectura final de TierraViva se organiza como un sistema IoT distribuido. Las medidas se obtienen en campo, la comunicación se realiza mediante red móvil, la información se publica en una plataforma cloud y la supervisión se ejecuta desde una Raspberry Pi 5. La actuación sobre el riego, sin embargo, no depende de una decisión externa enviada desde la Raspberry, sino de reglas locales ejecutadas en cada estación.

El sistema está formado por cuatro elementos principales: las estaciones autónomas de campo A y B, la plataforma ThingSpeak, la Raspberry Pi 5 y el dashboard web. Las estaciones se encargan de medir, decidir y actuar localmente. ThingSpeak funciona

como capa intermedia de telemetría y estado. La Raspberry Pi consulta los datos, mantiene un histórico local y expone una API. El dashboard permite supervisar el estado del sistema desde una interfaz web.

La Figura 4.1 muestra la arquitectura final del sistema.

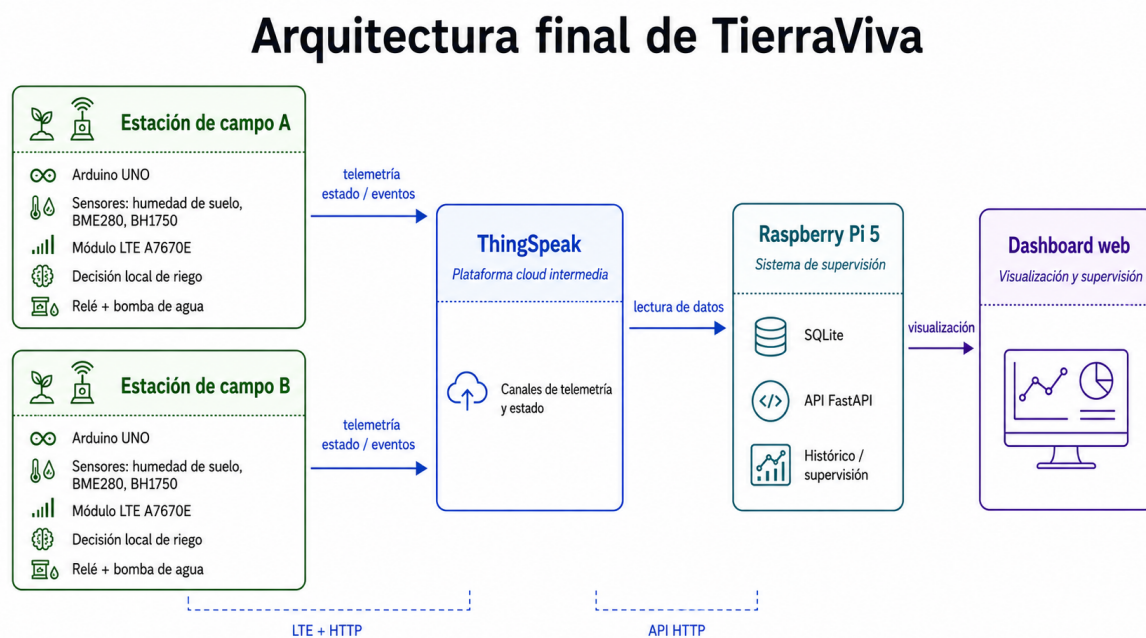


Figura 4.1: Arquitectura final de TierraViva con decisión local en las estaciones.

El flujo de funcionamiento comienza en las estaciones de campo. Cada estación toma lecturas de sus sensores, evalúa localmente si se cumplen las condiciones de riego y, si procede, activa la bomba de agua mediante el relé durante un tiempo limitado. Después publica en ThingSpeak tanto las medidas ambientales como el estado del relé y el código de estado correspondiente.

ThingSpeak actúa como punto de desacoplo entre los nodos de campo y el sistema de supervisión. Las estaciones no necesitan conocer la ubicación de la Raspberry ni mantener una conexión directa con ella; únicamente publican información en la nube. La Raspberry, por su parte, consulta periódicamente los canales de ThingSpeak y almacena los datos relevantes en su base de datos local.

La Raspberry Pi 5 funciona como sistema de supervisión. Su papel no es accionar directamente la bomba, leer sensores por cable ni decidir cuándo debe regarse. Su función es consultar la telemetría disponible, registrar el histórico en SQLite, exponer los datos mediante una API y servir el dashboard web. Así, una caída de la Raspberry no impide que la estación mantenga su comportamiento local de seguridad.

El dashboard web representa la capa de visualización. A partir de la información procesada por la Raspberry Pi, permite consultar las últimas lecturas, el estado de las

estaciones, el estado del relé y los estados de funcionamiento publicados por cada nodo. No participa directamente en la actuación, pero resulta fundamental para supervisar el comportamiento del prototipo y detectar posibles incidencias.

La arquitectura final puede resumirse de forma sencilla: las estaciones miden, deciden y actúan en campo; ThingSpeak comunica y almacena telemetría; la Raspberry registra y supervisa; y el dashboard permite visualizar. Esta organización facilita la validación por bloques, permite trabajar con más de una estación y mantiene una separación clara entre adquisición, comunicación, decisión local, actuación y supervisión.

4.2. Estaciones de campo A y B

Las estaciones de campo son los nodos autónomos de TierraViva. Su función dentro de la arquitectura final es adquirir datos del entorno, evaluar localmente reglas de riego, accionar la bomba cuando corresponda y publicar telemetría y estado en la plataforma cloud. En esta versión del sistema se contemplan dos estaciones, denominadas estación A y estación B, con una estructura hardware equivalente y una configuración lógica diferenciada.

Ambas estaciones siguen el mismo diseño base: Arduino UNO como controlador local, sensores de suelo y ambientales, módulo LTE A7670E-LASA para comunicación remota, relé para accionar la bomba de agua y alimentación principal a 12 V con regulación a 5 V mediante LM2596. La diferencia entre ellas no está en la arquitectura física, sino en el identificador de estación y el canal de ThingSpeak utilizado para publicar sus datos.

Elemento	Estación A	Estación B
Identificador lógico	TierraViva_A	TierraViva_B
Controlador local	Arduino UNO	Arduino UNO
Sensores	Humedad de suelo, BME280 y BH1750	Humedad de suelo, BME280 y BH1750
Comunicación	A7670E-LASA mediante LTE/HTTP	A7670E-LASA mediante LTE/HTTP
Canal ThingSpeak	Canal asociado a estación A	Canal asociado a estación B
Decisión de riego	Regla local en firmware	Regla local en firmware
Actuación	Relé y bomba de agua 12 V	Relé y bomba de agua 12 V
Alimentación	12 V con regulación a 5 V	12 V con regulación a 5 V

Tabla 4.1: Configuración lógica de las estaciones de campo

La Tabla 4.1 resume esta configuración. El objetivo de mantener estaciones

equivalentes es facilitar la reproducción del sistema: una vez validada la estación A, la estación B puede construirse replicando el mismo montaje y modificando únicamente los parámetros de identificación y comunicación. Esta decisión también simplifica la depuración, ya que cualquier diferencia de comportamiento entre estaciones puede analizarse a partir de una base hardware común.

Desde el punto de vista interno, cada estación se organiza en cuatro bloques funcionales. El primero es el bloque de adquisición, formado por el sensor capacitivo de humedad del suelo, el BME280 y el BH1750. El segundo es el bloque de decisión local, ejecutado en el firmware del Arduino. El tercero es el bloque de comunicación, formado por el A7670E-LASA y la lógica de publicación mediante HTTP. El cuarto es el bloque de actuación, formado por la salida digital del Arduino, el módulo relé y la bomba de agua de 12 V.

Bloque interno	Elementos implicados	Responsabilidad
Adquisición	Sensores y Arduino UNO	Obtener lecturas periódicas del suelo y del entorno.
Decisión local	Arduino UNO	Evaluar humedad del suelo, cooldown, errores y condiciones de seguridad.
Comunicación	Arduino UNO y A7670E-LASA	Publicar telemetría, estado y eventos en ThingSpeak.
Actuación	Arduino UNO, relé y bomba	Ejecutar el riego localmente durante un tiempo limitado.
Alimentación	Fuente 12 V, LM2596 y condensadores	Proporcionar tensión adecuada y estabilidad eléctrica al montaje.

Tabla 4.2: Bloques internos de una estación de campo

El firmware de estación debe mantener esta separación de responsabilidades. En primer lugar, realiza la lectura de sensores y genera una estructura de datos con las variables medidas. Después, comprueba si las lecturas son válidas y evalúa la regla local de riego. Si la humedad del suelo indica sequedad, no existe cooldown activo y no se detecta una condición de error, la estación puede activar el relé durante el tiempo máximo configurado.

La actuación se ejecuta siempre de forma local. Cuando una estación decide regar, activa el relé durante un tiempo limitado y posteriormente vuelve a dejar la bomba apagada. Durante las pruebas iniciales puede utilizarse una carga de baja potencia, como un LED, para comprobar la lógica de activación sin accionar todavía la bomba real. En el montaje final, esa carga de prueba se sustituye por la bomba de agua de 12 V.

La estación conserva la responsabilidad última sobre la ejecución segura. Antes de activar el relé debe comprobar que la lectura de humedad es válida, que la duración máxima no se supera, que no existe un error crítico y que no se está dentro del periodo mínimo entre riegos. Si alguna de estas comprobaciones falla, la estación mantiene la bomba apagada y publica un código de estado que permita diagnosticar la situación desde la Raspberry Pi.

La alimentación de cada estación parte de una tensión principal de 12 V, necesaria para el bloque de actuación. A partir de esa tensión se obtiene una línea de 5 V mediante el regulador LM2596 para alimentar la electrónica compatible. En el montaje se incorporan condensadores de estabilización para reducir inestabilidades asociadas a consumos variables, especialmente durante la comunicación LTE o los cambios de estado del bloque de actuación. El detalle de conexiones eléctricas se recoge en el Anexo B.

En resumen, las estaciones A y B son nodos de campo equivalentes desde el punto de vista hardware, pero diferenciados por configuración. Esta organización permite escalar el sistema sin rediseñar la arquitectura: añadir una nueva estación implicaría replicar el montaje, asignar un nuevo identificador, crear un nuevo canal de ThingSpeak y ajustar la lógica de supervisión para consultar ese nuevo origen de datos.

4.3. Arquitectura de comunicaciones LTE/HTTP

La comunicación de las estaciones de campo se basa en el módulo LTE A7670E-LASA y en el uso de peticiones HTTP hacia ThingSpeak. Esta arquitectura permite que cada estación publique telemetría y estado desde una ubicación remota sin depender de una red WiFi local ni de una conexión física con la Raspberry Pi. Desde el punto de vista funcional, el módulo LTE actúa como interfaz de salida a Internet de la estación.

El Arduino UNO no accede directamente a la red móvil. Su papel es comunicarse por UART con el A7670E-LASA mediante comandos AT. A través de esa interfaz serie, el firmware comprueba el estado del módulo, verifica la SIM, confirma el registro en red, configura el contexto de datos y lanza las peticiones HTTP necesarias para enviar información a la nube. Esta separación permite que el Arduino mantenga una lógica relativamente sencilla: preparar datos, aplicar la decisión local, enviarlos al módem y analizar la respuesta obtenida.

El flujo de comunicación se organiza en varias fases. La primera corresponde a la inicialización del módulo. En esta fase, la estación comprueba que el A7670E-LASA responde a comandos básicos y que la comunicación serie funciona correctamente.

Después, se verifica el estado de la SIM y el registro en la red móvil. Una estación no debería intentar enviar telemetría si el módulo no responde, si la SIM no está preparada o si no existe registro válido en red.

A continuación, se configura el contexto de datos. Para ello, el módulo debe disponer de un APN válido asociado a la tarjeta SIM utilizada. Este paso es necesario para que el módem pueda obtener conectividad IP. Una vez configurado el contexto y obtenida una dirección IP, la estación puede iniciar la secuencia HTTP.

La fase de envío consiste en construir una URL de actualización de ThingSpeak con los campos de telemetría correspondientes. Cada estación incluye sus medidas y estados en los campos definidos para su canal: humedad de suelo, temperatura, humedad relativa, luminosidad, presión, estado del relé, código de estado o evento y un campo auxiliar de diagnóstico. El A7670E-LASA realiza entonces una petición HTTP hacia la API de ThingSpeak.

Fase	Acción principal	Resultado esperado
Inicialización	Comprobar comunicación con el módulo mediante comandos AT.	El módulo responde correctamente y queda listo para continuar.
SIM	Verificar que la tarjeta SIM está disponible y sin PIN pendiente.	La SIM se encuentra preparada para registrarse en red.
Registro en red	Comprobar el registro LTE y la calidad de señal.	La estación confirma que existe cobertura móvil suficiente.
Contexto de datos	Configurar APN y comprobar dirección IP.	El módulo dispone de conectividad IP para realizar peticiones.
Envío HTTP	Construir la petición de actualización y enviarla a ThingSpeak.	La plataforma recibe una nueva entrada de telemetría y estado.
Gestión de respuesta	Analizar el código de respuesta y cerrar la sesión HTTP.	La estación confirma si el envío ha sido correcto o registra un error.

Tabla 4.3: Fases de comunicación LTE/HTTP en una estación de campo

La Tabla 4.3 resume el flujo seguido por la estación. Esta secuencia no se ejecuta únicamente como una lista fija de comandos AT, sino como una lógica de comunicación con comprobaciones intermedias. Si una fase falla, el firmware debe evitar continuar como si el envío hubiera sido correcto. Por ejemplo, no tiene sentido construir una petición HTTP si el módulo no está registrado en red o si no dispone de dirección IP.

En la comunicación con ThingSpeak se utiliza el modelo HTTP de petición-respuesta. La estación envía una petición de escritura al canal correspondiente y

espera una respuesta de la plataforma. En un envío correcto, ThingSpeak devuelve una confirmación que permite interpretar que la entrada ha sido registrada. Si la respuesta indica error, si no llega respuesta o si se agota el tiempo de espera, la estación debe tratar el envío como fallido.

La gestión de errores forma parte de la propia arquitectura de comunicaciones. Los fallos posibles no se limitan a una ausencia total de cobertura. También pueden aparecer reinicios del módulo por alimentación insuficiente, errores de APN, pérdida temporal de red, fallo de resolución o respuesta HTTP no válida. Por ello, el firmware debe diferenciar entre un dato no enviado y una actuación física sobre el riego. La pérdida temporal de comunicación puede impedir la supervisión remota, pero no debe dejar la bomba activa ni provocar una actuación no controlada.

La comunicación LTE/HTTP se emplea únicamente en sentido ascendente desde el punto de vista operativo de la estación: publicación de telemetría, estado y eventos. La Raspberry Pi realiza después lecturas de esos datos desde ThingSpeak, pero no envía órdenes de riego a las estaciones. Esta decisión simplifica el firmware, evita un flujo descendente de control y reduce el riesgo de que información antigua en la nube condicione una actuación física.

Esta arquitectura no proporciona comunicación en tiempo real estricto. La estación publica información de forma periódica, por lo que existe una latencia asociada a los intervalos de muestreo y envío. Sin embargo, para supervisión de riego esta latencia es aceptable. La decisión crítica no depende de la llegada inmediata de una orden externa, sino de la evaluación local realizada en la propia estación.

En conjunto, la arquitectura LTE/HTTP permite que las estaciones funcionen como nodos remotos independientes del sistema de supervisión físico. Cada estación puede publicar sus datos en la nube, mantener una lógica local de riego y permitir que la Raspberry Pi consulte posteriormente el estado del sistema sin conexión directa con el hardware de campo.

4.4. ThingSpeak como capa intermedia

ThingSpeak actúa en TierraViva como capa intermedia entre las estaciones de campo y la Raspberry Pi. Su función dentro de la arquitectura final es desacoplar la publicación de datos en campo de la supervisión posterior: las estaciones publican telemetría y estado mediante LTE/HTTP, mientras que la Raspberry Pi lee esa información, mantiene un histórico local y la muestra en el dashboard.

La estructura finalmente adoptada se apoya en dos canales diferenciados:

TierraViva_A y TierraViva_B. Cada canal corresponde a una estación de campo y contiene tanto las variables medidas como la información de estado generada por el firmware. En la arquitectura final no se utilizan canales de comandos, ya que la decisión de riego se realiza localmente en cada estación.

La Figura 4.2 muestra los canales creados en ThingSpeak para el prototipo.

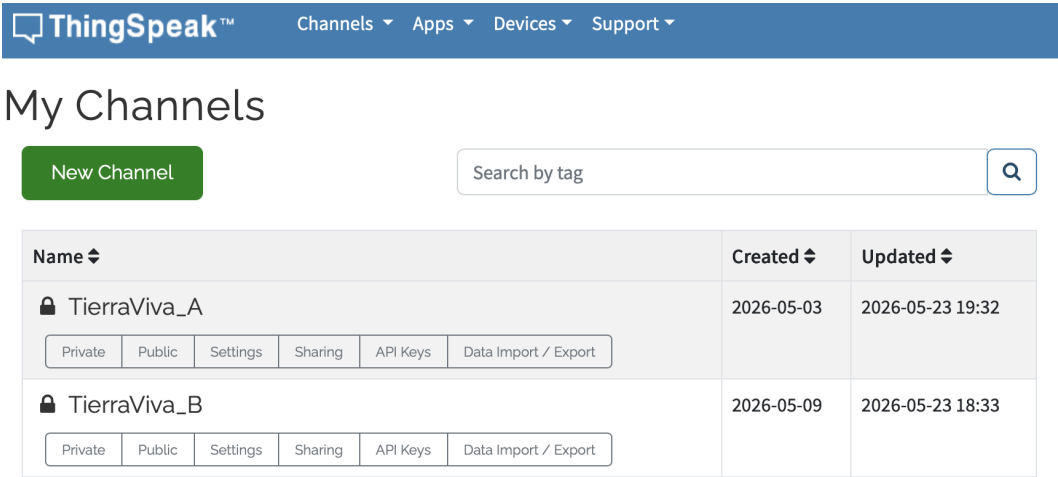


Figura 4.2: Canales ThingSpeak creados para las estaciones de TierraViva.

4.4.1. Canales de estación

Los canales TierraViva_A y TierraViva_B almacenan la telemetría generada por las estaciones A y B, respectivamente. Cada actualización enviada por una estación incluye medidas del entorno y datos de estado, lo que permite a la Raspberry Pi supervisar tanto las variables ambientales como el comportamiento operativo del nodo.

Campo	Dato	Descripción
field1	Humedad del suelo normalizada (%)	Valor calculado por el firmware a partir de la lectura bruta del sensor capacitivo. Se utiliza como entrada principal de la regla local de riego.
field2	Temperatura (°C)	Temperatura ambiente medida por el BME280.
field3	Humedad ambiente (%)	Humedad relativa medida por el BME280.
field4	Luminosidad (lux)	Nivel de luz ambiente medido por el BH1750.
field5	Presión (hPa)	Presión atmosférica medida por el BME280.
field6	Estado del relé	Estado del relé asociado al bloque de actuación: 0 para apagado y 1 para activo.
field7	Modo seguro	Indicador de seguridad de la estación: 0 para funcionamiento normal y 1 para modo seguro activo.
field8	Código de estado	Código numérico que resume el estado operativo del ciclo o la incidencia detectada.

Tabla 4.4: Asignación de campos en los canales ThingSpeak de estación

La Tabla 4.4 define el contrato de datos utilizado por las estaciones. Mantener la misma estructura en ambos canales simplifica el firmware y la lógica de supervisión, ya que la Raspberry puede aplicar el mismo proceso de lectura para la estación A y para la estación B, cambiando únicamente el identificador del canal y la clave de lectura correspondiente.

Esta estructura permite que el canal no se limite a representar medidas ambientales. En particular, los campos `field6`, `field7` y `field8` aportan información operativa sobre el estado del relé, la activación del modo seguro y el código de estado publicado por la estación. Gracias a ello, la Raspberry puede detectar si una estación ha actuado, si está en reposo o si ha publicado una incidencia.

La separación por canales evita mezclar datos de estaciones distintas y facilita la escalabilidad. Si en el futuro se añadiese una nueva estación, bastaría con replicar la estructura de campos en un nuevo canal, asignar las claves correspondientes y ampliar la lógica de supervisión para incluir ese nuevo origen de datos.

4.4.2. Descartes del control descendente desde la nube

En fases anteriores del diseño se valoró utilizar ThingSpeak también como mecanismo de control descendente desde la Raspberry hacia las estaciones. En ese planteamiento, el sistema de supervisión publicaría instrucciones en la nube y cada estación tendría que consultarlas periódicamente. Aunque la solución era viable, añadía complejidad: obligaba a gestionar identificadores, evitar repeticiones, descartar información obsoleta y garantizar que una entrada antigua no provocase una actuación no deseada.

En la arquitectura final se descarta ese flujo descendente de control. Cada estación decide localmente si debe regar a partir de sus propias lecturas, umbrales, cooldown y condiciones de seguridad. Esta decisión simplifica el sistema y reduce la dependencia de la Raspberry Pi. Si la Raspberry se apaga o pierde conectividad, la estación no queda esperando una orden externa para mantener su comportamiento seguro.

ThingSpeak se mantiene, por tanto, como plataforma de telemetría y estado. Su papel no es transportar órdenes de riego, sino permitir que las estaciones publiquen su información y que la Raspberry pueda consultarla posteriormente. Esta separación hace que la nube sea importante para la supervisión, pero no imprescindible para que la estación ejecute sus reglas locales de seguridad.

En conjunto, ThingSpeak se utiliza en TierraViva como una capa intermedia sencilla pero suficiente para sostener el flujo de información del sistema. Los canales de estación permiten almacenar medidas y estados, la API HTTP facilita la integración con el A7670E-LASA y la lectura desde Raspberry Pi permite construir un histórico propio y un dashboard de supervisión.

4.5. Raspberry Pi como sistema de supervisión

La Raspberry Pi 5 actúa como sistema de supervisión de TierraViva. Su papel dentro de la arquitectura final no es sustituir a las estaciones de campo, conectarse físicamente a los sensores ni decidir la activación de la bomba, sino consultar la información disponible en la nube, mantener un histórico local, exponer una API y servir el dashboard web.

Esta separación es importante para mantener la coherencia del sistema distribuido. Las estaciones se encuentran en campo y se comunican mediante LTE/HTTP con ThingSpeak. La Raspberry, por tanto, no recibe datos por USB ni por puerto serie desde las estaciones, sino que consulta periódicamente los canales publicados en la plataforma cloud. A partir de esos datos, construye una visión actualizada del estado

de cada estación.

El primer bloque funcional de la Raspberry es la lectura de datos desde ThingSpeak. Para cada estación, el sistema de supervisión consulta su canal correspondiente y obtiene las últimas medidas disponibles: humedad de suelo, temperatura, humedad relativa, luminosidad, presión atmosférica, estado del relé, código de estado y campo auxiliar. Esta lectura permite conocer tanto el estado ambiental de la zona monitorizada como el estado operativo de la estación.

Una vez recuperados los datos, la Raspberry debe validarlos antes de almacenarlos o mostrarlos. No todas las respuestas recibidas tienen por qué ser válidas: puede haber campos vacíos, valores fuera de rango, lecturas antiguas o errores de comunicación. Por ello, el sistema de supervisión debe comprobar que los datos recibidos son coherentes y que la marca temporal de la última lectura se encuentra dentro de un intervalo razonable. Si una estación no actualiza datos durante demasiado tiempo, el dashboard puede mostrarla como no disponible u obsoleta.

El segundo bloque funcional es el almacenamiento local en SQLite. Aunque ThingSpeak conserva las entradas publicadas en sus canales, TierraViva mantiene un histórico propio en la Raspberry Pi. Este histórico permite registrar las lecturas recibidas, los estados de relé, los eventos de riego publicados por las estaciones y las incidencias detectadas por el sistema de supervisión. De esta manera, la Raspberry no actúa únicamente como un visor de la nube, sino como un sistema con memoria propia del comportamiento del prototipo.

Bloque	Función	Resultado esperado
Lectura de ThingSpeak	Consulta periódica de los canales de las estaciones.	Últimas lecturas y estados disponibles.
Validación de datos	Comprobación de campos, rangos y antigüedad de las lecturas.	Detección de datos inválidos, ausentes u obsoletos.
SQLite	Almacenamiento local de lecturas, estados y eventos.	Histórico propio del sistema.
FastAPI	Exposición de datos mediante endpoints HTTP.	Respuestas estructuradas para el dashboard.
Dashboard web	Visualización del estado del sistema.	Interfaz de supervisión para el usuario.

Tabla 4.5: Funciones principales de la Raspberry Pi dentro de TierraViva

La API se desarrolla con FastAPI y actúa como capa intermedia entre la base de datos y el dashboard web. En lugar de que la interfaz consulte directamente SQLite o ThingSpeak, realiza peticiones a la API para obtener las últimas lecturas, el estado de

las estaciones o el histórico necesario. Esta separación facilita la evolución del dashboard sin modificar la lógica de almacenamiento o consulta.

El dashboard web constituye la parte visible del sistema de supervisión. A través de él se pueden consultar las últimas medidas recibidas, comprobar el estado de cada estación, observar si existe comunicación reciente y revisar el estado del relé o los eventos de riego publicados por las estaciones. Su función no es ejecutar directamente acciones sobre el hardware, sino permitir la supervisión del sistema de forma clara. Esto resulta útil tanto durante las pruebas como en una posible utilización real del prototipo.

Además de almacenar lecturas, la Raspberry puede registrar incidencias de supervisión. Por ejemplo, puede detectar que una estación lleva demasiado tiempo sin publicar datos, que un campo llega vacío, que una lectura está fuera de rango o que el estado del relé no cambia durante un periodo esperado. Estos eventos no sustituyen a la lógica de seguridad local, pero ayudan a documentar el comportamiento del sistema y facilitan la validación experimental.

El ciclo completo de funcionamiento de la Raspberry puede resumirse en cuatro pasos. Primero, consulta los canales de ThingSpeak. Después, valida y almacena los datos recibidos en SQLite. A continuación, expone esa información mediante FastAPI. Finalmente, el dashboard web consulta la API y actualiza la visualización para el usuario.

Con esta organización, la Raspberry Pi queda como plataforma software de supervisión de TierraViva. Su valor no está en decidir cuándo se riega, sino en integrar datos, histórico, API y visualización. Esta decisión reduce la dependencia del sistema respecto a un único equipo central y deja la actuación física bajo control de la estación de campo, que es donde se encuentran los sensores, el relé y la bomba.

4.6. Decisión local de riego y eventos

La decisión de riego en TierraViva se ejecuta localmente en cada estación de campo. Esta elección elimina la necesidad de enviar comandos remotos desde la Raspberry Pi y desplaza la responsabilidad crítica al nodo que está físicamente conectado al sistema de riego. La estación dispone de las lecturas de sensores, controla el relé y puede actuar de forma inmediata sin depender de una comunicación continua con el exterior.

El criterio de decisión inicial se basa en una regla verificable de humedad del suelo. Si la lectura indica una condición de sequedad, no existe un periodo de espera activo y no se ha detectado ninguna incidencia, la estación puede activar el relé durante

un tiempo limitado. Esta lógica no se plantea como inteligencia artificial, sino como automatización basada en reglas, adecuada para una primera versión del prototipo.

De forma conceptual, la decisión local puede resumirse así:

- si la lectura de humedad del suelo cumple la condición de sequedad definida para el prototipo;
- si no existe un periodo de *cooldown* activo;
- si la lectura de sensores es válida;
- si no se ha detectado un error crítico que afecte a los sensores, la alimentación o el estado seguro del relé;
- entonces la estación activa el relé durante un tiempo limitado y registra o publica el evento cuando la comunicación está disponible.

El uso de un *cooldown* evita activaciones repetidas en poco tiempo. Esta medida es necesaria porque las lecturas de humedad pueden oscilar alrededor del umbral o variar lentamente tras una actuación. Sin un periodo mínimo entre riegos, la estación podría activar la bomba varias veces de forma consecutiva. Por tanto, el firmware debe registrar el instante de la última actuación y bloquear nuevos riegos hasta que se supere el tiempo mínimo configurado.

La duración del riego se controla también desde el firmware. La estación no debe mantener el relé activo de forma indefinida ni depender de una orden externa de parada. El comportamiento seguro consiste en activar el relé, esperar como máximo el tiempo permitido y desactivarlo siempre al finalizar la actuación. Si durante ese proceso se detecta una condición anómala, el relé debe apagarse de forma inmediata.

Tras cada ciclo, la estación publica en ThingSpeak no solo las medidas ambientales, sino también información de estado. Esto permite que la Raspberry Pi supervise lo ocurrido sin participar en la decisión. En particular, el campo de estado o evento permite distinguir entre funcionamiento normal, riego ejecutado, *cooldown* activo, lectura inválida, error de sensor o activación bloqueada por seguridad.

Código	Significado	Interpretación
0	Funcionamiento normal	La estación mide y publica sin incidencias.
1	Sensor no disponible	Algún sensor no responde o no se ha podido obtener la lectura esperada.
2	Incidencia de comunicación	La estación ha detectado un fallo de publicación o inicialización de red y lo registra para supervisión.
3	Riego ejecutado	La estación ha activado el relé durante el tiempo permitido.
4	Riego no necesario	La humedad no cumple la condición de sequedad.
5	Lectura no válida para decisión	La lectura existe, pero se descarta por ser incoherente, fuera de rango o no fiable para activar el riego.
6	Cooldown activo	La estación evita un nuevo riego por periodo mínimo entre actuaciones.
7	Apagado de seguridad	El firmware fuerza el relé a apagado por condición no verificable.

Tabla 4.6: Códigos de estado o evento publicados por las estaciones

La Tabla 4.6 recoge una codificación inicial para interpretar el estado publicado por las estaciones. Estos códigos no tienen por qué representar errores graves; en algunos casos, como el código de cooldown o el de riego no necesario, simplemente permiten explicar por qué la estación no ha activado la bomba en un ciclo concreto.

Esta forma de funcionamiento ofrece trazabilidad sin necesidad de comandos remotos. La Raspberry Pi puede reconstruir qué ha ocurrido observando las lecturas, el estado del relé y el código de evento publicado por cada estación. Así se mantiene una separación clara entre decisión local y supervisión: la estación decide y actúa, mientras que la Raspberry registra y muestra.

4.7. Modo seguro

El modo seguro de TierraViva define el comportamiento que debe adoptar la estación cuando no puede garantizar que una actuación sobre el riego sea correcta. Este punto es especialmente importante porque el prototipo no se limita a monitorizar variables ambientales, sino que incorpora actuación física mediante relé y bomba de agua. Por tanto, un fallo en sensores, alimentación, comunicación o lógica interna no debe traducirse en una activación indefinida del riego.

El criterio general adoptado es conservador: ante una situación no verificable, la

bomba debe permanecer apagada. Esta decisión puede provocar que el sistema no riegue en un caso dudoso, pero evita el escenario de mayor riesgo para el prototipo, que sería mantener el riego activo sin control. En este proyecto se considera preferible perder una actuación puntual antes que activar la bomba de forma repetida o indefinida.

El modo seguro se aplica tanto en el arranque de la estación como durante el funcionamiento normal. Al iniciar el firmware, el pin de control del relé debe configurarse en estado desactivado antes de procesar sensores o comunicaciones. De esta forma, un reinicio de la placa no provoca una activación accidental de la bomba. El estado inicial del sistema es, por tanto, relé desactivado y bomba apagada.

Situación detectada	Riesgo asociado	Respuesta segura
Arranque o reinicio de la estación	Activación accidental del relé durante la inicialización.	Relé desactivado y bomba apagada.
Lectura de sensor inválida	Decisión de riego basada en un dato no fiable.	No iniciar riego y publicar estado de error si la comunicación está disponible.
Lectura fuera de rango	Interpretación incorrecta del estado del suelo.	Ignorar lectura y mantener bomba apagada.
Cooldown activo	Activaciones repetidas por oscilación de la medida.	No iniciar nuevo riego.
Tiempo máximo alcanzado	Riego indefinido por fallo de control o bloqueo del programa.	Desactivar relé al superar el límite.
Pérdida de comunicación	Falta de supervisión remota o publicación fallida.	Mantener comportamiento local seguro y registrar incidencia cuando sea posible.
Error interno	Estado no fiable de sensores, comunicación o lógica local.	Forzar relé a apagado y publicar código de seguridad si es posible.

Tabla 4.7: Situaciones contempladas por el modo seguro de la estación

La Tabla 4.7 resume las situaciones principales contempladas. Todas las respuestas siguen el mismo criterio: si el sistema no puede verificar que la actuación sea correcta, no debe activar la bomba.

La pérdida de comunicación se interpreta de forma distinta a como ocurriría en una arquitectura centralizada. En TierraViva, la estación no necesita recibir una orden externa para decidir el riego, por lo que una caída temporal de la Raspberry o de la supervisión remota no bloquea necesariamente la lógica local. Sin embargo, si el

módulo LTE no puede publicar datos durante un periodo prolongado, el firmware debe registrar la incidencia cuando la comunicación vuelva a estar disponible y mantener siempre el relé bajo control local. La pérdida de supervisión remota no debe dejar la bomba activa ni provocar una actuación no controlada; en caso de duda sobre el estado interno de la estación, se mantiene la bomba apagada.

Las lecturas inválidas constituyen otro caso crítico. Si el sensor de humedad devuelve un valor ausente, imposible o fuera del rango esperado, la estación no debe utilizar ese dato para activar la bomba. En su lugar, debe publicar un código de error o conservar el estado seguro hasta que las lecturas vuelvan a ser coherentes. Esta medida evita que una desconexión del sensor o una lectura corrupta se interprete como necesidad real de riego.

El tiempo máximo de actuación es una barrera de seguridad esencial. Aunque la regla local determine que el suelo está seco, la bomba solo puede permanecer activa durante el intervalo permitido. Finalizado ese tiempo, el relé debe desactivarse siempre. Esta desactivación no debe depender de la Raspberry Pi ni de ThingSpeak, sino del propio firmware de la estación.

El cooldown complementa al tiempo máximo. Tras una actuación, la estación debe esperar un periodo mínimo antes de volver a regar. Esto reduce el riesgo de activaciones repetidas provocadas por ruido, histéresis insuficiente o lenta estabilización de la humedad del suelo después del riego.

El modo seguro actúa, por tanto, como una capa local de protección. La Raspberry Pi puede supervisar, registrar y mostrar el estado del sistema, pero la estación conserva la responsabilidad última sobre la decisión y la ejecución segura de la actuación. Esta decisión es coherente con la arquitectura final de TierraViva: autonomía en campo, comunicación para supervisión y seguridad prioritaria frente a cualquier estado dudoso.

Capítulo 5

Implementación hardware y firmware de estaciones

Una vez definida la arquitectura final del sistema, este capítulo describe la implementación hardware y firmware de las estaciones de campo de TierraViva. El propósito no es repetir las decisiones de diseño ya justificadas en capítulos anteriores, sino documentar cómo se preparó, conectó y validó cada bloque de la estación: controlador, sensores, comunicación LTE, publicación de telemetría, actuación sobre el riego y alimentación.

En la arquitectura final, la estación no es un simple nodo que obedece instrucciones externas. Cada estación debe medir, evaluar reglas locales, accionar el riego cuando corresponda y publicar su estado. Por ello, la implementación hardware y el firmware tienen un papel central dentro del prototipo.

La implementación se ha desarrollado de forma incremental. En primer lugar, se preparó el entorno de trabajo con Arduino UNO y se comprobó la programación básica de la placa. A continuación, se validaron los sensores de forma individual y conjunta, se integró el módulo de comunicaciones LTE A7670E-LASA, se implementó el envío de telemetría hacia ThingSpeak y se preparó la lógica de actuación mediante relé y bomba de agua. Esta secuencia permitió reducir riesgos, ya que cada bloque fue probado antes de incorporarse al firmware completo de la estación.

5.1. Preparación de Arduino UNO

La primera fase de implementación consistió en preparar la placa Arduino UNO que actúa como controlador local de la estación de campo. Antes de conectar sensores, módulo LTE o relé, era necesario comprobar que el entorno de programación reconocía correctamente la placa y que era posible cargar un programa básico sin errores. Esta comprobación inicial evita arrastrar problemas de comunicación USB o configuración del entorno a fases posteriores del montaje.

El entorno de desarrollo utilizado fue Arduino IDE 2.3.8 sobre Windows 11. La

placa empleada no fue una Arduino UNO oficial, sino una placa compatible basada en el microcontrolador ATmega328P. Este tipo de placas es habitual en prototipos de bajo coste, pero puede requerir una preparación adicional cuando el conversor USB-serie no es reconocido automáticamente por el sistema operativo.

Durante la primera conexión por USB, la placa recibía alimentación y encendía sus indicadores luminosos, pero no aparecía ningún puerto disponible en el menú *Tools* → *Port* del Arduino IDE. Esto indicaba que el ordenador detectaba alimentación eléctrica, pero no disponía todavía de una comunicación serie utilizable para programar la placa.

La comprobación en el Administrador de dispositivos de Windows mostró el dispositivo como **USB2.0-Serial** dentro del apartado de otros dispositivos. Esta situación permitió identificar el problema: el sistema no tenía instalado el controlador correspondiente al conversor USB-serie de la placa. La inspección física del Arduino confirmó que el chip situado junto al conector USB era un **CH340G**, mientras que el microcontrolador principal era un **ATmega328P**.

Elemento comprobado	Resultado observado	Acción realizada
Sistema operativo	Windows 11	Preparación del entorno en PC.
Entorno de desarrollo	Arduino IDE 2.3.8	Instalación y configuración inicial.
Placa conectada	Arduino UNO compatible	Selección de placa Arduino Uno.
Detección inicial	USB2.0-Serial sin puerto COM disponible	Diagnóstico de falta de driver.
Conversor USB-serie	CH340G	Instalación del driver CH340/CH341.
Microcontrolador principal	ATmega328P	Validación de compatibilidad con Arduino UNO.
Puerto tras instalar driver	COM3	Selección del puerto en Arduino IDE.
Prueba básica	<i>Blink</i> cargado correctamente	Verificación de programación de la placa.

Tabla 5.1: Preparación inicial del Arduino UNO compatible

Una vez identificado el chip **CH340G**, se instaló el driver **CH340/CH341** correspondiente. Tras reconectar la placa, Windows pasó a reconocerla como un dispositivo serie asociado al puerto **COM3**. A partir de ese momento, el Arduino IDE permitió seleccionar la placa como *Arduino Uno* y utilizar el puerto serie correcto para la carga de programas.

La primera prueba realizada fue el ejemplo *Blink*, incluido en el propio Arduino IDE.

Esta prueba consiste en cargar un programa sencillo que conmuta periódicamente el LED integrado de la placa. Aunque funcionalmente es una prueba mínima, resulta útil como validación de la cadena completa de programación: detección USB, selección de placa, selección de puerto, compilación, carga del programa y ejecución en el microcontrolador.

El resultado fue correcto: el programa se cargó en la placa a través de COM3 y el LED integrado respondió según lo esperado. Además, el registro de carga mostró la comunicación con el microcontrolador ATmega328P, confirmando que la placa era programable desde el entorno configurado. Esta validación permitió continuar con las pruebas de sensores sin que la conexión USB o el driver fueran una fuente de error adicional.

Desde el punto de vista de documentación del proyecto, esta incidencia es relevante porque refleja una condición real del montaje. El uso de placas compatibles reduce el coste del prototipo, pero puede introducir pequeños problemas de preparación, como la necesidad de instalar drivers adicionales. Registrar este paso mejora la reproducibilidad del trabajo, ya que otra persona que replique el sistema podría encontrarse con el mismo síntoma: placa encendida, pero sin puerto COM disponible.

Para dejar evidencia de esta fase, se recomienda incluir en el anexo de pruebas iniciales una captura del Administrador de dispositivos antes y después de instalar el driver, una captura del Arduino IDE con la placa y el puerto seleccionados, y una fotografía de la placa donde se aprecien el CH340G y el ATmega328P. Estas evidencias no son necesarias en el cuerpo principal del capítulo, pero sí refuerzan la trazabilidad del proceso de implementación.

Con la placa correctamente reconocida y la prueba *Blink* superada, el Arduino UNO quedó preparado para la siguiente fase: la conexión y validación de los sensores de la estación de campo. Esta comprobación permitió utilizar la placa como base del firmware autónomo de TierraViva, encargado posteriormente de leer sensores, comunicarse con el módulo LTE, controlar el relé y aplicar la lógica local de riego.

5.2. Integración de sensores

Una vez preparada la placa Arduino UNO, la siguiente fase consistió en integrar y validar los sensores de la estación de campo. Esta etapa era necesaria antes de incorporar la comunicación LTE o la actuación sobre el riego, ya que el sistema completo depende de que las medidas de entrada sean coherentes. En TierraViva se utilizan tres bloques de sensórica: humedad de suelo, variables ambientales y luminosidad.

La integración se realizó de forma incremental. Primero se comprobó el bus I2C, después se validó el sensor de humedad de suelo mediante lectura analógica y, por último, se ejecutó una prueba conjunta con todos los sensores conectados. Este procedimiento permitió aislar fallos de cableado o configuración antes de pasar a un firmware más completo.

El BME280 y el BH1750 se conectan mediante el bus I2C del Arduino UNO. I2C, abreviatura de *Inter-Integrated Circuit*, es un bus de comunicación serie utilizado habitualmente para conectar sensores y circuitos integrados con un microcontrolador empleando únicamente dos líneas principales: SDA, para el intercambio de datos, y SCL, para la señal de reloj. En este tipo de bus, varios dispositivos pueden compartir las mismas líneas siempre que cada uno tenga una dirección diferente.

En Arduino UNO, las líneas I2C corresponden a los pines A4/SDA y A5/SCL, según la documentación oficial de la placa [Arduino, 2022, Arduino, 2026a]. Por este motivo, el BME280 y el BH1750 pueden conectarse simultáneamente a los mismos pines. En el montaje realizado, el BH1750 fue detectado en la dirección 0x23 y el BME280 en la dirección 0x76, por lo que ambos sensores pudieron trabajar en el mismo bus sin conflicto de direcciones.

El sensor capacitivo de humedad de suelo se conectó de forma independiente al bus I2C, ya que proporciona una salida analógica. Para esta prueba se utilizó la entrada A0 del Arduino UNO. Esta lectura no representa directamente un porcentaje de humedad, sino un valor bruto entre 0 y 1023, correspondiente al convertidor analógico-digital del microcontrolador. Por ello, antes de utilizarlo como porcentaje de humedad es necesario realizar una calibración o, al menos, identificar el sentido de variación de la medida.

La Tabla 5.2 resume las conexiones utilizadas durante la integración de sensores.

Sensor	Conexión Arduino UNO	Función
BME280	VCC a 5 V, GND a GND, SDA a A4, SCL a A5	Medición de temperatura, humedad relativa y presión atmosférica.
BH1750 / GY-302	VCC a 5 V, GND a GND, SDA a A4, SCL a A5	Medición de luminosidad ambiente en lux.
Sensor capacitivo de humedad de suelo	VCC a 5 V, GND a GND, A0 a A0	Lectura analógica del estado del sustrato.

Tabla 5.2: Conexiones de los sensores integrados en la estación de campo

Durante las pruebas se tuvo en cuenta que todos los módulos deben compartir una referencia común de masa. Este punto es importante porque una referencia de

GND incorrecta puede producir lecturas inestables o impedir la comunicación con los sensores digitales. En el caso del bus I2C, también se comprobó que no era necesario utilizar pines alternativos para SDA y SCL, ya que en Arduino UNO el bus hardware está asignado a A4 y A5.

La primera comprobación realizada sobre los sensores digitales fue un escaneo del bus I2C. Esta prueba no mide todavía variables físicas, pero permite verificar que los dispositivos están correctamente alimentados, cableados y accesibles desde Arduino. El resultado obtenido fue la detección de dos dispositivos: 0x23, correspondiente al BH1750, y 0x76, correspondiente al BME280.

Prueba	Resultado	Interpretación
Escaneo I2C con BME280	Dispositivo detectado en 0x76	El sensor ambiental responde correctamente en el bus I2C.
Escaneo I2C con BH1750 y BME280	Dispositivos detectados en 0x23 y 0x76	Ambos sensores pueden compartir el bus I2C sin conflicto de direcciones.
Lectura analógica del sensor de suelo	Valores variables en A0	El sensor capacitivo responde ante cambios de capacitancia/humedad.
Prueba conjunta	Lectura de suelo, luz, temperatura, humedad y presión	Los tres sensores pueden integrarse en un único firmware de adquisición.

Tabla 5.3: Pruebas iniciales de validación de sensores

El sensor de humedad de suelo se probó inicialmente mediante un sketch específico que imprimía por el monitor serie el valor bruto leído en A0. En reposo, el sensor mostró valores estables alrededor de 569--570. Al tocar la zona sensible del sensor, los valores descendieron aproximadamente al rango 248--356. Esta prueba confirmó que el sensor respondía ante cambios físicos y permitió identificar el sentido de variación: en el montaje probado, un aumento de capacitancia asociado a mayor humedad o contacto produce una disminución del valor analógico leído.

Esta prueba no debe interpretarse como una calibración definitiva de humedad de suelo. Para convertir el valor bruto en un porcentaje fiable sería necesario medir el sensor en condiciones controladas, por ejemplo en aire, en suelo seco y en suelo húmedo, manteniendo la electrónica protegida del agua. Sin embargo, para esta fase de integración, el resultado fue suficiente para comprobar que el sensor estaba vivo, conectado al pin correcto y entregaba una señal analógica utilizable.

El BH1750 se integró como sensor digital de luminosidad. Su lectura se realiza en lux, por lo que resulta más directa que la del sensor de suelo. Durante las pruebas

conjuntas, este sensor se inicializó en modo de alta resolución continua mediante la librería correspondiente. Además, al estar conectado al mismo bus que el BME280, su detección en 0x23 permitió confirmar que no existía conflicto con la dirección 0x76 del sensor ambiental.

El BME280 se utilizó para obtener temperatura, humedad relativa y presión atmosférica. En el firmware de prueba se inicializó mediante la dirección 0x76. Esta comprobación era relevante porque algunos módulos comerciales se anuncian como BME/BMP280 y pueden generar dudas durante el montaje. En el caso de TierraViva, la integración se preparó utilizando la librería del BME280, ya que el sistema necesita también la humedad relativa ambiental, no solo temperatura y presión.

Tras validar los sensores por separado, se cargó un sketch integrado para leer todos los valores en una misma ejecución. Este programa inicializaba el bus I2C, configuraba el BH1750, configuraba el BME280 y leía el sensor de suelo en A0. La salida por monitor serie se estructuró en una línea con los campos principales: `soil_raw`, `lux`, `temp_c`, `hum_pct` y `pres_hpa`. Esta forma de imprimir los datos facilita la depuración y deja preparada la estructura para fases posteriores, donde las lecturas deben enviarse como telemetría.

Variable	Origen	Uso dentro de la estación
<code>soil_raw</code>	Sensor capacitivo en A0	Variable principal para interpretar el estado del sustrato.
<code>lux</code>	BH1750	Información sobre la exposición luminosa del entorno.
<code>temp_c</code>	BME280	Temperatura ambiente en grados Celsius.
<code>hum_pct</code>	BME280	Humedad relativa ambiental en porcentaje.
<code>pres_hpa</code>	BME280	Presión atmosférica en hectopascales.

Tabla 5.4: Variables obtenidas en la prueba conjunta de sensores

La integración conjunta permitió comprobar que el sensor analógico no interfería con el bus I2C y que los sensores digitales podían trabajar simultáneamente. Esta prueba es importante porque el firmware final de la estación no lee los sensores de forma aislada, sino que debe construir una muestra completa de telemetría en cada ciclo de adquisición.

Desde el punto de vista de interpretación de datos, la humedad de suelo requiere especial atención. A diferencia de la temperatura o la luminosidad, que se obtienen en unidades físicas directamente interpretables, el sensor capacitivo entrega un valor

dependiente del propio sensor, del suelo, de la alimentación y de las condiciones de montaje. Por ello, en las primeras pruebas se mantuvo como `soil_raw`. Más adelante, este valor puede transformarse en porcentaje mediante una calibración con valores de referencia seco y húmedo.

También se decidió mantener las variables ambientales como información de contexto. La temperatura, la humedad relativa, la presión y la luminosidad no determinan por sí solas la activación del riego en esta fase, pero permiten interpretar mejor el estado de la estación. Por ejemplo, una humedad de suelo baja puede tener consecuencias distintas en un momento de baja luminosidad que en una situación de alta exposición solar y temperatura elevada.

Con esta prueba quedó validado el bloque de sensórica de la estación. Se confirmó la comunicación I2C con el BME280 y el BH1750, la lectura analógica del sensor de humedad de suelo y la posibilidad de obtener todas las variables en un único ciclo de adquisición. Con ello, la estación quedó preparada para el siguiente paso de implementación: estructurar el firmware de adquisición y adaptar las lecturas al formato de telemetría utilizado por TierraViva.

5.3. Firmware de adquisición

Una vez comprobado el funcionamiento de los sensores, el siguiente paso consistió en organizar el firmware de adquisición de la estación. Este firmware no se limita a leer variables y enviarlas a la nube, sino que también incorpora la lógica local mínima necesaria para decidir si debe activarse el riego.

El firmware se organiza en varios bloques funcionales. En primer lugar, se inicializan los sensores y el módem LTE. Después, en cada ciclo de ejecución, se leen las variables ambientales y de suelo. A partir de esas lecturas, se actualiza el estado interno de seguridad de la estación y se decide si procede activar el riego. Por último, la estación publica una actualización en ThingSpeak con las medidas y el estado operativo.

Bloque del firmware	Función principal
Inicialización	Configurar puerto serie, bus I2C, sensores, pin del relé y módulo LTE.
Adquisición	Leer humedad de suelo, temperatura, humedad relativa, luminosidad y presión.
Interpretación	Convertir la lectura del sensor de suelo a una escala porcentual y comprobar la validez de las medidas.
Decisión local	Evaluar umbrales, ciclos de confirmación, cooldown y estado seguro.
Actuación	Activar el relé durante un tiempo limitado cuando procede el riego.
Telemetría	Enviar a ThingSpeak las variables medidas y el estado operativo de la estación.

Tabla 5.5: Bloques principales del firmware de adquisición y decisión local

La inicialización se realiza en la función `setup()`. En esta fase se configura el puerto serie de depuración, se inicializa el bus I2C, se comprueba la disponibilidad de los sensores y se prepara el pin de control del relé. Un punto importante es que el relé debe quedar en estado desactivado desde el arranque. De esta forma, **un reinicio del Arduino no puede provocar una activación accidental de la bomba.**

El bloque de adquisición lee las mismas variables validadas en el apartado anterior. La humedad de suelo se obtiene a partir del sensor capacitivo conectado a la entrada analógica. El BME280 proporciona temperatura, humedad relativa y presión atmosférica. El BH1750 proporciona la luminosidad ambiente en lux. Estas medidas se almacenan en variables internas del firmware para poder ser utilizadas tanto en la decisión local como en la publicación de telemetría.

En el caso de la humedad de suelo, el firmware no trabaja únicamente con el valor bruto del conversor analógico-digital. A partir de los valores de referencia obtenidos durante las pruebas, la lectura se transforma a una escala porcentual mediante una función de conversión. Esta transformación facilita la interpretación posterior: un valor bajo de porcentaje indica una situación de suelo seco, mientras que un valor más alto representa mayor humedad del sustrato.

Un fragmento representativo de esta parte del firmware se muestra en el Código 5.1. No se incluye aquí el sketch completo, que se recoge en el anexo correspondiente, sino únicamente la lógica esencial de adquisición.

```
void readSensors() {
    lastSoilRaw = readSoilRaw();
    soilPercent = soilRawToPercent(lastSoilRaw);

    temperatureC = bme.readTemperature();
    humidityPct = bme.readHumidity();
    pressureHpa = (int)(bme.readPressure() / 100.0F + 0.5F);

    float luxF = lightMeter.readLightLevel();
    if (luxF < 0) luxF = 0;
    luxValue = (int)(luxF + 0.5F);
}
```

Código 5.1: Lectura agrupada de sensores en la estación

La decisión local se apoya en tres ideas: umbral de humedad, confirmación por varios ciclos y tiempo mínimo entre riegos. No se activa el riego por una única lectura aislada, ya que una medida puntual puede verse afectada por ruido, contacto irregular del sensor o pequeñas variaciones del montaje. Por ello, el firmware acumula ciclos consecutivos en los que el suelo aparece seco antes de considerar que realmente procede regar.

Además, se incorpora un periodo de *cooldown*. Después de un riego, la estación bloquea nuevas activaciones durante un tiempo mínimo. Esta medida evita ciclos repetidos de riego en poco tiempo y reduce el riesgo de sobreactuación. La función mostrada en el Código 5.2 permite comprobar si todavía se encuentra activo ese periodo de espera.

```
bool cooldownActive() {
    if (lastIrrigationEndMs == 0) return false;
    return (millis() - lastIrrigationEndMs) < COOLDOWN_MS;
}
```

Código 5.2: Comprobación del periodo de espera entre riegos

El firmware también mantiene un modo seguro local. Si las lecturas de sensores no son válidas, si el sistema no ha acumulado suficientes ciclos correctos o si se detecta una condición no fiable, la estación evita activar el riego. Este modo seguro no depende de una orden externa de la Raspberry Pi; forma parte del comportamiento interno de la estación. En el código, esta lógica se refleja mediante variables como **safeMode**, **sensorOk**, **validCycleCount** y **dryCycleCount**.

La lógica de decisión local se resume en el Código 5.3. La estación comprueba

primero si las lecturas son válidas y si existe un periodo de espera activo. Después aplica el umbral correspondiente según se encuentre o no en modo seguro.

```
bool shouldIrrigate() {
    if (!sensorOk) {
        statusCode = 400;
        return false;
    }

    if (cooldownActive()) {
        statusCode = 429;
        return false;
    }

    if (safeMode) {
        return (soilPercent <= SAFE_SOIL_THRESHOLD) &&
            (dryCycleCount >= SAFE_CONFIRM_DRY_CYCLES);
    }

    return (soilPercent <= NORMAL_SOIL_THRESHOLD) &&
        (dryCycleCount >= NORMAL_CONFIRM_DRY_CYCLES);
}
```

Código 5.3: Decisión local de riego en la estación

Cuando la estación decide regar, la actuación se realiza de forma local mediante el relé. El Código 5.4 muestra de forma simplificada esta activación temporizada.

```
void irrigateForSeconds(unsigned long seconds) {
    relayActive = true;
    digitalWrite(RELAY_PIN, RELAY_ON);
    statusCode = 200;

    delay(seconds * 1000UL);

    digitalWrite(RELAY_PIN, RELAY_OFF);
    relayActive = false;
    lastIrrigationEndMs = millis();
}
```

Código 5.4: Activación temporizada del relé de riego

La publicación en ThingSpeak se realiza después de leer sensores y actualizar el estado interno de la estación. En la arquitectura final, ThingSpeak se utiliza como plataforma de telemetría y supervisión, no como canal de comandos. Cada estación publica en su canal los valores medidos y tres campos de estado que permiten interpretar el comportamiento del nodo.

Campo	Dato enviado	Descripción
field1	Humedad suelo (%)	Porcentaje normalizado para el prototipo, calculado a partir de valores de referencia del sensor capacitivo.
field2	Temperatura	Temperatura ambiente medida por el BME280.
field3	Humedad ambiente	Humedad relativa medida por el BME280.
field4	Luminosidad	Nivel de luz ambiente medido por el BH1750.
field5	Presión	Presión atmosférica medida por el BME280.
field6	Relé activo	Indica si el relé se encuentra activado durante el envío.
field7	Modo seguro	Indica si la estación se encuentra en modo seguro.
field8	Código de estado	Código que resume el estado operativo o un posible error.

Tabla 5.6: Campos de telemetría enviados por el firmware de estación

Esta estructura permite que el canal de ThingSpeak no almacene solo medidas ambientales, sino también información operativa. El campo del relé permite comprobar si el actuador estaba activo en el momento de la publicación. El campo de modo seguro informa de si la estación está aplicando una lógica más conservadora. El código de estado permite distinguir entre funcionamiento normal, error de sensor, cooldown u otra condición relevante.

El envío se construye mediante una URL de actualización de ThingSpeak. En el firmware se prepara la petición HTTP incluyendo la clave de escritura y los ocho campos definidos. De forma simplificada, la estructura de la petición HTTP enviada a ThingSpeak se muestra en el Código 5.5.

```
http://api.thingspeak.com/update?api_key=...
&field1=soilPercent
&field2=temperatureC
&field3=humidityPct
&field4=luxValue
&field5=pressureHpa
&field6=relayActive
&field7=safeMode
&field8=statusCode
```

Código 5.5: Estructura de la petición HTTP de telemetría enviada a ThingSpeak

La estación conserva así un ciclo completo de funcionamiento autónomo: mide,

interpreta, decide, actúa y publica. La Raspberry Pi puede consultar posteriormente esos datos y representarlos en el dashboard, pero no interviene en el cierre del bucle de control. Esta separación mejora la robustez del prototipo, ya que una caída de la Raspberry no impide que la estación siga aplicando sus reglas locales.

El resultado de esta fase fue un firmware de estación más completo que el empleado únicamente para pruebas de sensores. La estación quedó preparada no solo para generar telemetría, sino para comportarse como un nodo sensor-actuador autónomo, con lógica de riego local, control del relé, cooldown y modo seguro. A partir de este punto, la siguiente fase de implementación se centra en la integración del módulo LTE y en la publicación estable de estos datos en ThingSpeak.

5.4. Integración del A7670E

La integración del módulo A7670E-LASA fue una de las fases más sensibles de la estación de campo, ya que de este bloque depende que el nodo pueda publicar datos en ThingSpeak sin utilizar WiFi ni una conexión física con la Raspberry Pi. A diferencia de los sensores, que se comunican directamente con Arduino mediante entradas analógicas o bus I2C, el módulo LTE requiere alimentación estable, comunicación serie, configuración de red móvil y ejecución de comandos AT.

Antes de conectar el A7670E al Arduino, el módulo se validó de forma independiente mediante conexión USB y comandos AT. Esta prueba previa permitió comprobar que el módem respondía correctamente, que la tarjeta SIM estaba operativa, que existía cobertura suficiente, que el registro en red LTE era correcto y que era posible realizar peticiones HTTP. Esta validación redujo el riesgo de mezclar problemas de red móvil con problemas propios de la integración con Arduino.

La Tabla 5.7 resume las comprobaciones realizadas antes de la integración definitiva con la estación.

Elemento comprobado	Resultado	Interpretación
Puerto AT por USB	Correcto	El ordenador podía comunicarse con el módem mediante comandos AT.
Respuesta básica del módulo	Correcta	El comando AT devolvía respuesta válida.
Tarjeta SIM	Correcta	La SIM estaba preparada para operar en red.
Cobertura	Buena	La calidad de señal era suficiente para continuar las pruebas.
Registro LTE	Correcto	El módulo conseguía registrarse en la red móvil.
Datos móviles	Correctos	El módem obtenía conectividad IP.
Ping	Correcto	Se comprobó conectividad hacia Internet.
HTTP GET	Correcto	Se validó la capacidad de realizar peticiones HTTP y leer la respuesta.

Tabla 5.7: Validación previa del módulo A7670E-LASA

Una vez validado el módulo por separado, se abordó la conexión con Arduino UNO. El A7670E se comunica con el microcontrolador mediante una interfaz UART. UART, abreviatura de *Universal Asynchronous Receiver-Transmitter*, es una interfaz serie asíncrona muy utilizada para comunicar microcontroladores con módulos externos. A diferencia de I2C, no emplea una línea de reloj compartida, sino dos líneas principales de datos: una de transmisión y otra de recepción. Por este motivo, las conexiones deben cruzarse: el pin TXD del módem se conecta a la entrada de recepción del Arduino, y la salida de transmisión del Arduino se conecta al pin RXD del módem. Además, ambos dispositivos deben compartir una referencia común de masa para que los niveles eléctricos de la comunicación sean interpretados correctamente.

Como el Arduino UNO solo dispone de un puerto serie hardware, utilizado además para la programación y depuración por USB, se empleó `SoftwareSerial` para crear un puerto serie adicional destinado al módem.

En el montaje de la estación se mantuvieron los pines D10 y D11, ya cableados y probados durante las fases anteriores. El pin D10 del Arduino se utiliza como recepción del puerto serie software y se conecta al pin TXD del A7670E. El pin D11 se utiliza como transmisión hacia el módem y se conecta al pin RXD del A7670E mediante un divisor resistivo.

La necesidad del divisor resistivo se debe a la diferencia de niveles lógicos entre

Arduino UNO y el módulo LTE. Arduino UNO trabaja con señales de 5 V, mientras que la entrada serie del módulo no debe recibir directamente ese nivel si no existe adaptación integrada. Para reducir la tensión de la señal transmitida desde Arduino hacia el A7670E, se incorporó un divisor formado por resistencias de 1 k Ω y 2 k Ω . De este modo, la señal enviada desde D11 se reduce antes de llegar al pin RXD del módem.

La Tabla 5.8 recoge las conexiones principales utilizadas.

Pin A7670E-LASA	Conexión Arduino / alimentación	Función
TXD	D10 del Arduino	Línea de transmisión del módem hacia Arduino.
RXD	D11 del Arduino mediante divisor resistivo	Línea de recepción del módem desde Arduino, con adaptación de nivel.
GND	GND común	Referencia común entre Arduino, módem, sensores y alimentación.
VCC	Salida regulada de 5 V	Alimentación del módulo según las especificaciones de la placa utilizada.
PWRKEY	Sin control desde Arduino en el montaje base	Pin de encendido no utilizado en el sketch base salvo necesidad específica del módulo.

Tabla 5.8: Conexiones principales entre Arduino UNO y el A7670E-LASA

La alimentación del módulo se trató como un aspecto crítico. Los módulos de comunicación celular pueden presentar consumos variables, especialmente durante el registro en red y la transmisión de datos, por lo que la alimentación y el cableado deben respetar las recomendaciones de diseño del fabricante y de la placa de evaluación utilizada [SIMCom Wireless Solutions, 2021, Waveshare, 2026]. Por ello, el A7670E no debe alimentarse de forma improvisada desde un pin débil del microcontrolador, sino desde una línea de alimentación capaz de proporcionar corriente suficiente. En la estación, la alimentación parte de una fuente principal de 12 V y se regula mediante el LM2596 ajustado a 5 V para los elementos compatibles. Además, se incorporan condensadores de estabilización para reducir caídas puntuales de tensión y mejorar el comportamiento del conjunto.

En el firmware, la comunicación con el módem se realiza mediante `SoftwareSerial`. Se mantuvo la configuración de pines que había sido validada en el montaje físico y se evitó cambiar a otros pines o a velocidades más altas sin necesidad. Esta decisión fue importante porque el uso de `SoftwareSerial` en Arduino UNO puede volverse inestable a baudios elevados. En las pruebas, el funcionamiento estable se consiguió

forzando el módem a 9600 baudios.

El Código 5.6 muestra el fragmento básico de configuración del puerto serie software utilizado para el A7670E.

```
#include <SoftwareSerial.h>

SoftwareSerial modem(10, 11);
// D10: RX Arduino <- TXD del A7670E
// D11: TX Arduino -> RXD del A7670E mediante divisor resistivo
```

Código 5.6: Configuración serie del A7670E en Arduino UNO

Durante la integración se incorporó una lógica de detección de baudios del módem. Esta función prueba varias velocidades habituales y comprueba en cuál responde correctamente el comando AT. Si el módem responde a una velocidad distinta de 9600, el firmware intenta reconfigurarlo mediante el comando AT+IPR=9600. Esta medida evita depender del estado previo del módem y mejora la reproducibilidad del montaje.

El Código 5.7 muestra de forma simplificada esta lógica.

```
bool probeBaud(long baud) {
    modem.end();
    delay(250);
    modem.begin(baud);
    delay(800);

    clearModemBuffer();
    modem.println("AT");

    String resp = readATResponse(2000);
    return resp.indexOf("OK") >= 0;
}

long detectModemBaud() {
    const long bauds[] = {9600, 115200, 57600, 38400, 19200};

    for (unsigned int i = 0; i < sizeof(bauds) / sizeof(bauds[0]); i++) {
        if (probeBaud(bauds[i])) {
            return bauds[i];
        }
    }

    return -1;
}
```

Código 5.7: Detección de velocidad serie del módulo LTE

La secuencia de comandos AT utilizada por la estación se divide en varias fases. Primero se comprueba que el módulo responde. Después se verifica el registro en red LTE. A continuación, se configura el contexto de datos con el APN correspondiente y se activa la conexión de datos. Finalmente, se realizan peticiones HTTP para publicar datos en ThingSpeak.

Los comandos AT empleados en esta fase no se definieron de forma arbitraria, sino a partir del manual de comandos de la familia SIMCom A76XX [SIMCom Wireless Solutions, 2024]. En el firmware solo se incorporaron aquellos comandos necesarios para el funcionamiento del prototipo: comprobación básica del módem, validación de SIM, consulta de cobertura, registro en red, activación del contexto de datos y ejecución de peticiones HTTP.

Fase	Comando AT	Objetivo
Comprobación básica	AT	Verificar que el módem responde.
SIM	AT+CPIN?	Comprobar que la tarjeta SIM está preparada.
Cobertura	AT+CSQ	Obtener una medida de calidad de señal.
Registro LTE	AT+CEREG?	Verificar registro en red móvil.
Operador	AT+COPS?	Consultar operador de red.
Contexto IP	AT+CGDCONT=1,"IP",internet	Configurar el APN de datos.
Adjunto a red	AT+CGATT=1	Activar adjunto a la red de paquetes.
Activación PDP	AT+CGACT=1,1	Activar el contexto de datos.
Dirección IP	AT+CGPADDR=1	Comprobar dirección IP asignada.

Tabla 5.9: Comandos AT utilizados en la inicialización de red del A7670E

El registro en red se comprobó mediante **AT+CEREG?**. En el firmware se considera válido que el módulo devuelva un registro local o en itinerancia, representado por respuestas como **+CEREG: 0,1** o **+CEREG: 0,5**. Si el módulo no se encuentra registrado, la estación no debe intentar publicar datos en ThingSpeak, ya que todavía no dispone de conectividad móvil válida.

El Código 5.8 muestra la comprobación empleada para detectar si el módulo está registrado en red.

```

bool isNetworkRegistered() {
    clearModemBuffer();
    modem.println("AT+CEREG?");

    String resp = readATResponse(3000);

    return (resp.indexOf("+CEREG: 0,1") >= 0) ||
        (resp.indexOf("+CEREG: 0,5") >= 0);
}

```

Código 5.8: Comprobación de registro LTE del A7670E

Una vez registrado en red, el firmware configura el contexto de datos. Para ello utiliza el APN definido para la tarjeta SIM, en este caso **internet**. Después activa el adjunto a red y el contexto de datos. Si esta fase falla, la variable interna que indica disponibilidad de datos móviles se marca como no preparada y el sistema reintenta la conexión en ciclos posteriores.

La publicación de datos en ThingSpeak se realiza mediante HTTP. La estación construye una URL con la clave de escritura del canal y los campos de telemetría, siguiendo el formato de actualización de canal definido por ThingSpeak [MathWorks, 2026c]. A continuación, el A7670E ejecuta la secuencia de comandos HTTP proporcionada por el manual AT de la serie A76XX: inicialización del servicio, configuración del contexto, asignación de URL, ejecución de la petición y lectura de la respuesta [SIMCom Wireless Solutions, 2024]. En las pruebas previas del módulo ya se había validado la ejecución de HTTP GET y la lectura de la respuesta, por lo que la integración con ThingSpeak se apoyó en un comportamiento ya comprobado.

La secuencia HTTP utilizada en el firmware se resume en la Tabla 5.10.

Paso	Comando AT	Función
Finalización previa	AT+HTTPTERM	Cerrar posibles sesiones HTTP anteriores.
Inicialización HTTP	AT+HTTPINIT	Iniciar el servicio HTTP del módem.
Contexto de datos	AT+HTTPPARA=¿ID",1	Asociar la petición al contexto de datos activo.
URL	AT+HTTPPARA=¿URL", "..."	Configurar la URL de destino.
Acción HTTP	AT+HTTPACTION=0	Ejecutar una petición HTTP GET.
Lectura de respuesta	AT+HTTPREAD	Leer el cuerpo de la respuesta devuelta.
Cierre HTTP	AT+HTTPTERM	Liberar el servicio HTTP tras la operación.

Tabla 5.10: Secuencia HTTP utilizada con el módulo A7670E

El Código 5.9 muestra una versión simplificada de la función empleada para realizar una petición HTTP GET a través del módem. Se ha omitido parte del tratamiento de errores para centrar el fragmento en la secuencia principal.

```

bool httpGETUrl(const char *url, String &bodyOut) {
    char urlCmd[300];

    sendATAny("AT+HTTPTERM", 1500);
    if (!sendAT("AT+HTTPINIT", 2500)) return false;
    if (!sendAT("AT+HTTTPARA=\"CID\",1", 1500)) return false;

    snprintf(urlCmd, sizeof(urlCmd),
             "AT+HTTTPARA=\"URL\", \"%s\"", url);

    if (!sendAT(urlCmd, 4000)) return false;

    modem.println("AT+HTTPACTION=0");
    String actionResp = readATResponse(12000);

    if (actionResp.indexOf("+HTTPACTION: 0,200") < 0 &&
        actionResp.indexOf(",200,") < 0) {
        sendATAny("AT+HTTPTERM", 1500);
        return false;
    }

    modem.println("AT+HTTPREAD");
    String readResp = readATResponse(8000);

    sendATAny("AT+HTTPTERM", 1500);
    bodyOut = extractHttpBody(readResp);
    return bodyOut.length() > 0;
}

```

Código 5.9: Petición HTTP GET mediante el A7670E

La gestión de errores es una parte importante de la integración. El firmware no asume que la red móvil esté siempre disponible. Si el módem no responde, si el registro LTE no se completa, si falla la activación de datos o si la respuesta HTTP no es válida, la estación marca la conectividad como no preparada y vuelve a intentarlo posteriormente. Esta decisión evita bloquear el programa completo ante un fallo temporal de red.

También se tuvo en cuenta que la comunicación HTTP puede tardar varios segundos. Por ello, las funciones de lectura de respuesta incorporan tiempos máximos de espera. Si se supera el tiempo previsto sin recibir una respuesta válida, la operación se considera fallida. Este comportamiento permite que la estación continúe ejecutando su ciclo principal en lugar de quedar detenida indefinidamente esperando al módem.

Desde el punto de vista de la implementación, uno de los criterios más importantes fue no modificar simultáneamente demasiadas variables. Una vez que se consiguió una

combinación estable de pines, divisor resistivo, velocidad de 9600 baudios, secuencia AT y envío HTTP, esa configuración se mantuvo como base del firmware final. Cambiar pines, librerías, velocidad de serie o secuencia de inicialización al mismo tiempo habría dificultado la depuración y aumentado el riesgo de perder una integración ya validada.

El resultado de esta fase fue la integración estable del A7670E-LASA dentro de la estación de campo. El módulo quedó conectado al Arduino mediante UART software, con adaptación de nivel en la línea hacia RXD, alimentación estabilizada y una secuencia de comandos capaz de registrar el módem en red, activar datos móviles y realizar peticiones HTTP. Con ello, la estación quedó preparada para publicar telemetría real en ThingSpeak utilizando únicamente conectividad LTE.

Durante la depuración del envío HTTP se tuvo en cuenta una limitación propia del Arduino UNO: la memoria SRAM disponible es reducida. La construcción de URLs largas, el uso de buffers locales grandes y la impresión completa de cadenas por el monitor serie pueden provocar reinicios o comportamientos inestables. Por este motivo, el firmware evita imprimir la URL completa durante el funcionamiento normal, utiliza textos constantes con `F()` cuando procede y mantiene buffers de tamaño controlado para la respuesta HTTP.

5.5. Envío de datos a ThingSpeak

Una vez integrados los sensores y validado el módulo LTE, la siguiente fase consistió en publicar la telemetría de la estación en ThingSpeak. Este paso conecta el firmware de adquisición con la plataforma cloud utilizada para almacenar y visualizar los datos. En TierraViva, ThingSpeak actúa como destino de telemetría: cada estación publica periódicamente sus medidas y su estado operativo en el canal correspondiente.

El envío de datos se realiza mediante una petición HTTP de actualización de canal. ThingSpeak permite escribir datos en un canal mediante una URL que incluye la clave de escritura y los campos que se desean actualizar [MathWorks, 2026c]. En el caso de TierraViva, la estación construye esa URL a partir de las variables medidas por los sensores y de varios campos internos de estado.

La asociación entre variables y campos no se deja implícita en el código, sino que se mantiene fija para facilitar la lectura posterior desde la plataforma y desde la Raspberry Pi. La Tabla 5.11 recoge el mapeo utilizado por el firmware de estación.

Campo	Variable enviada	Descripción
field1	soilPercent	Humedad de suelo expresada en porcentaje.
field2	temperatureC	Temperatura ambiente medida por el BME280.
field3	humidityPct	Humedad relativa ambiental medida por el BME280.
field4	luxValue	Luminosidad ambiente medida por el BH1750.
field5	pressureHpa	Presión atmosférica medida por el BME280.
field6	relayActive	Estado del relé de actuación.
field7	safeMode	Estado del modo seguro local.
field8	statusCode	Código de estado o error de la estación.

Tabla 5.11: Mapeo de variables del firmware a campos de ThingSpeak

Los cinco primeros campos corresponden a variables físicas medidas por los sensores. Los tres últimos no son medidas ambientales, sino información operativa. Esta separación permite que una entrada de ThingSpeak represente no solo el estado del entorno, sino también el estado de la estación en el momento del envío. Por ejemplo, el campo `field6` permite saber si el relé estaba activo, `field7` indica si la estación se encontraba en modo seguro y `field8` resume el estado del firmware mediante un código numérico.

La construcción de la petición se realiza en el firmware mediante una cadena de texto. En ella se incorpora la clave de escritura del canal y los ocho campos definidos. El Código 5.10 muestra de forma simplificada esta construcción. La clave real de escritura no debe aparecer en la memoria ni en el repositorio, por lo que se representa mediante una constante.

```

snprintf(url, sizeof(url),
    "http://api.thingspeak.com/update?api_key=%s"
    "&field1=%d"
    "&field2=%.2f"
    "&field3=%.2f"
    "&field4=%d"
    "&field5=%d"
    "&field6=%d"
    "&field7=%d"
    "&field8=%d",
    THINGSPEAK_WRITE_KEY,
    soilPercent,
    temperatureC,
    humidityPct,
    luxValue,
    pressureHpa,
    relayActive ? 1 : 0,
    safeMode ? 1 : 0,
    statusCode);

```

Código 5.10: Construcción de la URL de actualización de ThingSpeak

El formato elegido mantiene la petición simple y compatible con el servicio HTTP del A7670E. La estación no necesita generar un cuerpo JSON ni establecer una sesión persistente: basta con construir una URL de actualización y solicitarla mediante una petición HTTP GET. Esta decisión reduce la complejidad del firmware y encaja con las limitaciones de memoria y procesamiento del Arduino UNO.

El envío propiamente dicho se delega en la función de comunicación HTTP implementada sobre el A7670E. Desde el punto de vista del firmware principal, el flujo consiste en preparar la URL, comprobar que el módem se encuentra listo, realizar la petición y analizar el resultado. El Código 5.11 muestra una versión simplificada de este proceso.

```
bool sendTelemetryToThingSpeak() {
    char url[350];
    String body;

    buildThingSpeakUrl(url, sizeof(url));

    if (!modemReady) {
        statusCode = 502;
        return false;
    }

    bool ok = httpGETUrl(url, body);

    if (!ok) {
        statusCode = 503;
        return false;
    }

    statusCode = 201;
    return true;
}
```

Código 5.11: Envío de telemetría a ThingSpeak desde la estación

La gestión de respuesta es importante porque el hecho de enviar una petición al módem no garantiza por sí solo que ThingSpeak haya registrado una nueva entrada. La función HTTP debe comprobar que la acción devuelve una respuesta correcta y que el cuerpo recibido es coherente. En ThingSpeak, una escritura correcta suele devolver el identificador de la entrada creada en el canal. En cambio, una respuesta vacía, un código HTTP incorrecto o un fallo de lectura deben tratarse como envío fallido.

En el firmware, este resultado se utiliza para actualizar el estado interno de la estación. Si el envío se realiza correctamente, se considera que la telemetría ha quedado publicada. Si falla, la estación no debe bloquearse indefinidamente ni asumir que el dato está disponible en la nube. En su lugar, marca el fallo mediante un código de estado y continúa con el ciclo de funcionamiento.

La Tabla 5.12 resume los principales errores considerados durante el envío.

Situación	Efecto	Tratamiento
Módem no preparado	No se puede iniciar la petición HTTP.	Marcar error y reintentar más adelante.
Sin registro LTE	No existe conectividad móvil válida.	Evitar el envío y mantener operación local.
Error al abrir HTTP	El servicio HTTP del módem no queda inicializado.	Cerrar sesión HTTP y reintentar en el siguiente ciclo.
Respuesta no válida	No se confirma la escritura en ThingSpeak.	Registrar fallo de envío.
Tiempo de espera agotado	La estación no recibe respuesta dentro del intervalo previsto.	Cancelar operación y continuar ciclo.

Tabla 5.12: Errores considerados durante el envío de telemetría a ThingSpeak

Los reintentos se plantean de forma controlada. No resulta conveniente realizar bucles indefinidos de envío, ya que podrían bloquear el firmware, aumentar el consumo y dificultar la actuación local. Por ello, si una publicación falla, la estación puede marcar el error y volver a intentarlo en el siguiente ciclo de telemetría. Este enfoque mantiene la estación operativa incluso cuando la conectividad móvil o la plataforma cloud no están disponibles temporalmente.

También se evita que un fallo de publicación afecte directamente a la seguridad del riego. La estación toma la decisión de actuación de forma local, por lo que una pérdida temporal de conectividad con ThingSpeak no impide aplicar las reglas básicas de riego y seguridad. La consecuencia principal de un fallo de envío es la pérdida de una muestra en la nube, no la pérdida completa del control de la bomba.

El tratamiento de errores se apoya en el campo `statusCode`. Este campo permite que la estación informe posteriormente de situaciones relevantes, como fallo de comunicación, módem no preparado, cooldown activo, error de sensores o publicación correcta. Aunque el sistema no pretende implementar un protocolo complejo de diagnóstico, estos códigos ayudan a interpretar el comportamiento observado en ThingSpeak y en el dashboard.

Desde el punto de vista de privacidad y seguridad práctica, la clave de escritura del canal debe tratarse como información sensible. En el código del repositorio no debería aparecer la clave real utilizada durante las pruebas. Para la memoria y los anexos, las claves deben ocultarse o sustituirse por valores genéricos. Este criterio evita que un tercero pueda escribir datos en los canales del proyecto.

El resultado de esta fase fue la publicación de una muestra completa de estación en ThingSpeak. Cada envío incluye las variables de suelo y ambiente, junto con el estado

del relé, el modo seguro y el código de estado. Con ello, la estación queda conectada a la plataforma cloud y puede ser supervisada de forma remota sin depender de una conexión local con la Raspberry Pi.

5.6. Control de relé, LED y bomba de agua

La actuación física sobre el riego se realiza mediante un módulo relé controlado desde el Arduino UNO. La estación no envía únicamente datos a la nube, sino que también puede activar localmente el sistema de riego cuando se cumplen las condiciones definidas en el firmware. Para ello, el Arduino gobierna una salida digital conectada al módulo relé, y este conmuta la alimentación de la bomba de agua de 12 V.

Durante las primeras pruebas de actuación se puede utilizar un LED o una carga de baja potencia como indicador visual del estado del relé. Esta prueba permite comprobar la lógica de activación sin accionar todavía la bomba real. Una vez validado el comportamiento del firmware, la carga de prueba se sustituye por la bomba de agua, manteniendo el mismo principio de control: el Arduino no alimenta directamente la carga de potencia, sino que controla el relé que abre o cierra el circuito de 12 V.

El pin utilizado para el control del relé es el pin digital D7 del Arduino UNO. Este pin se configura como salida en la inicialización del firmware y se deja en estado desactivado desde el arranque. Así se evita que un reinicio de la placa active accidentalmente la bomba. El estado seguro por defecto de la estación es, por tanto, relé desactivado y bomba apagada.

Elemento	Conexión	Función
Pin de control	D7 del Arduino UNO	Señal digital que activa o desactiva el módulo relé.
Relé VCC	5 V	Alimentación lógica del módulo relé.
Relé GND	GND común	Referencia común con Arduino y alimentación del sistema.
Relé IN	D7	Entrada de control procedente del Arduino.
Contacto COM	Línea positiva de 12 V	Entrada de potencia hacia el contacto del relé.
Contacto NO	Positivo de la bomba	Salida hacia la bomba cuando el relé se activa.
Negativo de la bomba	GND de la fuente de 12 V	Retorno de alimentación de la bomba.

Tabla 5.13: Conexiones principales del bloque de actuación mediante relé

La conexión de potencia se realiza utilizando los contactos COM y NO del relé. El contacto NO, abreviatura de *Normally Open*, permanece abierto mientras el relé no está activado. De esta forma, aunque exista alimentación de 12 V disponible, la bomba permanece apagada hasta que el Arduino active expresamente el relé.

Cuando el firmware activa el relé, el contacto COM queda unido al contacto NO, cerrando el circuito de alimentación de la bomba. La bomba recibe entonces 12 V y comienza el riego. Al finalizar el tiempo de actuación, el firmware desactiva el pin de control, el relé vuelve a abrir el circuito y la bomba queda apagada.

Un aspecto práctico que debe comprobarse durante las pruebas es si el módulo relé es activo alto o activo bajo. Algunos módulos se activan cuando la entrada IN recibe un nivel alto, mientras que otros lo hacen con nivel bajo. Por ello, en el firmware se definen constantes específicas para representar el estado de activación y desactivación del relé, evitando escribir directamente HIGH o LOW en toda la lógica del programa.

```
const int RELAY_PIN = 7;

const byte RELAY_ON = HIGH;
const byte RELAY_OFF = LOW;
```

Código 5.12: Definición del pin y estados del relé

La inicialización del relé se realiza al comienzo del firmware. Primero se configura el pin como salida y después se fuerza el estado apagado. Esta secuencia evita que el pin quede flotante durante el arranque y reduce el riesgo de una activación no deseada.

```
pinMode(RELAY_PIN, OUTPUT);
digitalWrite(RELAY_PIN, RELAY_OFF);
relayActive = false;
```

Código 5.13: Inicialización segura del relé durante el arranque

El control de la bomba no se implementa como una activación indefinida, sino como una actuación temporizada. Cuando el firmware decide regar, activa el relé, registra el instante de inicio y guarda la duración prevista. A partir de ese momento, el propio firmware debe encargarse de apagar la bomba al finalizar el intervalo configurado.

```
void startRelay(int durationSec) {
    unsigned long now = millis();

    digitalWrite(RELAY_PIN, RELAY_ON);
    relayActive = true;
    relayStartMs = now;
    relayDurationMs = (unsigned long)durationSec * 1000UL;

    statusCode = 200;
    forceTelemetry = true;
}
```

Código 5.14: Activación temporizada del relé de riego

La desactivación del relé se centraliza en una función específica. Esto evita que existan varios puntos del programa apagando la bomba de forma distinta. Siempre que se finaliza una actuación, ya sea por duración completada, por timeout de seguridad o por error, se utiliza la misma lógica de apagado.

```
void stopRelay(const char *reason, bool emergencyTimeout) {
    digitalWrite(RELAY_PIN, RELAY_OFF);
    relayActive = false;
    lastIrrigationEndMs = millis();

    if (emergencyTimeout) {
        statusCode = 408;
    }

    forceTelemetry = true;
}
```

Código 5.15: Desactivación segura del relé

La función de servicio del relé comprueba periódicamente si existe una actuación en curso. Si el tiempo transcurrido supera la duración prevista, se apaga la bomba. Además, se incorpora un tiempo máximo de seguridad para evitar que un fallo de lógica deje el relé activado durante más tiempo del permitido.

```

void serviceRelayState() {
    if (!relayActive) return;

    unsigned long elapsed = millis() - relayStartMs;

    if (elapsed >= relayDurationMs) {
        stopRelay("duration_completed", false);
    } else if (elapsed >= FAILSAFE_TIMEOUT_MS) {
        stopRelay("failsafe_timeout", true);
    }
}

```

Código 5.16: Supervisión del estado del relé durante el riego

Esta lógica permite separar la decisión de riego de la seguridad de actuación. La estación puede decidir activar la bomba cuando la humedad del suelo se encuentra por debajo del umbral definido, pero la duración real de esa activación queda limitada por el firmware. La bomba no depende de una orden externa de apagado ni de la Raspberry Pi; el apagado se realiza localmente en la propia estación.

Además del tiempo máximo de actuación, se aplica un periodo mínimo entre riegos o *cooldown*. Este intervalo evita que la bomba se active repetidamente si las lecturas de humedad oscilan alrededor del umbral. Si el firmware detecta que no ha transcurrido suficiente tiempo desde el último riego, no inicia una nueva actuación y actualiza el código de estado para dejar constancia de la condición.

Condición	Comportamiento del firmware	Estado resultante
Arranque de la estación	Configurar D7 como salida y escribir RELAY_OFF.	Bomba apagada.
Suelo seco y condiciones válidas	Activar relé durante el tiempo configurado.	Bomba activa temporalmente.
Duración completada	Desactivar relé mediante stopRelay().	Bomba apagada.
Cooldown activo	No iniciar nuevo riego.	Bomba apagada.
Duración superior al máximo	Limitar o rechazar la actuación según configuración.	Actuación controlada o no ejecutada.
Timeout de seguridad	Forzar apagado del relé.	Bomba apagada y estado de error.
Error interno o lectura no válida	No activar relé o forzar apagado si estaba activo.	Bomba apagada.

Tabla 5.14: Comportamiento del bloque de actuación ante distintas condiciones

El LED de prueba resulta útil durante la validación porque permite observar

visualmente el estado de activación sin conectar todavía una carga hidráulica. En esta fase, el LED puede conectarse como carga de prueba o utilizarse el propio indicador del módulo relé si el módulo lo incorpora. Una vez comprobado que la lógica de activación y apagado funciona correctamente, la prueba debe repetirse con la bomba real durante tiempos cortos y en condiciones controladas.

Eléctricamente, la bomba de agua se comporta como una carga de potencia y puede introducir perturbaciones al conectarse o desconectarse. Por ello, el bloque de actuación no debe alimentarse desde el Arduino, sino desde la línea de 12 V prevista para la carga. El Arduino únicamente entrega la señal de control al módulo relé. Además, todos los bloques deben compartir masa común para que la señal de control sea interpretada correctamente.

Si la bomba utilizada es de corriente continua, es recomendable incorporar protección frente a transitorios, especialmente al tratarse de una carga inductiva. Esta protección puede realizarse mediante un diodo de rueda libre o *flyback* colocado en paralelo con la bomba, con la polaridad adecuada, o mediante un módulo de actuación que ya incorpore protección equivalente.

La actuación ante emergencia o condición no verificable sigue el mismo principio que el modo seguro general del sistema: apagar la bomba. Una emergencia puede corresponder a un timeout de actuación, una lectura inválida, una condición interna incoherente o un fallo detectado durante el ciclo de control. En todos estos casos, el firmware debe desactivar el relé y publicar, cuando la comunicación esté disponible, un código de estado que permita identificar el motivo de la parada.

El resultado de esta fase es un bloque de actuación local, temporizado y verificable. La estación no se limita a encender o apagar una salida digital, sino que controla el relé con criterios de seguridad: arranque apagado, activación limitada, apagado centralizado, cooldown entre riegos, timeout de seguridad y publicación de estado.

5.7. Alimentación de estación

La estación de campo de TierraViva se alimenta a partir de una línea principal de 12 V. Esta tensión permite alimentar el bloque de actuación, formado por el relé y la bomba de agua, y sirve como entrada para obtener la tensión de 5 V necesaria para la electrónica de control. La alimentación se ha considerado un bloque crítico del montaje, especialmente por la presencia del módulo LTE A7670E-LASA y de una carga de potencia como la bomba.

El esquema de alimentación se basa en baterías de moto de 12 V y un conversor

reductor LM2596 ajustado a 5 V. El LM2596S es un convertidor DC-DC de tipo *step-down*, por lo que permite obtener una tensión de salida inferior y regulada a partir de una tensión de entrada mayor [AZ-Delivery, 2024]. La línea de 12 V se utiliza para la bomba de agua, conmutada mediante los contactos del relé. La salida de 5 V del LM2596 alimenta los elementos de electrónica de baja tensión: Arduino UNO, sensores, módulo relé y módulo A7670E-LASA, siempre que la placa concreta del módem admita entrada de 5 V según su documentación.

Antes de conectar los módulos electrónicos, el LM2596 debe ajustarse y verificarse con un multímetro. Esta comprobación evita aplicar una tensión incorrecta al Arduino, sensores o módem. En el prototipo se trabaja con una salida regulada de 5 V, pero la entrada procede de una tensión superior, por lo que un ajuste incorrecto del conversor podría dañar componentes sensibles.

Elemento	Alimentación	Observación
Bomba de agua	12 V	Alimentada desde la línea principal a través del relé.
Arduino UNO	5 V regulados	Alimentación procedente del LM2596.
A7670E-LASA	5 V regulados	Debe alimentarse desde una línea estable y con corriente suficiente.
BME280	5 V regulados	Siempre que el módulo utilizado admita 5 V.
BH1750	5 V regulados	Alimentación del módulo de luminosidad.
Sensor de humedad de suelo	5 V regulados	Alimentación del sensor capacitivo.
Módulo relé	5 V regulados	Alimentación de la lógica del relé.

Tabla 5.15: Distribución de alimentación en la estación de campo

Un requisito importante del montaje es que todos los bloques compartan una masa común. El Arduino, el A7670E, los sensores, el relé, el LM2596 y la fuente de 12 V deben tener una referencia común de GND. Sin esta referencia compartida, las señales de control y comunicación podrían no interpretarse correctamente. Esto afecta tanto a la señal digital que activa el relé como a la comunicación serie con el módem LTE y a las lecturas de sensores.

El módulo A7670E-LASA merece una atención especial. Los módulos de comunicación celular pueden presentar consumos variables durante el registro en red y la transmisión de datos, por lo que la alimentación debe ser más robusta que en un

montaje formado únicamente por sensores. Para reducir inestabilidades, se incorporan condensadores de estabilización cerca de la entrada de alimentación del módem.

En concreto, se utiliza un condensador electrolítico de entre $470\ \mu\text{F}$ y $1000\ \mu\text{F}$ entre VCC y GND del A7670E, acompañado de un condensador cerámico de $100\ \text{nF}$ en paralelo. El condensador electrolítico ayuda a absorber caídas momentáneas de tensión asociadas a picos de consumo, mientras que el condensador cerámico contribuye al filtrado de ruido de alta frecuencia. Ambos deben colocarse físicamente lo más cerca posible del módulo LTE.

Condensador	Valor utilizado	Función
Electrolítico	$470\ \mu\text{F} - 1000\ \mu\text{F}$	Reserva de energía ante picos de corriente del módem LTE.
Cerámico	$100\ \text{nF}$	Filtrado de ruido rápido y alta frecuencia.

Tabla 5.16: Condensadores de estabilización en la alimentación del A7670E

La conexión recomendada sitúa ambos condensadores entre la salida positiva de 5 V del LM2596 y la masa común, en paralelo con la alimentación del A7670E. En el caso del condensador electrolítico debe respetarse la polaridad: el terminal positivo se conecta a VCC y el terminal negativo a GND. También se debe utilizar una tensión nominal superior a la tensión de trabajo; por ejemplo, para una línea de 5 V resulta adecuado emplear condensadores de 10 V o 16 V.

La bomba de agua se mantiene separada de la electrónica de control desde el punto de vista de potencia. El Arduino no alimenta la bomba, sino que únicamente activa el módulo relé. La corriente de la bomba circula por la línea de 12 V y por los contactos del relé. Esta separación reduce el riesgo de sobrecargar el microcontrolador y permite que la actuación tenga una alimentación adecuada.

Aunque el sistema se valida inicialmente con una fuente o batería de 12 V, la arquitectura deja preparada una evolución hacia alimentación autónoma mediante panel solar. En una versión futura, el panel no alimentaría directamente la estación, sino que cargaría una batería mediante un regulador de carga adecuado. La estación seguiría tomando energía de la batería, manteniendo el LM2596 como etapa de adaptación a 5 V para la electrónica.

Esta futura alimentación solar requeriría dimensionar la batería, el panel y el regulador de carga a partir del consumo real de la estación. En ese cálculo habría que considerar el consumo del Arduino, sensores, módulo LTE, relé y bomba, así como la frecuencia de envío de telemetría y los tiempos de actuación. Por tanto, la integración

solar se plantea como una línea de mejora, no como una funcionalidad completamente validada en esta fase del prototipo.

El resultado de esta etapa es un esquema de alimentación coherente con las necesidades de la estación: entrada principal de 12 V, conversión regulada a 5 V, masas comunes, estabilización próxima al módem LTE y separación entre electrónica de control y carga de potencia. Esta organización permite alimentar sensores, comunicación y actuación manteniendo un criterio de seguridad y reproducibilidad adecuado para el prototipo.

Apéndice A

Materiales y componentes empleados

Este anexo recoge los principales componentes utilizados en el prototipo TierraViva. La finalidad no es presentar una lista comercial de compra, sino documentar los elementos físicos empleados, su función dentro del sistema y su estado de validación durante el desarrollo.

Los componentes se han agrupado según su papel dentro de la arquitectura: estación de campo, comunicaciones, sensorización, actuación, alimentación y centro de control. Esta organización facilita relacionar cada material con los bloques descritos en el cuerpo principal de la memoria.

Componente	Cant.	Función	Estado	Observaciones
Arduino UNO	2	Controlador de estación de campo.	Probado	Lectura de sensores y control local del relé.
A7670E-LASA	2	Comunicación LTE de estación.	Validado por comandos AT	Envío remoto de telemetría y consulta de comandos.
BME280	2	Medición de temperatura, humedad relativa y presión atmosférica.	Probado	Sensor ambiental digital mediante I2C.
BH1750 / GY-302	2	Medición de luminosidad ambiente.	Probado	Sensor digital de luz mediante I2C.
Sensor capacitivo de humedad de suelo V1.2	2	Medición de humedad del sustrato.	Probado parcialmente	Lectura analógica; requiere calibración con suelo seco y húmedo.
Módulo relé	2	Conmutación de la alimentación de la bomba.	Pendiente/probado según fase	Actuación local desde Arduino.
Bomba de agua 12 V	2	Actuación física sobre el riego.	Pendiente/probado según fase	Estado seguro: bomba apagada.
LM2596	2	Regulación de tensión de 12 V a 5 V.	Pendiente/probado según fase	Debe ajustarse a 5,0 V antes de alimentar la electrónica.
Fuente de alimentación 12 V	2	Alimentación principal de estación y bomba.	Pendiente/probado según fase	Base para pruebas de estación en banco.
Raspberry Pi 5	1	Centro de control del sistema.	Probada	Ejecución de SQLite, FastAPI, dashboard y reglas de decisión.
Tarjeta SIM de datos	1-2	Acceso a red móvil para el módulo LTE.	Probada parcialmente	Necesaria para comunicación remota de las estaciones.
Resistencias 1 k Ω y 2 k Ω	Varias	Divisor de tensión para la línea RXD del A7670E.	Pendiente/probado según fase	Adaptación de nivel desde Arduino hacia el módulo LTE.
Protoboard y cableado Dupont	Varias unidades	Montaje y pruebas de conexiones.	Usado en pruebas	Permite modificar el prototipo durante la validación.

La Tabla A.1 recoge los materiales principales del prototipo. El estado indicado no debe interpretarse como una certificación industrial del componente, sino como el grado de validación alcanzado durante el desarrollo del TFG. En algunos casos, como los sensores o el módulo LTE, se han realizado pruebas específicas de funcionamiento antes de su integración completa. En otros, como la bomba de agua o la alimentación final de estación, la validación queda asociada a las fases de actuación y pruebas de integración.

El uso de componentes habituales en prototipado responde a dos criterios. Por un lado, permite reducir el coste y facilitar la sustitución de piezas durante el desarrollo. Por otro, favorece la reproducibilidad del sistema, ya que se trata de módulos disponibles comercialmente y con documentación accesible. Esta decisión es coherente con el alcance académico del proyecto, cuyo objetivo es validar una arquitectura funcional y no desarrollar una solución industrial cerrada.

A.1. Referencias comerciales de compra

Algunos componentes se adquirieron a través de distribuidores generalistas, principalmente para facilitar la disponibilidad y reducir el tiempo de aprovisionamiento. Estas referencias se incluyen únicamente como trazabilidad de compra. Para la justificación técnica se han utilizado, siempre que ha sido posible, hojas de datos o documentación del fabricante del componente.

Componente	Referencia comercial	Uso en el prototipo
Sensor capacitivo de humedad de suelo	AZDelivery, módulo higrómetro capacitivo V1.2 compatible con Arduino.	Lectura analógica de humedad del sustrato.
BME280	Módulo BME280 compatible con alimentación de 5 V.	Medición ambiental: temperatura, humedad relativa y presión.
BH1750 / GY-302	Módulo GY-302 basado en sensor BH1750.	Medición de luminosidad ambiente en lux.

Tabla A.2: Referencias comerciales de sensores empleados

La referencia comercial permite identificar el tipo de módulo utilizado, pero no sustituye a la documentación técnica. Por este motivo, en el cuerpo principal se citan fuentes técnicas específicas para justificar las características principales de cada sensor: documentación de AZ-Delivery para el sensor capacitivo, hoja de datos de Bosch Sensortec para el BME280 y hoja de datos del BH1750FVI para el sensor de luminosidad.

Esquemas eléctricos y conexiones

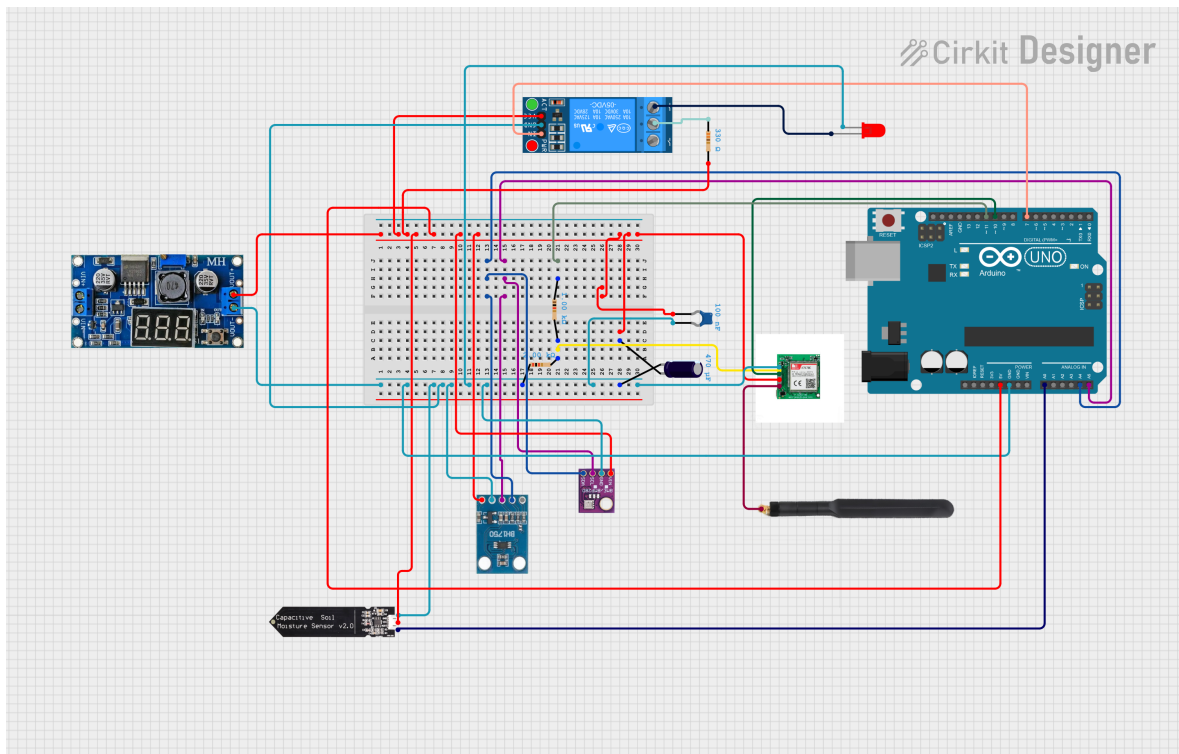


Figura B.1: Esquema de conexión de una estación de campo de TierraViva. En la fase de prueba se representa un LED como carga de verificación; en el montaje final se sustituye por la bomba de agua de 12 V controlada mediante relé.

Apéndice C

Pruebas iniciales de sensores

Apéndice D

Validación del módulo LTE A7670E-LASA

Apéndice E

Fragmentos de código relevantes

Apéndice F

Estructura de canales ThingSpeak y codificación de comandos

Bibliografía

- [Abioye et al., 2020] Abioye, E. A., Abidin, M. S. Z., Mahmud, M. S. A., Buyamin, S., Ishak, M. H. I., Rahman, M. K. I. A., Otuoze, A. O., Onotu, P., and Ramli, M. S. A. (2020). A review on monitoring and advanced control strategies for precision irrigation. *Computers and Electronics in Agriculture*, 173:105441.
- [Adafruit, 2026] Adafruit (2026). Adafruit IO HTTP API. <https://io.adafruit.com/api/docs/>.
- [Adelantado et al., 2017] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., and Watteyne, T. (2017). Understanding the Limits of LoRaWAN. *IEEE Communications Magazine*, 55(9):34–40.
- [Al-Fuqaha et al., 2015] Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., and Ayyash, M. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials*, 17(4):2347–2376.
- [Arduino, 2022] Arduino (2022). Arduino UNO R3 Full Pinout. <https://docs.arduino.cc/resources/pinouts/A000066-full-pinout.pdf>.
- [Arduino, 2026a] Arduino (2026a). Arduino UNO R3 Datasheet. <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>.
- [Arduino, 2026b] Arduino (2026b). Arduino UNO Rev3: Tech Specs. <https://store.arduino.cc/products/arduino-uno-rev3>.
- [Augustin et al., 2016] Augustin, A., Yi, J., Clausen, T., and Townsley, W. M. (2016). A Study of LoRa: Long Range & Low Power Networks for the Internet of Things. *Sensors*, 16(9):1466.
- [AZ-Delivery, 2024] AZ-Delivery (2024). LM2596S Step-Down Module Quick Start Manual. <https://>

- www.manualslib.com/manual/3528199/Az-Delivery-Lm2596s.html. Manual de inicio rápido del módulo reductor LM2596S. Consultado el 22 de mayo de 2026.
- [AZ-Delivery, 2026] AZ-Delivery (2026). Soil Moisture Sensor Hygrometer Module V1.2 Capacitive. <https://www.az-delivery.de/en/products/bodenfeuchte-sensor-modul-v1-2>.
- [Blynk, 2025] Blynk (2025). Datastreams. <https://docs.blynk.io/en/blynk.console/templates/datastreams>.
- [Bosch Sensortec, 2024] Bosch Sensortec (2024). BME280 Datasheet: Combined Humidity and Pressure Sensor. <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>.
- [Boursianis et al., 2021] Boursianis, A. D., Papadopoulou, M. S., Gotsis, A., Wan, S., Sarigiannidis, P., Nikolaidis, S., and Goudos, S. K. (2021). Smart Irrigation System for Precision Agriculture—The AREThOU5A IoT Platform. *IEEE Sensors Journal*, 21(16):17539–17547.
- [Bwambale et al., 2022] Bwambale, E., Abagale, F. K., and Anornu, G. K. (2022). Smart irrigation monitoring and control strategies for improving water use efficiency in precision agriculture: A review. *Agricultural Water Management*, 260:107324.
- [FastAPI, 2026] FastAPI (2026). FastAPI Documentation. <https://fastapi.tiangolo.com/>.
- [Fielding et al., 2022] Fielding, R. T., Nottingham, M., and Reschke, J. F. (2022). RFC 9110: HTTP Semantics. <https://www.rfc-editor.org/rfc/rfc9110.html>. IETF Internet Standard.
- [Food and Agriculture Organization of the United Nations, 2021] Food and Agriculture Organization of the United Nations (2021). AQUASTAT: Water use. <https://www.fao.org/aquastat/en/overview/methodology/water-use/>.
- [García et al., 2020] García, L., Parra, L., Jimenez, J. M., Lloret, J., and Lorenz, P. (2020). Iot-based smart irrigation systems: An overview on the recent trends on sensors and iot systems for irrigation in precision agriculture. *Sensors*, 20(4):1042.
- [Hanes et al., 2017] Hanes, D., Salgueiro, G., Grossetete, P., Barton, R., and Henry, J. (2017). *IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things*. Cisco Press.

- [IEEE, 2020] IEEE (2020). IEEE Standard for an Architectural Framework for the Internet of Things (IoT). IEEE Std 2413-2019.
- [International Society of Precision Agriculture, 2024] International Society of Precision Agriculture (2024). Precision Agriculture Definition. <https://www.ispag.org/resources/definition>.
- [International Telecommunication Union, 2012] International Telecommunication Union (2012). Recommendation ITU-T Y.2060: Overview of the Internet of Things. <https://www.itu.int/rec/T-REC-Y.2060-201206-I/en>. Accessed: 2026-05-20.
- [LoRa Alliance, 2020] LoRa Alliance (2020). LoRaWAN L2 1.0.4 Specification. <https://lora-alliance.org/wp-content/uploads/2021/11/LoRaWAN-Link-Layer-Specification-v1.0.4.pdf>. Technical specification.
- [MathWorks, 2026a] MathWorks (2026a). Read Data from Channel with HTTP GET. <https://www.mathworks.com/help/thingspeak/readdata.html>.
- [MathWorks, 2026b] MathWorks (2026b). ThingSpeak Documentation. <https://www.mathworks.com/help/thingspeak/index.html>.
- [MathWorks, 2026c] MathWorks (2026c). Write Data: Update Channel Data with HTTP GET or POST. <https://www.mathworks.com/help/thingspeak/writedata.html>.
- [Naqvi et al., 2025] Naqvi, S. M. Z. A., Tahir, M. N., Raghavan, V., Awais, M., Hu, J., Said, Y., and Khan, M. I. (2025). Ai-enhanced iot sensors for real-time crop monitoring: an era towards self-monitored agriculture. *Telecommunication Systems*, 88:100.
- [OASIS, 2019] OASIS (2019). MQTT Version 5.0. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. OASIS Standard.
- [Raspberry Pi Ltd, 2026] Raspberry Pi Ltd (2026). Raspberry Pi 5 Product Brief. <https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>.
- [Raza et al., 2017] Raza, U., Kulkarni, P., and Sooriyabandara, M. (2017). Low Power Wide Area Networks: An Overview. *IEEE Communications Surveys & Tutorials*, 19(2):855–873.

- [ROHM Semiconductor, 2011] ROHM Semiconductor (2011). BH1750FVI: Digital 16bit Serial Output Type Ambient Light Sensor IC. <https://www.mouser.com/datasheet/2/348/bh1750fvi-e-186247.pdf>.
- [Semtech Corporation, 2015] Semtech Corporation (2015). AN1200.22 LoRa Modulation Basics. <https://www.semtech.com/uploads/documents/an1200.22.pdf>. Application note.
- [Shafi et al., 2019] Shafi, U., Mumtaz, R., García-Nieto, J., Hassan, S. A., Zaidi, S. A. R., and Iqbal, N. (2019). Precision agriculture techniques and practices: From considerations to applications. *Sensors*, 19(17):3796.
- [Shelby et al., 2014] Shelby, Z., Hartke, K., and Bormann, C. (2014). RFC 7252: The Constrained Application Protocol (CoAP). <https://www.rfc-editor.org/rfc/rfc7252.html>. IETF Proposed Standard.
- [SIMCom Wireless Solutions, 2021] SIMCom Wireless Solutions (2021). A7670 Series Hardware Design. <https://www.simcom.com/product/A7670X.html>. Documentación técnica del fabricante.
- [SIMCom Wireless Solutions, 2024] SIMCom Wireless Solutions (2024). A76XX Series AT Command Manual. https://files.waveshare.com/wiki/A7670E-Cat-1-GNSS-HAT/A76XX_Series_AT_Command_Manual_V2.00.pdf. Manual de comandos AT para la serie A76XX.
- [SIMCom Wireless Solutions, 2026] SIMCom Wireless Solutions (2026). A7670E LTE Cat 1 Module. <https://www.simcom.com/product/A7670E.html>.
- [Soussi et al., 2024] Soussi, A., Zero, E., Sacile, R., Trincherro, D., and Fossa, M. (2024). Smart Sensors and Smart Data for Precision Agriculture: A Review. *Sensors*, 24(8):2647.
- [SQLite, 2026] SQLite (2026). About SQLite. <https://sqlite.org/about.html>.
- [Taha et al., 2025] Taha, M. F., Mao, H., Zhang, Z., Elmasry, G., Awad, M. A., Abdalla, A., Mousa, S., Elwakeel, A. E., and Elsherbiny, O. (2025). Emerging technologies for precision crop management towards agriculture 5.0: A comprehensive overview. *Agriculture*, 15(6):582.

- [Tejedor de la Cruz, 2026] Tejedor de la Cruz, L. (2026). TierraViva: prototipo local MQTT. https://github.com/ltejedorcruz/TierraViva/tree/main/prototipos/prototipo_local_mqtt. Repositorio de código del prototipo local previo.
- [ThingsBoard, 2026] ThingsBoard (2026). Working with telemetry data. <https://thingsboard.io/docs/user-guide/telemetry/>.
- [Ubidots, 2025] Ubidots (2025). Ubidots Basics: Devices, Variables, Dashboards, and Alerts. <https://help.ubidots.com/en/articles/854333-ubidots-basics-devices-variables-dashboards-and-alerts>.
- [Uvicorn, 2026] Uvicorn (2026). Uvicorn Documentation. <https://www.uvicorn.dev/>.
- [Waveshare, 2026] Waveshare (2026). A7670E Cat-1 HAT Wiki. https://www.waveshare.com/wiki/A7670E_Cat-1_HAT. Documentación de placa de evaluación basada en A7670E.